

计算复杂性理论导读

傅育熙

2020 年 10 月 11 日

目录

第 1 章 时间复杂性	4
1.1 图灵机	6
1.2 时间可构造	10
1.3 通用图灵机	11
1.4 对角线方法	16
1.5 丘奇-图灵论题	16
1.6 加速定理	20
1.7 时间复杂性类	24
1.8 非确定图灵机	25
1.9 时间谱系定理	28
1.10 间隙定理	31
第 2 章 难解性	33
2.1 NP-完备性	37
2.2 库克-莱文定理	38
2.3 拉德纳定理	42
2.4 贝克-吉尔-索罗维定理	45
第 3 章 空间复杂性	48
3.1 对数空间类	51
3.2 多项式空间类	53
3.3 伊默尔曼-斯泽勒普森伊定理	55
3.4 时空定理	58

第 4 章	电路复杂性	59
4.1	电路谱系定理	60
4.2	一致电路	62
4.3	P_{poly}	64
4.4	NC 和 AC	65
4.5	P -完备性	68
第 5 章	多项式谱系	70
5.1	多项式谱系	71
5.2	谱系的逻辑刻画	72
5.3	谱系的交替机刻画	74
5.4	无限谱系假设	77
第 6 章	随机计算	79
6.1	尾分布	79
6.2	概率图灵机	79
6.3	PP	79
6.4	BPP	79
6.5	ZPP	79
6.6	随机游走和 RL	79
第 7 章	扩张图与去随机	80
第 8 章	计数复杂性	81
第 9 章	交互证明系统	82
第 10 章	PCP 定理	83

第 1 章 时间复杂性

Diophantine
Hilbert

Matiyasevich

efficient
何谓难？见第2章
Babai
quasi-polynomial
文章未见正式发表

计算理论要回答三个问题。第一个问题是：什么是计算？什么问题可以借助机器求解？著名的丢番图方程要求找出整系数多项式方程 $a_1x_1^{n_1} + a_2x_2^{n_2} + \dots + a_kx_k^{n_k} = 0$ 的整数解。希尔伯特第十问题问：是否存在一个计算过程，判定任给的一个丢番图方程是否有整数解。数学家对什么是计算这个问题颇感兴趣。从机械化的角度看，证明过程是一个寻找满足一定数学和逻辑性质的符号串，读者在理解这个证明时，要进行一个形式化验证过程。数学家的兴趣是，证明过程在多大程度上可以机械化。二十世纪上半叶在数学基础和计算基础领域的研究最终达成了共识：计算是一个独立于任何模型的概念，所有计算模型定义的计算都是等价的。建立在这一共识基础上的可计算理论回答了第一个问题。希尔伯特第十问题最终被年青的苏联数学家马蒂雅谢维奇于 1970 年解决，他证明了希尔伯特第十问题的答案是否定的 [41]。若一个问题可以借助于机器求解，我们总会设法让机器代替人类解决该问题，与人类相比，机器有不可比拟的优势。所以第二个问题是：如何让机器求解一个可计算问题？一台专用设备可以解决某一类特定问题，一台冯诺依曼体系架构的计算机可以通过预置一段程序来解决一个问题。无论是用专用计算设备还是通用计算设备解题，核心是算法。算法理论研究的，正是如何让机器解决问题。尽管人类对算法的兴趣历史悠久，但作为一门理论，系统性的研究和理论突破发生在计算机出现之后。一个著名的例子是素数分解。这是数论中一个古老问题，直到 2004 年，人们才发现这个问题有高效算法 [3]。另一个著名的问题是图同构问题。种种迹象表明，这不是个难问题，但一直没有找到它的高效算法。巴柏在 2016 年发表了图同构问题的一个准多项式时间算法 [4]，被发现了一个错误后，巴柏在几天之后公布了一个更新。这些例子，把我们带到了第三个问题：给定一个计算问题，解决该问题需要多少资源？资源包括时间、空间，但最根本的

资源限制是能量。围棋机器人可以完败人类选手，但在下棋过程中，前者所消耗的能量远大于后者。能源限制是实实在在的。实际应用中，我们关心一个问题是否有可行算法，即它是否有一个多项式时间算法。如果一个可计算问题没有可行解，它就是一个理论上可计算但实际中不可计算的问题，我们得想其它办法。再看一个著名的问题。给定一个图，该图的一个顶点覆盖是一个结点子集，图中的任何一条边都和该结点子集中的某点关联。最小顶点覆盖问题要求计算出一个图的极小的顶点覆盖集。实际中，这就是探头安装问题，我们要用最少的探头，监控到一个楼面的所有走廊。遗憾的是，探头安装问题没有可行解。计算复杂性理论研究如何根据解决问题所消耗的资源量对问题进行复杂性分类。它试图刻画一个问题的绝对复杂性，即给出该问题所需的最小资源，尽管在这方面计算复杂性理论还不太成功。它还希望比较不同问题对资源的消耗量，在这方面计算复杂性理论非常成功。计算复杂性理论的一个核心关注就是可行计算，为了可行性，可以在一定范围内牺牲正确性、精度、完全自动化，甚至可以同时牺牲三者。

feasible algorithm

1988 年 5 月 27 日，一封哥德尔在 1956 年 3 月 20 日写给病中的冯诺依曼的信重见天日。信中，哥德尔本人试图绕过他的不完备定理给数学带来的桎梏。他写道：“...容易构造一台图灵机，对每个一阶谓词公式 F 和每个自然数 n ，判定是否存在一个长度是 n 的 F 的证明。设 $\psi(F, n)$ 是机器计算这个问题的步数，设 $\phi(n) = \max_F \psi(F, n)$ 。问题是，对一个最优化的机器而言， $\phi(n)$ 的增长速度有多快？”如果 $\phi(n)$ 的增长速度是低的，那么“只要取一个足够大的 n ，当机器找不到一个证明时，继续想那个命题就没有任何意义”。的确，倘若世界上最优秀的数学家都无法在一生中理解一个定理的叙述或该定理的证明，又有谁会在乎那个很长的符号串呢！做过严肃数学的人都不愿意相信 $\phi(n)$ 会是一个增长速度缓慢的函数。在读完本书的第二章，读者会确信，哥德尔定义的这个问题是一个 NP-完备问题。所以，那台图灵机不会对数学家有什么帮助。哥德尔不仅是第一位对计算进行形式化研究的那个人，也是第一位提出“NP 是否等于 P?”的那个人。

Gödel

von Neumann

原文及英译见 [50]

长度是指公式的长度加证明的长度

“可行计算就是实际可计算”这一思想贯穿于计算复杂性理论的研究。比如，当我们判断一个串是否是随机串时，我们的标准是看是否存在一个多项式算法将这个串和一个真正意义上的同样长度的随机串区分开来。如果没有任何多项式算法能做出有意义的区分，在实际中那个串就可被当成一个随机串。

基于这一标准的伪随机理论 [67] 在计算机科学中有广泛应用。非对称密码学基于同样的标准。如果必须消耗巨大的资源（时间、能量）才能破译一段密文，比如用蛮力算法，破译者不会花五十年的时间进行破译，届时明文所说的可能早已是公开的秘密。在区块链领域得到很好应用的零知识证明理论里，我们也是假定验证者无法在多项式时间内从交互中获得任何有用信息。另一方面，如果我们必须为某个无可行解的问题设计一个程序解决方案，我们只能放弃一些原则。有时我们不能要求程序在所有的输入上都给出正确的结果，有时我们不得不满足于次优解，有时我们允许程序在计算过程中停下来，让人类导航下一步计算。在一些应用领域，比如机器学习，我们会综合使用这些方法。如果读者熟知当今计算技术的几大应用领域，一定会同意作者的观点：“出错 + 概率 + 交互”是我们这个行业流行的口号。

本书各章节将围绕上述主线展开。

复杂性理论孕育了计算机科学中许多伟大的定理。要想理解这些定理，读者必须对它们的证明有个透彻理解。伟大的思想都在伟大的证明里。计算理论中的证明会用到递归论的、算术的、组合的、代数的、概率的、统计的、图论的方法，对这些方法的熟悉过程也是学习复杂性理论的一部分。

在进入主题之前，有一个决定，我们现在就得做。我们应该基于哪个计算模型来展开复杂性理论的讨论？图灵机模型 [63, 64]。

1.1 图灵机

给定两个自然数 a, b ，我们可以用读小学时学的算法计算 $a + b$ 。如果这两个数很大，我们需要一张纸把这两个数写下来。我们希望这张纸足够大，可以把我们计算过程中得到的中间结果写下来。如果一张纸不够用，我们希望我们有足够多张纸让我们用。写在纸上的内容是我们识别的，所以得有一套事先规定的符号系统，比如阿拉伯数字、算术运算、括号等。最后，我们还得把这个加法计算的结果写在纸上。在做加法运算时，我们要用到加法运算规则，运算的不同阶段会用到不同的规则。这些规则的使用是如此地机械，让我们觉得机器都能做加法运算。图灵在二十世纪三十年代设计的一类机器将加法运算中所涉及的各个方面严格地形式化了。

Turing

Turing machine

一台 k -带图灵机 (TM) M 有 k -条带子。第一条带子称为输入带，用来存放输入数据，输入带是只读带。其余 $k-1$ 条带子是工作带，既可以从工作带

上读信息，也可以往工作带上写内容。图灵机的输出写在最后一条工作带，即第 k 条带子。一条带子被分成了潜在无穷多个格子，每个格子内可以存放一个符号，用 $0, 1, 2, \dots$ 标注这些格子的地址。为了对带子上的内容进行读写，每条带子有一个读写头。在任何时刻，读写头指向带子上的某一格，在该时刻，读写头只能对这一格里的符号进行读或改写。

定义 1.1. 一台 k -带图灵机是一个三元组 (Γ, Q, δ) ，其中

1. Γ 是有限符号集，符号集至少包含 $0, 1, \square, \triangleright$ 这四个符号；
2. Q 是有限状态集，状态集必须包含起始状态 q_{start} 和终止状态 q_{halt} ；
3. $\delta : Q \times \Gamma^k \rightarrow Q \times \Gamma^{k-1} \times \{L, S, R\}^k$ 是迁移函数。

transition function

图灵机运行前，必须对带子进行初始化。初始化包含如下三项：

1. 符号 $\triangleright$ 预置在每条带子的最左端的那个格子里，起到标注带子端点的作用。图灵机不对第 0 格做修改，也不在带子的其它地方写 $\triangleright$ 。
2. 所有工作带的格子里均放着空符号 $\square$ 。空符号表示无用信息。
3. 输入带上除写有输入符号的那些格子之外的格子全部放着空符号 $\square$ 。

图灵机计算始于 q_{start} 状态，止于 q_{halt} 状态。当输入一个 $(k+1)$ -元组 $a_1, \dots, a_k$ ，迁移函数 δ 会给出一个 $2k$ -元组

$$\delta(q, a_1, \dots, a_k) = (q', a'_2, \dots, a'_k, A_1, \dots, A_k). \quad (1.1.1)$$

我们用符号

$$(q, a_1, \dots, a_k) \rightarrow (q', a'_2, \dots, a'_k, A_1, \dots, A_k) \quad (1.1.2)$$

表示等式 (1.1.1)，并称 (1.1.2) 为一条指令。按定义，图灵机只有有限条指令。当图灵机处于状态 q ， k 个读写头所指的格子内依次放着 $a_1, \dots, a_k$ 时，机器可执行指令 (1.1.2)，该指令引发如下操作：将工作带上的符号 $a_2, \dots, a_k$ 分别改写成 $a'_2, \dots, a'_k$ ； k 个读写头分别按 $A_1, \dots, A_k \in \{L, S, R\}$ 所指示的进行移动，即 L 表示向左移一格， R 表示向右移一格， S 表示不移动；机器进入 q' 状态。注意，因为输入带是只读带，所以不需要说明对 a_1 的修改。完成这些操作后，图灵机完成了一步计算。当输入时 $(q_{halt}, a_1, a_2, \dots, a_k)$ ，规定

$\delta(q_{\text{halt}}, a_1, a_2, \dots, a_k) = (q_{\text{halt}}, a_2, \dots, a_k, S, \dots, S)$ 。换言之，当机器进入停机状态 q_{halt} 时，机器不再进行任何计算。

用 $M(x)$ 表示图灵机 M 的输入带预置了输入 x 。 $M(x)$ 会按相关指令计算。在计算的任何时刻 t ， $M(x)$ 的格局 σ_t 是一个 $2k$ -元组 $(q, \kappa_2, \dots, \kappa_k, h_1, \dots, h_k)$ ，其中 q 是时刻 t 时的状态，符号串 $\kappa_2, \dots, \kappa_k$ 分别是时刻 t 时 $k-1$ 条工作带上的内容（非 $\square$ 的符号串），自然数 $h_1, \dots, h_k$ 是时刻 t 时 k 个读写头的位置。格局 σ_t 表示 $M(x)$ 计算了 t 步后的系统参数。 $M(x)$ 的初始格局是 $(q_{\text{start}}, \epsilon, \dots, \epsilon, 0, \dots, 0)$ ，初始格局是唯一的。 $M(x)$ 的终止格局是状态为 q_{halt} 的格局。一步计算可表示成一个格局的迁移 $\sigma_t \rightarrow \sigma_{t+1}$ 。 $M(x)$ 从初始格局开始的格局迁移序列 $\sigma_0 \rightarrow \sigma_1 \rightarrow \dots \rightarrow \sigma_t \rightarrow \dots$ 描述了 $M(x)$ 的计算。若该计算终止，记为 $M(x)\downarrow$ ；若该计算不终止，记为 $M(x)\uparrow$ 。若 $M(x)$ 的计算终止于格局 σ_T ，用 $\sigma_0 \rightarrow \sigma_1 \rightarrow \dots \rightarrow \sigma_T$ 表示其计算路径。

time function

设 M 为图灵机， x 为输入串。若 $M(x)$ 的计算终止，称其计算路径长度为该计算的计算步数或计算时间。通常用时间函数界定图灵机的计算时间。一个时间函数是从自然数到自然数的函数，其输入为串的长度，输出为计算时间的一个上界。我们用 $|x|$ 表示串的长度，比如 $|1^{1024}| = 1024$ 。约定俗成，经常将一个时间函数记为 $T(n)$ ，而非 $T(x)$ ，其中 n 强调是图灵机输入串的长度。

problem

设计图灵机的目的是解决计算问题。在讨论用图灵机求解问题之前，得先对什么是问题做形式化描述。先看一个例子：对带权重的有向图，求总权重最小的哈密尔顿回路的权重。假定权重为非 0 自然数，用输出 0 表示没有哈密尔顿回路。因为问题实例（即所有的带权重的有向图）均为有限对象，可以用 0-1 串进行编码，所有的输出值可以用二进制表示，所以这个问题本质上是一个 0-1 串到 0-1 串的函数。推而广之，一个函数 $f: \{0, 1\}^* \rightarrow \{0, 1\}^*$ 就是一个问题，反之亦然。用“ $M(x) = y$ ”表示“ $M(x)\downarrow$ 且计算终止时输出带上的内容是 y ”，在此种情况下，称 y 为 $M(x)$ 的计算结果。若对任意输入 $x \in \{0, 1\}^*$ ，有 $M(x) = f(x)$ ，称 M 计算了 f ，或 M 解决了问题 f 。一个判定问题是一个特殊的函数 $d: \{0, 1\}^* \rightarrow \{0, 1\}$ ，其值域为布尔集 $\{0, 1\}$ 。若 M 解决 d ，亦称 M 判定 d 。称 $\{0, 1\}^*$ 的一个子集 L 为语言。若图灵机 M 判定 L 的特征函数

L^* 表示 L 中元素的有限串集合

$$L(x) = \begin{cases} 1, & \text{if } x \in L, \\ 0, & \text{if } x \notin L, \end{cases}$$

称 M 接受语言 L 。

看一个大家都熟悉的问题的图灵机解。

例 1.1. 一个 0-1 串回文，指的是从左到右和从右到左看是同一个串。一台判定回文问题的双带图灵机先将输入复制到工作带，然后将输入带的读写头移到最左端，最后两个读写头相向同步移动对所读符号进行比较。下面是此图灵机的指令。

“计算机算计”

$$\begin{aligned} &\langle q_{start}, \triangleright, \square \rangle \rightarrow \langle q_c, \triangleright, R, R \rangle, \\ &\langle q_c, 0, \square \rangle \rightarrow \langle q_c, 0, R, R \rangle, \quad \langle q_c, 1, \square \rangle \rightarrow \langle q_c, 1, R, R \rangle, \quad \langle q_c, \square, \square \rangle \rightarrow \langle q_l, \square, L, S \rangle, \\ &\langle q_l, 0, \square \rangle \rightarrow \langle q_l, \square, L, S \rangle, \quad \langle q_l, 1, \square \rangle \rightarrow \langle q_l, \square, L, S \rangle, \quad \langle q_l, \triangleright, \square \rangle \rightarrow \langle q_t, \square, R, L \rangle, \\ &\langle q_t, 0, 0 \rangle \rightarrow \langle q_t, \square, R, L \rangle, \quad \langle q_t, 1, 1 \rangle \rightarrow \langle q_t, \square, R, L \rangle, \\ &\langle q_t, \square, \triangleright \rangle \rightarrow \langle q_y, \triangleright, S, R \rangle, \quad \langle q_y, \square, \square \rangle \rightarrow \langle q_{halt}, 1, S, S \rangle, \\ &\langle q_t, 0, 1 \rangle \rightarrow \langle q_n, \square, S, L \rangle, \quad \langle q_t, 1, 0 \rangle \rightarrow \langle q_n, \square, S, L \rangle, \\ &\langle q_n, 0, 0 \rangle \rightarrow \langle q_n, \square, S, L \rangle, \quad \langle q_n, 0, 1 \rangle \rightarrow \langle q_n, \square, S, L \rangle, \\ &\langle q_n, 1, 0 \rangle \rightarrow \langle q_n, \square, S, L \rangle, \quad \langle q_n, 1, 1 \rangle \rightarrow \langle q_n, \square, S, L \rangle, \\ &\langle q_n, 0, \triangleright \rangle \rightarrow \langle q_n, \triangleright, S, R \rangle, \quad \langle q_n, 1, \triangleright \rangle \rightarrow \langle q_n, \triangleright, S, R \rangle, \\ &\langle q_n, 0, \square \rangle \rightarrow \langle q_{halt}, 0, S, S \rangle, \quad \langle q_n, 1, \square \rangle \rightarrow \langle q_{halt}, 0, S, S \rangle. \end{aligned}$$

严格说，这个指令集并不完全，它并不唯一确定迁移函数，在有些输入上的值未定，不过这些值可以随意取。

这是本书唯一一次将图灵机的全部指令明确地写出来。在绝大多数情况下，我们将用自然语言和程序语言的混合语言定义图灵机的计算行为。

本书中，当我们在陈述问题时，所有的变量都是十进制算术变量，所有的输出也以十进制表示。如果一个数学表达式的求值结果不是整数，自动向上取整。

解决下述四个问题的图灵机留给读者思考。

1. 一进制长度函数 $u(x) = 1^x$ ，这里 1^x 表示长度为 x 的全 1 串。我们也将把长度为 x 的全 0 串记为 0^x 。
2. 计数器 $s(x) = x + 1$ 。假定输入为 1^n ，计数器初始值为 0。连续做 n 次加一操作，能否在线性时间内完成？
3. 指数函数 $e(x) = 2^x$ 。
4. 对数函数 $l(x) = \log(x)$ 。

我们将“当且仅当”
缩写成“当仅当”
Turing computable
Turing solvable

称一个函数是图灵机可计算的，或该问题是图灵机可解的，当仅当有一台图灵机计算它。一个自然的问题是：哪些 0-1 串到 0-1 串的函数是图灵机可计算的。想必读者知道答案：绝大多数这样的函数是图灵机不可计算的。0-1 串到 0-1 串的函数集是 $\aleph_1$ 的，而图灵机集是 $\aleph_0$ 的。当我们用自然语言和程序语言的混合体定义一台图灵机时，我们要确保我们描述的是一个可计算过程。在 1.5 节我们会再次讨论这件事。

1.2 时间可构造

设 $T(n) : \mathbf{N} \rightarrow \mathbf{N}$ 为时间函数且 $L \subseteq \{0,1\}^*$ 为可判定问题。若存在图灵机 M 和常数 $c > 0$ ，对任意输入 x ， M 在 $cT(|x|)$ 步内接受 L ，则称 L 在 $\mathbf{TIME}(T(n))$ 中。 $\mathbf{TIME}(T(n))$ 是一个问题集合，一个问题在此集合中当仅当该问题在 $T(n)$ 的一个常数倍时间内可判定。一般地， $\mathbf{TIME}(T(n))$ 依赖于所用模型。若模型是所有双带图灵机， $\mathbf{TIME}(n)$ 包含回文问题；若模型是具有单条读写带的图灵机，可以证明，回文问题不在 $\mathbf{TIME}(n)$ 中，事实上，它在 $\mathbf{TIME}(n^2)$ 中。

我们始终假定我们关注的图灵机必须将输入完整地读一遍，这相当于说所有的时间函数 $T(n)$ 都必须满足不等式 $T(n) \geq n$ 。有些时间函数非常怪异，本章最后一节将给出一个例子。为了排除这类函数，我们引入时间可构造性。设 $T(n)$ 为时间函数。

time constructible

定义 1.2. 若有图灵机在 $O(T(n))$ 时间内计算函数 $1^n \mapsto \lfloor T(n) \rfloor$ ，则称 $T(n)$ 是时间可构造的。

可在 $c + \log(n)$ 空间完成 n 次加一操作

假定 M 在输入 1^n 上准确地计算了 $T(n)$ 步后停机。构造图灵机 M' ， M' 有一个初值为 0 的计数器，当它的输入带上写了 1^n 后，按 M 进行计算，每计算一步后，对计数器加一，最后将计数器的值写到输出带上。因为对计数器进行 n 次加一操作需要线性时间，所以 M' 在 $O(T(n))$ 时间内计算函数 $1^n \mapsto \lfloor T(n) \rfloor$ 。此推导表明，下面定义的时间可构造性要强于定义 1.2 描述的时间可构造性。

定义 1.3. 若有图灵机在输入 1^n 上计算 $T(n)$ 步后停机，则称 $T(n)$ 是时间可构造的。

本书中使用的时间函数均为时间可构造的。利用时间可构造函数，可以定义扮演时钟角色的图灵机。设 $M = (Q_0, \Gamma_0, \rightarrow_0)$ 为一行为不规则的 k_0 -带图灵

机，其不同输入上计算时间会差异很大，设 $\mathbb{T} = (Q_1, \Gamma_1, \rightarrow_1)$ 是一台 k_1 -带时钟图灵机。将这两台图灵机复合成一台 (k_0+k_1) -带图灵机，其定义如下：

- $Q = Q_0 \times Q_1$ ，并对所有 $q \in Q_0$ 引入等式 $(q, q_{\text{halt}}^1) = q_{\text{halt}}$ ，同样，对所有 $q \in Q_1$ 引入等式 $(q_{\text{halt}}^0, q) = q_{\text{halt}}$ ；
- $\Gamma = \Gamma_0 \times \Gamma_1$ ；
- $\rightarrow = \rightarrow_0 \times \rightarrow_1$ 。

给定特定输入，复合后的图灵机的计算时间等于 $\mathbb{M}$ 的计算时间和 $\mathbb{T}$ 的计算时间中的那个，其效果相当于用时钟图灵机 $\mathbb{T}$ 对图灵机 $\mathbb{M}$ 的计算掐表，强迫后者在规定时间内停机。称复合后的图灵机为 $\mathbb{M}$ 和 $\mathbb{T}$ 的硬连接。在本书中，每当说“让图灵机 $\mathbb{M}$ 计算 $T(n)$ 步”，我们的意思是 $T(n)$ 是在定义1.3或定义1.2意义下的时间可构造函数，因此有 $T(n)$ -时钟图灵机 $\mathbb{T}$ ，当输入长度是 n 时，计算 $T(n)$ 或 $O(T(n))$ 步停机，通过将 $\mathbb{M}$ 和 $\mathbb{T}$ 硬连接，强迫 $\mathbb{M}$ 在 $T(n)$ 步内终止。

hard-wired

1.3 通用图灵机

图灵机是有限对象，因此可用 0-1 串对其编码。一台图灵机有一个有限状态集、一个有限符号集、一个有限指令集。给出状态和符号的编码后，图灵机的编码就是指令集的编码。若图灵机有七个状态和十五个符号，一条指令 $(p, a, b, c) \rightarrow (q, d, e, R, S, L)$ 可用如下的串编码：

001†1010†1100†0000†011†1111†0000†01†00†10.

一个指令集集可用如下的串进行编码：

$$\ddagger_ \ddagger \dots \ddagger_ \ddagger. \quad (1.3.1)$$

很容易将 (1.3.1) 转换成二进制串，比如可以用如下定义的映射：

$$\begin{aligned} 0 &\mapsto 01, \\ 1 &\mapsto 10, \\ \dagger &\mapsto 00, \end{aligned}$$

$$\ddagger \mapsto 11.$$

这个映射确保图灵机的二进制编码的第一位和最后一位都是 1。我们将固定这个对图灵机的二进制编码。

关于编码的一个事实是：无法设计一个编码系统，使得本质上相同的图灵机有相同的编码。例如，如果我们对状态给了另外一个编码，图灵机的编码就会不同。另外，我们可以给一台图灵机加一些冗余的状态、符号、指令，得到一台“胖”的图灵机。显然有无穷多台“胖”图灵机。一台“胖”图灵机和原来那台“瘦”的图灵机本质上是同一台图灵机，在很多构造和证明里可以互换，但它们的编码不一样。为了避免叙述繁琐，我们将做一个实用主义的假定，即每台图灵机有无穷多个编码。另一方面，我们希望将每个二进制串看成一个图灵机的编码。为了做到这一点，我们只需将所有那些不是图灵机编码的二进制串解释成唯一的那个状态集和符号集均为空集的图灵机的编码。我们用 $\llbracket \mathbb{M} \rrbracket$ 表示图灵机 $\mathbb{M}$ 的二进制编码， $\mathbb{M}_\alpha$ 表示编码为 α 的那台图灵机。

effective
enumeration

将每个十进制数理解成一个二进制 0-1 串，得到图灵机的一个有效枚举：

$$\mathbb{M}_0, \mathbb{M}_1, \dots, \mathbb{M}_i, \dots \quad (1.3.2)$$

index
Gödel encoding

称 i 为图灵机 $\mathbb{M}_i$ 的下标或哥德尔编码。设 ϕ_i 是 $\mathbb{M}_i$ 所计算的函数，由 (1.3.2) 得到图灵机可计算函数的一个枚举

$$\phi_0, \phi_1, \dots, \phi_i, \dots \quad (1.3.3)$$

同样，称 i 为可计算函数 ϕ_i 的下标。正如一个图灵机在 (1.3.2) 中出现无穷多次，每个图灵机可计算函数在 (1.3.3) 中出现无穷多次。需要说明的是，一般地， ϕ_i 是偏函数。称 ϕ_i 在输入 x 上无定义，记为 $\phi_i(x) \uparrow$ ，当仅当 $\mathbb{M}_i(x) \uparrow$ 。类似地， $\phi_i(x) \downarrow$ 当仅当 $\mathbb{M}_i(x) \downarrow$ 。

既然已经选定了一个图灵机编码，我们可以将一个二进制串理解成一个数据或一台图灵机。给定任意两个二进制串 α, x ，我们可以模拟 $\mathbb{M}_\alpha$ 在输入 x 上的计算。直观上这个模拟过程是有效的，所以直觉上有一台图灵机 $\mathbb{U}$ ，满足

$$\mathbb{U}(\alpha, x) \simeq \mathbb{M}_\alpha(x). \quad (1.3.4)$$

universal TM

称 $\mathbb{U}$ 为通用图灵机。在 (1.3.4) 中，等价关系 $\simeq$ 的定义如下：若一边有定义，则另一边也有定义且两边相等；若一边无定义，则另一边也无定义。

定理 1.1 (枚举定理, Enumeration Theorem). 存在通用图灵机 $\mathbb{U}$, 对任意 $\alpha, x \in \{0, 1\}^*$, 等式 $\mathbb{U}(\alpha, x) \simeq \mathbb{M}_\alpha(x)$ 成立。

枚举定理的证明将在定理1.2的证明中一并给出。在复杂性理论中, 我们不仅关心通用图灵机的存在性, 还关心它的模拟效率。下述定理是枚举定理的复杂性版本。

定理 1.2. 存在通用图灵机 $\mathbb{U}$ 和多项式 c . 对任意长度为 n 的输入串 x , 若 $\mathbb{M}_\alpha(x)$ 在 $T(n)$ 步内停机, 则 $\mathbb{U}(\alpha, x)$ 在 $c(|\alpha|)T(n) \log T(n)$ 步内停机。

证明. 关于 $\mathbb{U}$ 要回答的第一个问题是: 它应该有多少条工作带? 因为 $\mathbb{M}_\alpha$ 的工作带数可能大于任意指定的自然数, 一个合理的选择是, $\mathbb{U}$ 用一条工作带存储 $\mathbb{M}_\alpha$ 的所有带子上的内容; 另外, $\mathbb{U}$ 还需要用第二条工作带来存储中间结果和提高数据移动效率。为行文方便, 假定 $\mathbb{U}$ 的带子是双向潜在无限的。 $\mathbb{U}$ 用 0-1 串对 $\mathbb{M}_\alpha$ 的状态和符号进行编码。若 $\mathbb{M}_\alpha$ 有 k 条带子, 我们把 $\mathbb{U}$ 的第一条工作带分成虚拟的 k 层, 每层存放 $\mathbb{M}_\alpha$ 的一条带子。这 k 条带子以 k 个读写头所指那格对齐, 换言之, 我们用 $\mathbb{U}$ 的第一条工作带上的读写头虚拟了 $\mathbb{M}_\alpha$ 的 k 个读写头。 $\mathbb{U}$ 用第一条工作带上的一片区域存放一个 k 元组, k 元组的每个分量是 $\mathbb{M}_\alpha$ 中一个符号的二进制表示, 这片区域的大小依赖于 $\mathbb{M}_\alpha$ 的符号集大小。 $\mathbb{M}_\alpha$ 的 k 条带子上的内容以一串 k -元组的形式存放在 $\mathbb{U}$ 的第一条工作带上。

定义了 $\mathbb{U}$ 使用的数据结构后, 继续定义 $\mathbb{U}$ 如何模拟 $\mathbb{M}_\alpha$ 的计算。模拟 $\mathbb{M}_\alpha$ 读写头的读写是简单的, 模拟读写头的移动也不难。若 $\mathbb{M}_\alpha$ 的一个读写头向左移一格, $\mathbb{U}$ 将第一条工作带上相应的那层整体向右移一格。问题似乎全解决了。但是, 因为对 $\mathbb{M}_\alpha$ 的一步计算涉及到将整个一层的内容向左或向右移动, $\mathbb{U}$ 模拟 $\mathbb{M}_\alpha$ 所需的时间在最坏情况下是 $T(|x|)^2$ 。

我们需要引进更加精细的数据结构来减少因整层移动所带来的时间开销。解决方案是在每一层上引入冗余信息。因为冗余信息可以被有用信息覆盖, 所以在大多数情况下, 我们不需要移动整层内容, 而只需移动其中的一小部分。我们希望冗余信息比较均匀地分布在每一层上, 否则还是无法避免一层内容的大量移动。下面是解决方案:

1. $\mathbb{U}$ 用符号 $\square$ 表示冗余信息。想象 $\square$ 存放在一层中的一个格子里。

2. $\mathbb{U}$ 将第一条工作带分成若干区间。中心区间只包含地址是 0 的那个格子。中心区间的左边和右边分别是 L_0 和 R_0 区间, 各自包含 2 个格子; L_0 的左边是 L_1 区间, R_0 的右边是 R_1 区间, 各自包含 4 个格子; 依此类推, 最左的区间是 $L_{\log T(n)}$, 最右的区间是 $R_{\log T(n)}$, 各自包含 $2 \cdot 2^{\log T(n)}$ 个格子。
3. 任何时候, 中心区间里放的必须是非 $\square$ 的符号, 一个非中心区间的任一层要么是满的 (即不包含 $\square$, 但可以包含被模拟带的空格符号 $\square$), 要么是半满的 (即一半的格子里放的是 $\square$), 要么是空的 (即所有格子里放的都是 $\square$)。 $\mathbb{U}$ 要求 $\square$ 的分布是均匀的: 对任意 $i \in [0, \log T(n)]$, 若 L_i 是满的, 则 R_i 是空的; 若 L_i 是半满的, 则 R_i 是半满的; 若 L_i 是空的, 则 R_i 是满的。

当第一条工作带上的某层向左移动时, 只需从 R_0 开始找到第一个非空区间 R_i , 将其中的 2^i 个非 $\square$ 符号依次移到中心区间和 $R_0, \dots, R_{i-1}$ 中。显然, 移动后区间 $R_0, \dots, R_{i-1}$ 均为半满。为了保持均匀, $\mathbb{U}$ 将 L_{i-1} 中的全部符号 (均为非 $\square$ 符号) 移到 L_i 中, 将 L_{i-2} 中的全部符号移到 L_{i-1} 中, 依此类推, 最后将原来在中心区间里的符号移到 L_0 中; $L_0, \dots, L_{i-1}$ 的其余格子里全部存放 $\square$ 。当 $\mathbb{U}$ 完成了这层移动, 接下来的 $2^i - 1$ 次该层的移动只会涉及 $L_{i-1}, \dots, L_0, R_0, \dots, R_{i-1}$ 和中心区间。因此, 在 $\mathbb{U}$ 的整个模拟过程中, 涉及对区间 $L_i, \dots, L_0, R_0, \dots, R_i$ 的移动最多只有 $k \frac{T}{2^i}$ 次。我们尚需算出在区间 $L_i, \dots, L_0, R_0, \dots, R_i$ 内进行数据移动的时间开销。不难看出, 借助于第二条工作带, 我们可以一边复制一边移动, 所有的数据移动都可在线性时间里完成。因 $L_i, \dots, L_0, R_0, \dots, R_i$ 的格子数不超过 2^{i+3} , 所以 $\mathbb{U}$ 的计算时间是

$$O \left(k \sum_{i=1}^{\log(T(n))} \frac{T(n)}{2^i} 2^i \right) = C \cdot T(n) \log(T(n)).$$

对系数 C 的估算如下: M_α 的状态编码长度、符号编码长度、迁移函数的编码长度、 k 均不超过 $|\alpha|$, 所以每一步模拟所进行的移动开销捕获超过 $|\alpha|$ 的一个多项式。对计数器的操作是 $T(n) \log(T(n))$ 的一个常数倍, 其它额外计算开销也不超过 $T(n) \log(T(n))$ 的一个常数被。所以存在多项式 c , 使得 $C \leq |c(\alpha)|$ 。定理得证。 $\square$

上述证明中的通用图灵机构造出现在亨尼和斯特恩斯的 1966 年文章里 [21]。在哈特马尼斯和斯特恩斯发表于 1965 年的文章中 [20]，通用图灵机的时间复杂性是 $T(n)^2$ 。哈特马尼斯和斯特恩斯的文章是奠基性的，“计算复杂性”这个术语就源自该文。

定理 1.2 可进一步加强。先引入一个概念。若一台图灵机在计算的任意时刻读写头的位置不依赖于输入的内容，而只依赖于输入的长度和该时刻已经进行的计算步数，称其为健忘图灵机。

推论 1.1. 设 $T(n)$ 是时间可构造的， L 可在 $T(n)$ 时间内被一台图灵机判定。一定存在存在多项式 c 和在 $c(|\alpha|)T(n)\log T(n)$ 时间内判定 L 的健忘图灵机。

证明. 设 $\mathbb{M}_\alpha$ 在 $T(n)$ 时间内判定 L 。由定理 1.1 知， $\mathbb{U}$ 在 $c(|\alpha|)T(n)\log(T(n))$ 时间内判定 L ，其中 c 为一多项式。适当修改 $\mathbb{U}$ 的定义，使其成为健忘图灵机。可做如下改动：

1. 因 $T(n)\log(T(n))$ 为时间可构造性，可预先将所有从 $L_{\log T(n)}$ 到 $L_{\log T(n)}$ 的区间置为半满。
2. 定期调整每一层的内容安排。引入一个计数器，用来统计被模拟图灵机的计算步数。每当计数器加一后，总有一位且仅有一位会从 0 变成了 1，若这一位是第 i 位， $\mathbb{U}$ 通过扫描区间 $L_i, \dots, L_0, R_0, \dots, R_i$ 若干遍，将区间 $L_{i-1}, \dots, L_0, R_0, \dots, R_{i-1}$ 均置为半满。

在这些改动下， $\mathbb{U}$ 变成了健忘图灵机。我们必须确保第二个改动是正确的。计数器每计算到 2^{i-1} 步，其第 i 位会从 0 变成 1。若 $L_{i-1}, \dots, L_0, R_0, \dots, R_{i-1}$ 为半满，移动 2^{i-1} 步内所涉及的区间是 $L_i, \dots, L_0, R_0, \dots, R_i$ 。特别要注意的是，当不得不调整 $L_{i-1}, \dots, L_0, R_0, \dots, R_{i-1}$ 时， L_i, R_i 均为半满。 $\square$

下述结果是定理 1.2 的一个简单推论。

推论 1.2. 设 f 可在 $T(n)$ 时间内由一台具有 k 条读写带的图灵机所计算。一定存在一台运行时间为 $O(T(n)\log T(n))$ 的具有两条读写带的图灵机计算 f ，也一定存在一台运行时间为 $O(T(n)^2)$ 的具有一条读写带的图灵机计算 f 。

证明. 将 $\mathbb{U}$ 的第二条工作带和输入带合二为一。如果只允许一条带子，数据移动需要 $O(T(n)^2)$ 的时间。 $\square$

Hennie, Stearns

Hartmanis

因对复杂性理论基础的贡献，哈特马尼斯和斯特恩斯获 1993 年度图灵奖
oblivious

$h(x) = O(h'(x))$ 当且仅当 $h(x)$ 的增长速度不超过 $h'(x)$ 的增长速度

1.4 对角线方法

diagonal method

有了通用图灵机的概念，我们可以解释在递归论和复杂性理论研究中广泛应用的`对角线方法`。`对角线方法`是用来证明否定结果的。作为一个最简单的例子，考察如下定义的判定问题：

$$A(\alpha) = \begin{cases} 0, & \text{if } M_\alpha(\alpha) = 1, \\ 1, & \text{if } M_\alpha(\alpha) \neq 1. \end{cases}$$

假定 A 可由某个图灵机 A 判定。按定义有： $A(\perp A \perp) = 0$ 当且仅当 $A(\perp A \perp) = 1$ 。此矛盾说明我们的假定是错的，即没有任何一台图灵机判定 A 。换言之，函数 A 不是图灵机可判定的。函数 A 的定义用到了 $M_0(0), M_1(1), M_{01}(01), \dots$ 。若将 $(M_i(j))_{i,j}$ 看成一个无限矩阵， $M_0(0), M_1(1), M_{01}(01), \dots$ 处于对角线位置。对角线证明的基本思路是：从假设出发，定义一个图灵机可计算函数，将对角线上的计算结果进行反转，说明该图灵机不同于任何一台图灵机，由此推得矛盾。直观上，当 $M_\alpha(\alpha) \neq 1$ ，我们想要的程序是无法输出 1 的，因为 $M_\alpha(\alpha)$ 的计算停不下来，程序永远无法知道 $M_\alpha(\alpha)$ 的计算下一步会停下来，还是永远停不下来。

利用 A 的图灵机不可判定性，我们可以讨论著名的停机问题。定义函数

$$H(\alpha, x) = \begin{cases} 1, & \text{if } M_\alpha(x) \downarrow, \\ 0, & \text{if } M_\alpha(x) \uparrow. \end{cases}$$

若 H 可由某个图灵机 M_H 判定，下面定义的图灵机就能判定 A 。

$$M_U(\alpha) = \begin{cases} 0, & \text{if } M_H(\alpha, \alpha) = 1 \wedge M_\alpha(\alpha) = 1, \\ 1, & \text{otherwise.} \end{cases}$$

此矛盾证明了 H 也是不可判定的。换言之，不存在一段程序，当输入一段程序 P 和一个输入数据 x ，该程序能判定 $P(x)$ 的计算是否终止。这就是所谓的“停机问题不可解”。在这个推导里，我们使用了归约方法，将问题 A 有效地归约到了问题 H 。

注意，别把归约方向搞反了

1.5 丘奇-图灵论题

我们定义了什么叫图灵机可计算，图灵机可计算的函数都是我们直觉意义上可计算的。对于读者这样的编程高手，将一台图灵机转换成一段高级语言程序是

小菜一碟。让我们感到不好回答的问题是：是否每个直觉意义上可计算的函数都是图灵机可计算的？严格地说，我们无法回答这个问题。“直觉意义上可计算”本身就不是一个形式化的概念，图灵机模型本意就是要给出计算的一个形式化模型。但有很强的证据让我们相信，图灵机差不多能计算所有我们认为是计算的东西。这些证据包括：

1. 没有找到一个直觉上可计算但图灵机不可计算的例子。我们尝试得越多，越觉得不太可能找到。
2. 研究者为了计算设计了很多其它模型，如递归函数 [49]、 λ -演算 [11]、波斯特系统 [47]、马尔可夫算法 [40]、明斯基机器 [42]、无穷寄存器机器 [57]，其中的任何一个模型所计算的 0-1 函数都是图灵机可计算的。

几个著名的计算模型是二十世纪三十年代提出的。让我们花点篇幅回顾一下递归函数模型的提出和完善过程，其中的一些概念在下一节会用到。希尔伯特被誉为是二十世纪最伟大的数学家。在 1930 年德国哥尼斯堡召开的德国科学和医学年会上，希尔伯特用下面这句著名的话重申了他对数学形式化的信念：“Wir müssen wissen. Wir werden wissen”。在希尔伯特讲话的前一天，一位来自维也纳的青年人哥德尔在一个圆桌会议上宣布了他的不完备定理，彻底否定了希尔伯特的宏伟蓝图。在不完备定理的证明中，哥德尔使用了后来成为计算理论核心技术的哥德尔编码。为了进行编码，哥德尔定义了原始递归函数 [19]。该类函数可归纳定义如下：

“我们必须知道，我们终将知道。”

primitive recursive
function

1. 常值函数、后继函数、投影函数是原始递归函数。
2. 若 $f(y_1, \dots, y_k)$ 是 k -元原始递归函数， $g_1(\tilde{x}), \dots, g_k(\tilde{x})$ 是 n -元原始递归函数，那么复合函数

$$f(g_1(\tilde{x}), \dots, g_k(\tilde{x}))$$

是 n -元原始递归函数。这里， $\tilde{x}$ 是对 $x_1, \dots, x_n$ 的缩写。

3. 若 $f(\tilde{x})$ 是 n -元原始递归函数， $g(\tilde{x}, y, z)$ 是 $(n+2)$ -元原始递归函数，则由如下两等式递归定义的函数是 $(n+1)$ -元原始递归函数。

$$\begin{aligned} h(\tilde{x}, 0) &= f(\tilde{x}), \\ h(\tilde{x}, y+1) &= g(\tilde{x}, y, h(\tilde{x}, y)). \end{aligned}$$

Ackermann
Herbrand
哥德尔的 1934 年
讲义可在 [15] 里找到
Kleene

原始递归函数均为全函数。利用对角线方法可以定义出一个直观上可计算，但不等于任何一个原始递归函数的全函数。哥德尔在 1931 年已知晓阿克曼函数，在 1934 年的一个讲座里，哥德尔采纳了埃尔布朗的建议后，将原始递归函数类扩充成了哥德尔-埃尔布朗递归函数类。丘奇的学生克林于 1936 年给出了一个与哥德尔-埃尔布朗递归函数类等价的 μ -递归函数类 [30]，后者可在原始递归函数的基础上引入如下定义的极小化算子得到：

$$\begin{aligned} g(\tilde{x}) &= \mu y R(\tilde{x}, y) \\ &= \begin{cases} \text{the least } y \text{ such that } R(\tilde{x}, y) \text{ holds,} & \text{if there is such a } y, \\ \text{undefined,} & \text{otherwise.} \end{cases} \end{aligned}$$

recursive function

换言之， μ -递归函数类在极小化过程中封闭。鉴于哥德尔-埃尔布朗递归函数类和 μ -递归函数类的等价性，我们将这类函数简称为递归函数。容易证明，所有递归函数都是图灵机可计算的。反方向的包含关系，即所有图灵机可计算函数都是递归函数，由克林证得 [32]。

Church
Rosser

在图灵提出其机器模型之前，丘奇在数学基础研究中提出了 λ -演算 [11]。由于克林和罗瑟悖论发现了 λ -演算作为逻辑系统是不一致的 [34]，从数学基础角度而言 λ -演算是不成功的，但作为程序和编程模型， λ -演算非常成功 [6, 2]。在 λ -演算里，基本的语法对象是如下定义的 λ -项：

$$M := x \mid \lambda x.M \mid M \cdot M',$$

其中 x 为变量， $\lambda x.M$ 为函数项， $M \cdot M'$ 为函数应用项。直观上， $\lambda x.M$ 是形参为 x 的函数。 λ -演算的一步计算由 β -规则和结构化规则所定义， β -规则为：

$$(\lambda x.M) \cdot N \rightarrow M\{N/x\},$$

其中 $M\{N/x\}$ 是将 M 中的变量 x 用 N 替换后所得的项。设

$$\mathbf{Y}_f = (\lambda x.f \cdot (x \cdot x))(\lambda x.f \cdot (x \cdot x)).$$

根据 β -规则有一步计算 $\mathbf{Y}_f \rightarrow f \cdot (\mathbf{Y}_f)$ ，表明 $\mathbf{Y}_f$ 是 f 的一个不动点。在 λ -演算中我们可以定义函数，尽管 λ -演算非常简单，但丘奇 [11] 和克林 [31] 证明了递归函数都是 λ -可定义函数。

之后，图灵证明了图灵机可计算函数和 λ -可定义函数是相同的 [65]。所以在计算函数这件事上，数学模型、程序模型和机器模型是等价的。这一重要的等价性结果让早期的研究者达成一个共识，即所谓的丘奇-图灵论题 [33]。

丘奇-图灵论题. 可计算函数就是图灵机可计算函数。

Church-Turing
Thesis

到目前为止，所有的计算模型，包括量子计算模型，都支持这一论题。丘奇-图灵论题有多种叙述形式，上述陈述中用了图灵机模型，想强调丘奇-图灵论题的物理属性。正如哥德尔和丘奇指出的，图灵机模型是对直觉上可计算概念的一个最简单和最直接的描述，它不依赖于任何形式化系统，强调了计算的物理可实现性。物理可实现性是对计算的最科学的刻画。在此观点下，上述论题可叙述如下：

丘奇-图灵论题. 物理可实现的计算就是图灵机计算。

丘奇-图灵论题不是一个数学定理，它更像一条物理学定律。

丘奇-图灵论题告诉我们，没有必要谈论图灵机可计算或递归函数可计算或 λ -演算可计算，只需谈论可计算，计算这一概念是独立于任何模型的。

丘奇-图灵论题还告诉我们什么是不可计算。建立在丘奇-图灵论题上的递归论 [49] 既研究判定问题的分类，也研究不可判定问题的分类（图林度）。我们简单回顾一下递归论的两个基础性定理，不仅因为第1.6节会用到这些定理，也想借机说明在实际中我们是如何使用丘奇-图灵论题的。

recursion theory

在 (1.3.3) 中，只考虑了一元可计算函数。一般地，我们可以枚举多元可计算函数。设 $\phi_0^m, \phi_1^m, \dots, \phi_i^m, \dots$ 为 m -元可计算函数的一个枚举。S-m-n 定理讨论的是如何将下标也看成是输入变量。

定理 1.3 (S-m-n 定理, S-m-n Theorem). 设 $m, n > 1$ 。存在 $(m+1)$ -元单射原始递归函数 $s_n^m(x, \tilde{x})$ ，该函数满足：对所有 e ，有 $\phi_e^{m+n}(\tilde{x}, \tilde{y}) \simeq \phi_{s_n^m(e, \tilde{x})}^n(\tilde{y})$ 。

证明. 给定 $(m+n)$ -元可计算函数 ϕ_e^{m+n} 的图灵机 M_e ， $(m+1)$ -元函数 $s_n^m(e, \tilde{x})$ 定义如下：给定输入 $\tilde{c} = c_1, \dots, c_m$ ，定义图灵机 M 为：“当输入 $\tilde{d} = d_1, \dots, d_n$ ，将 $\tilde{c}\tilde{d}$ 写在第一条工作带上，然后模拟 ϕ_e^{m+n} 在 $\tilde{c}\tilde{d}$ 上的计算”；将 M 的哥德尔编码定义为 $s_n^m(e, \tilde{c})$ 的值。哥德尔编码过程是原始递归的。单射性可通过添加冗余指令得到保证。 $\square$

下面的定理是克林发现的 [32]。递归论中深刻的结果都要用到此定理。在 λ -演算里，此定理的证明特别简单。

定理 1.4 (递归定理, Recursion Theorem). 设 f 是一元可计算全函数。存在下标 n 满足等式 $\phi_{f(n)} \simeq \phi_n$ 。

证明. 由 S-m-n 定理知，存在原始递归单射全函数 $s(x)$ ，对任意 x ，有

$$\phi_{s(x)}(y) \simeq \begin{cases} \phi_{\phi_x(x)}(y), & \text{if } \phi_x(x) \downarrow, \\ \uparrow, & \text{otherwise.} \end{cases} \quad (1.5.1)$$

定义 (1.5.1) 中，“ $\uparrow$ ”表示“无定义”。设 v 满足等式 $\phi_v(x) = f(s(x))$ 。显然 ϕ_v 为全函数，故 $\phi_v(v) \downarrow$ 。由 (1.5.1) 推出

$$\phi_{s(v)} \simeq \phi_{\phi_v(v)} = \phi_{f(s(v))}.$$

设 $n = s(v)$ ，定理得证。 □

设 $g(z, u, x)$ 为可计算三元函数。根据 S-m-n 定理，存在可计算全函数 $s(z)$ ，满足 $\phi_{s(z)}^2(u, x) \simeq g(z, u, x)$ 。用递归定理知，存在 e 满足

$$\phi_e^2(u, x) = \phi_{s(e)}^2(u, x) \simeq g(e, u, x).$$

这一等式表明，可用可计算函数 $g(e, u, x)$ 定义 $\phi_e^2(u, x)$ ，前提是：在 $g(e, u, x)$ 的定义中， e 是一个形参，而非一个具体的数。

1.6 加速定理

给定一个问题，是否存在一个最好的解该问题的算法？如果“最好”指的是“计算步数最少”，我们想知道的是：该问题是否存在一个计算步数最少的算法？布鲁姆加速定理告诉我们，一般地，这个计算步数最少的算法是不存在的 [8]。布鲁姆加速定理的结论非常强，为了叙述该定理，引入下述定义。

Blum

Blum complexity
measure

定义 1.4. 若如下两条满足，称 $\{(\phi_i, \Phi_i)\}_{i \in \omega}$ 为一个布鲁姆复杂性度量：

1. 函数 Φ_i 与 ϕ_i 的定义域相等，其中 ϕ_i 表示第 i -台图灵机 $\mathbb{M}_i$ 计算的函数。

2. $\Phi_i(x) \leq n$ 是可判定的。

一个布鲁姆复杂性度量例子是布鲁姆时间函数 $\{(\phi_i, time_i)\}_{i \in \omega}$ ，其中的函数值 $time_i(x)$ 表示 $\mathbb{M}_i(x)$ 的最少计算步数。对确定图灵机，计算路径是唯一的，对下一章要引入的非确定图灵机， $\mathbb{M}_i(x)$ 可以有多条计算路径，所以一般地，函数 $time_i$ 可如下定义。

$$time_i(x) = \mu t. (\mathbb{M}_i(x) \text{ terminates in } t \text{ steps}).$$

容易验证下述事实。

引理 1.1. $\{(\phi_i, time_i)\}_{i \in \omega}$ 是布鲁姆复杂性度量。

布鲁姆加速定理的证明是一个典型的递归论证明，其核心部分是下面引理的证明。

引理 1.2. 设 r 为可计算全函数。存在可计算全函数 f ，对任意计算 f 的图灵机 $\mathbb{M}_i$ ，可有效地计算出满足如下条件的图灵机 $\mathbb{M}_j$ ：

1. ϕ_j 是全函数，并且等式 $\phi_j(x) = f(x)$ 几乎处处成立。
2. 不等式 $r(time_j(x)) < time_i(x)$ 几乎处处成立。

证明. 所谓“ $\phi_j(x) = f(x)$ 几乎处处成立”指的是 = 只在有限个输入上不成立。我们要找一个有效过程，即一个可计算全函数 $s'(z)$ ，给定 i ，能算出 $j = s'(i)$ 。我们还要用对角线方法，将不满足不等式 $r(time_{s'(i)}(x)) < time_i(x)$ 的图灵机 $\mathbb{M}_i$ 排除掉。这两个构造相互交织。首先，根据 s-m-n 定理，有原始递归全函数 $s(e, u)$ ，满足

$$\phi_{s(e,u)}(x) \simeq \phi_e^{(2)}(u, x). \quad (1.6.1)$$

为了定义 f ，我们构造一个可计算函数 $g(e, u, x)$ 。由第1.5节的最后一段知，存在下标 e ，满足

$$\phi_e^{(2)}(u, x) \simeq g(e, u, x). \quad (1.6.2)$$

需要用对角线方法构造函数 $g(e, u, x)$ 。对输入 x ，先定义注销集 $C_{e,u,x}$ 。设注销集 $C_{e,u,0}, \dots, C_{e,u,x-1}$ 已被定义。若对所有 $i \in \{u, \dots, x-1\}$ ， $\mathbf{time}_{s(e,i+1)}(x)$

均有定义, 那么 $C_{e,u,x}$ 的定义如下:

$$C_{e,u,x} = \{i \mid u \leq i \leq x-1, \text{time}_i(x) \leq r(\text{time}_{s(e,i+1)}(x))\} \setminus \bigcup_{y < x} C_{e,u,y},$$

否则 $C_{e,u,x}$ 无定义。因假定 $\text{time}_{s(e,i+1)}(x)$ 有定义且 r 是全函数, 故 $\text{time}_i(x) \leq r(\text{time}_{s(e,i+1)}(x))$ 是可判定的, 所以 $C_{e,u,x}$ 是可计算的, 并且对所有 $i \in C_{e,u,x}$, $\phi_i(x)$ 有定义。如果把下标 j 定义为 $s(e, i+1)$, 那么注销集 $C_{e,u,x}$ 包含所有 $[u, x)$ 中不满足引理第二条性质的下标。有了可计算集 $C_{e,u,x}$, 对角化方法告诉我们可将函数 $g(e, u, x)$ 定义如下:

$$g(e, u, x) \simeq \begin{cases} 1 + \max\{\phi_i(x) \mid i \in C_{e,u,x}\}, & \text{若 } C_{e,u,x} \downarrow, \\ \uparrow, & \text{若 } C_{e,u,x} \uparrow. \end{cases}$$

因为 $C_{e,u,x}$ 是可计算的, 所以 $g(e, u, x)$ 是可计算的。函数 $g(e, u, x)$ 有若干好的性质, 分别讨论如下。

1. 首先, $g(e, u, x)$ 及其关联的函数 $\phi_e^{(2)}(u, x)$ 和 $\phi_{s(e,u)}(x)$ 均为全函数。此性质可用向下归纳法证明如下: 设对所有 $y < x$, $g(e, u, y)$ 均有定义。若 $u \geq x$, 按定义有 $C_{e,u,x} = \emptyset$, 因而 $g(e, u, x) = 1$ 。若 $u < x$, 且 $g(e, x, x), \dots, g(e, u+1, x)$ 有定义, 由 (1.6.1) 和 (1.6.2) 知, $\phi_{s(e,x)}(x), \dots, \phi_{s(e,u+1)}(x)$ 均有定义。由此推出 $\text{time}_{s(e,x)}(x), \dots, \text{time}_{s(e,u+1)}(x)$ 均有定义。所以注销集 $C_{e,u,x}$ 有定义, 因而 $g(e, u, x)$ 有定义。这就完成了向下归纳。
2. 存在 v , 对所有 $x > v$, 有 $\phi_e^{(2)}(0, x) = \phi_e^{(2)}(u, x)$ 。此性质证明如下: 对任意 $i < u$, 若 $i \in C_{e,u,y}$, 则对所有 $x > y$, 有 $i \notin C_{e,u,x}$, 这是因为一个下标最多只能出现在一个注销集中。定义

$$v = \max\{y \mid C_{e,0,y} \text{ contains an index } i < u\}.$$

下标 v 的直观意思是: 当 v 足够大时, 在区间 $[0, u)$ 内终将被注销的下标均已被注销, 这些下标不会再出现在 $C_{e,u,v}$ 中。因此, 等式 $C_{e,0,x} = C_{e,u,x}$ 对所有 $x > v$ 都成立。

3. 若下标 i 满足 $\phi_i(x) = \phi_e^{(2)}(0, x)$, 则 $r(\text{time}_{s(e,i+1)}(x)) < \text{time}_i(x)$ 几乎处处成立。否则不等式 $\text{time}_i(x) \leq r(\text{time}_{s(e,i+1)}(x))$ 在无穷多个输入 x

上成立。按全函数 g 的定义, 必有 $x > i$ 使得

$$\phi_i(x) \neq g(e, 0, x) = \phi_e^{(2)}(0, x).$$

这和假定矛盾。

由上述三个性质, 加上等式 (1.6.1) 和 (1.6.2), 易见, 若定义 $f(x) = \phi_e^{(2)}(0, x)$, 引理的两条性质均成立。□

对上述证明稍加修改, 就能证明布鲁姆加速定理。

定理 1.5 (加速定理, Speedup Theorem). 设 r 为可计算全函数。存在可计算全函数 f , 对任意计算 f 的图灵机 $\mathbb{M}_i$, 存在计算 f 的图灵机 $\mathbb{M}_j$, 使得 $r(\text{time}_j(x)) < \text{time}_i(x)$ 几乎处处成立。

证明. 不失一般性, 设 r 为增函数。将引理1.2证明中的 $C_{e,u,x}$ 的定义稍做修改, 即将 $\text{time}_i(x) \leq r(\text{time}_{s(e,i+1)}(x))$ 改为 $\text{time}_i(x) \leq r(\text{time}_{s(e,i+1)}(x) + x)$, 就能有效地计算出 $\mathbb{M}_k$, 使得下述两性质成立:

1. ϕ_k 是全函数, 并且等式 $\phi_k(x) = f(x)$ 几乎处处成立。
2. 不等式 $r(\text{time}_k(x) + x) < \text{time}_i(x)$ 几乎处处成立。

根据性质 1, 一定存在自然数 N 使得对所有 $x \geq N$, 等式 $\phi_k(x) = f(x)$ 成立。对 $\mathbb{M}_k$ 做如下改动: 将有限关系 $\{(0, f(0)), (1, f(1)), \dots, (N-1, f(N-1))\}$ 所表示的输入输出行为用图灵机的指令实现, 用程序语言的话说, 就是增加一条 case 语句处理所有在 $[0, N)$ 中的输入。将修改后得到的图灵机记为 $\mathbb{M}_j$, 这台图灵机计算 f 。引入 case 语句可能会增加计算时间。但因为 N 是固定的, 当输入不超过 N 时其计算时间不超过一个常数, 而当输入大于 N 时, 其计算时间就是比较输入 x 和 N 大小的时间, 即线性时间。所以当输入 x 大于一定值时, $\mathbb{M}_j$ 的计算时间不超过 $\text{time}_k(|x|) + |x|$ 。□

布鲁姆加速定理是复杂性理论的基础性定理之一, 它传达了如下重要的信息: 我们不应该定义问题的复杂性, 但我们总是可以定义解的复杂性。在复杂性理论里, 我们通过对问题的解进行分类来研究问题的分类。

在布鲁姆证明其加速定理之前, 哈特马尼斯和斯特恩斯于 1965 年证明了所谓的线性加速定理 [20]。

定理 1.6 (线性加速, Linear Speedup Theorem). 设图灵机 M 在 $T(n)$ 步内判定 L 。对任意 $\epsilon > 0$, 存在图灵机 M' , M' 能在 $\epsilon T(n) + n + 2$ 步内判定 L 。

证明. 事实上, M' 可有效地从 $M = (Q, \Gamma, \delta)$ 构造出来。一个简单的想法是: $M' = (Q', \Gamma', \delta')$ 的符号集要包含 Γ 和笛卡尔集 Γ^m ; 换言之, 长度是 m 的 Γ^* 中的符号串可以用 Γ' 中的一个符号进行编码。 M' 的计算定义如下:

- M' 用 $n + 2$ 步将输入转换成长度是 n/m 的内部编码。
- M' 用 n/m 步将读写头移到第 0 格。
- 设 M' 的下一步要模拟 M 的 m 步计算, 后者的一条带子上的读写头停在被 M' 用一个符号编码的连续 m 格中的某一格。在 m 步计算中, 读写头可能跑到了 m 格的左边或右边。所以 M' 在模拟时, 也得看左右格的内容, 然后决定内容修改和读写头的下一步位置。在最坏情况下, M' 用 5 步模拟 M 的 m 步。

显然, M' 的总计算时间不超过 $n + 2 + \frac{n}{m} + \frac{5}{m}T(n) \leq n + 2 + \frac{6}{m}T(n)$ 。设 $m = 6/\epsilon$, 定理得证。 $\square$

1.7 时间复杂性类

一个复杂性类是一个模型无关的问题类。在这个意义下, $\mathbf{TIME}(n)$ 不是一个复杂性类。最著名的一个复杂性类是多项式时间类, 其定义如下:

$$\mathbf{P} = \bigcup_{c \geq 1} \mathbf{TIME}(n^c).$$

$\mathbf{P}$ 的定义不依赖于任何模型。现有模型都是可以在多项式时间内相互模拟的, 见文献 [22, 23]。这一现象支持了强丘奇-图灵论题 [16]。

Strong Church-
Turing Thesis

强丘奇-图灵论题. 图灵机可模拟任何物理可实现的计算装置, 其计算时间不会超过被模拟装置计算时间的一个多项式值。

读者会立刻想到量子计算模型 [43]。到目前为止, 我们既不能确信量子计算模型是否物理可实现, 也没有一个量子计算模型可在多项式时间内解但图灵机模型不能在多项式时间内解的例子。

在计算机科学里，我们将 $\mathbf{P}$ 等同于有高效算法的问题或有可行算法的问题。这一观点不是没有争议的，本书无意介入那些哲学讨论，只想提出一个观点：强丘奇-图灵论题可以反驳对 $\mathbf{P}$ 的所有质疑。

用同样的方法，我们可以定义其它复杂性类，比如指数时间复杂性类：

$$\mathbf{EXP} = \bigcup_{c \geq 1} \mathbf{TIME}(2^{n^c}).$$

注意，不能将 $\mathbf{TIME}(2^{n^c})$ 换成 $\mathbf{TIME}(2^{cn})$ 。若我们有个时间函数为 $\Theta(2^{cn})$ 的算法 $\mathbf{A}$ 和一个时间函数是 $\Theta(n^3)$ 的多项式算法 $\mathbf{B}$ ，复合算法 $\mathbf{A} \circ \mathbf{B}$ 也是一个指数算法，无论常数 c 多大， $\mathbf{A} \circ \mathbf{B}$ 都不在 $\mathbf{TIME}(2^{cn})$ 里。

$f(x) = \Theta(g(x))$ 当且仅当 $f(x)$ 和 $g(x)$ 增长得一样快

1.8 非确定图灵机

图灵机的一个重要特征是物理可实现性。本节介绍的非确定图灵机不是物理可实现的，但这类图灵机在理论研究中扮演着非常重要的角色，特别是当我们研究判定问题的时候。

一台非确定图灵机 $\mathbf{N}$ 可以简单地定义为具有两个迁移函数 δ_0, δ_1 和一个额外状态 q_{accept} 的图灵机。非确定图灵机在运行的任意时刻，可以不确定地选择迁移函数 δ_0 或迁移函数 δ_1 进行计算。我们可以把一台非确定图灵机在一个给定输入上的所有可能的计算路径想象成一棵二叉树，二叉树的叶子节点是状态为 q_{halt} 或 q_{accept} 的格局，它的一次计算或者是树中从根节点到某个叶子的路径，或者是一条不终止路径，至于选择哪条路径完全是不确定的。之所以要引入状态 q_{accept} ，是因为需要指明某条计算路径终止时，机器接受了输入串。一台非确定图灵机不能通过在输出带上写 1 表示接受，在输出带上写 0 表示拒绝，因为这台机器可能在某条计算路径上输出 1，在另一条计算路径上输出 0。基于同样的理由，我们并不引进一个拒绝状态 q_{reject} 。一台非确定图灵机的一次运行要么接受输入串，要么不接受输入串，但不会拒绝输入串。

定义 1.5. 设 $\mathbf{N}$ 为非确定图灵机，当输入 x 时，若 $\mathbf{N}(x)$ 有一条计算路径终止于接受状态 q_{accept} ，称 $\mathbf{N}$ 接受 x ，记为 $\mathbf{N}(x) = 1$ 。

注意， $\mathbf{N}(x)$ 的其它计算路径可能都终止于停机状态 q_{halt} ，但只要有一条计算路径终止于接受状态，就称 $\mathbf{N}$ 接受 x 。只有当 $\mathbf{N}(x)$ 的所有计算路径都

终止于停机状态 q_{halt} 时, 才称 N 拒绝 x 。设 L 为一语言, 若 $x \in L$ 当且仅当 $N(x) = 1$, 称 N 接受 L 。

非确定图灵机强于确定图灵机的地方在于它们的猜测能力。我们用顶点覆盖问题来解释什么是猜测。设 (G, N) 为顶点覆盖问题的一个输入实例, 其中 $G = (V, E)$ 。用一个长度是 $|V|$ 的 0-1 串表示 V 的一个子集, 一台非确定图灵机用它的两个迁移函数不确定地产生一个长度是 $|V|$ 的 0-1 串, 验证该 0-1 串所对应的 V 的子集大小是否为 N , 若是, 进一步验证该子集是否是一个顶点覆盖。在此描述中, “不确定地产生一个长度是 $|V|$ 的 0-1 串”的过程就是“猜测 V 的一个子集”的过程。显然, 所有 V 的子集都能被猜到。若输入图的确含有一个大小是 N 的顶点覆盖, 一定会有一条计算路径成功地猜测到该覆盖, 并终止于接受状态。

设 $T : \mathbf{N} \rightarrow \mathbf{N}$ 为时间函数。对任意输入 x , 若非确定图灵机 N 的任一计算路径的长度都不超过 $T(|x|)$, 称 $T(n)$ 为 N 的时间函数。设 $L \subseteq \{0, 1\}^*$ 。记号 $L \in \mathbf{NTIME}(T(n))$ 表示存在接受 L 的非确定图灵机 N 和常数 $c > 0$, $cT(n)$ 为 N 的时间函数。用此记号, 可定义不确定复杂性类, 比如:

$$\mathbf{NP} = \bigcup_{c \geq 1} \mathbf{NTIME}(n^c),$$

$$\mathbf{NEXP} = \bigcup_{c \geq 1} \mathbf{NTIME}(2^{n^c}).$$

$\mathbf{NP}$ 就是著名的“不确定多项式时间类”。与之相关的问题, 即 “ $\mathbf{NP} \stackrel{?}{=} \mathbf{P}$ ”, 是计算机科学中最重要的基础性问题。

定理 1.7. $\mathbf{P} \subseteq \mathbf{NP} \subseteq \mathbf{EXP} \subseteq \mathbf{NEXP}$ 。

证明. 第一和第三个子集包含关系成立, 因为图灵机是非确定图灵机的特例。第二个包含关系成立的理由是: 在指数时间, 一台图灵机可以把一台多项式时间的非确定图灵机的所有计算路径都走一遍。□

非确定图灵机均为有限对象, 我们可以定义它们的哥德尔编码。固定一个编码和一个 $\{0, 1\}^*$ 与自然数集 $\mathbf{N}$ 之间的有效一一对应, 我们可以有效地枚举所有的非确定图灵机

$$N_0, N_1, \dots, N_i, \dots \quad (1.8.1)$$

序列 (1.8.1) 可以由一台确定的图灵机枚举。借用确定图灵机的研究路径，我们下一个该问的问题是：是否存在通用非确定图灵机，该图灵机可以模拟任一台非确定图灵机的计算？稍做思考就会发现，确定图灵机的通用图灵机构造可以移植到非确定图灵机上，通用非确定图灵机在模拟输入机器的一步之前，不确定地选被模拟图灵机的 δ_0 或 δ_1 。我们要介绍的是一个并非完美但额外的时间开销非常小的“通用”非确定图灵机。为定义此图灵机，引入如下概念。

定义 1.6. 设 N 为 k -带非确定图灵机， x 为输入。 $N(x)$ 在第 t 步的快照为 $k+1$ 元组

$$\langle q, a_1, \dots, a_k \rangle \in Q \times \underbrace{\Gamma \times \dots \times \Gamma}_k,$$

其中， q 为 t 时刻的机器状态， $a_1, \dots, a_k$ 为 t 时刻的读写头所指格中的符号。

与格局不同，快照的大小只依赖于 M ，不依赖于输入长度。 $N(x)$ 的一条计算路径诱导出一个快照序列。由于快照不依赖于任何输入，所以算法可以不用看输入就可以猜测一个快照序列。

本书中用到的“通用”非确定图灵机 V 定义如下：

1. 输入非确定图灵机编码 α 和 0-1 串 x 。
2. V 猜测 $N_\alpha(x)$ 的一个终止于接受状态的计算路径的快照序列和一个同等长度的 0-1 串，后者表示 N_α 在计算时做的不确定选择，0 表示选 δ_0 ，1 表示选 δ_1 。在猜测快照序列时， V 只跟踪输入带上的符号而忽略工作带，工作带上的符号是猜出来的。
3. 对每一条工作带， V 验证在猜测阶段对该条工作带上猜测的符号是否正确。 V 使用另一条工作带来记录 N_α 在该工作带上的实际读写过程。
4. 若所有验证均成功， V 接受，若发现错误， V 停机。

若 V 的所有验证均通过，那么它对工作带上的所有符号的猜测都正确无误。换言之，所猜测的快照序列的确反映了 $N_\alpha(x)$ 的一个终止于接受状态的计算路径。与确定图灵机的通用机相比， V 的最大优势是它几乎没有额外的时间开销，能在输入机器计算时间的一个常数倍里模拟输入机器的运行。 V 的缺点是，它不一定终止，没有任何方法能够阻止它在猜测阶段不停地猜测。在实际

使用 $\forall$ 时，通常是用它来模拟一类具有某个时间函数的非确定图灵机，在此情况下，我们可以利用时间可构造性来叫停一个超时的模拟过程。

1.9 时间谱系定理

给定时间函数 f, g ，若 $f \leq g$ ，则必有 $\mathbf{TIME}(f(n)) \subseteq \mathbf{TIME}(g(n))$ 。我们感兴趣的是，在什么情况下，包含关系 $\mathbf{TIME}(f(n)) \subseteq \mathbf{TIME}(g(n))$ 是严格的。哈特马尼斯和斯特恩斯很早就研究了这个问题。下述定理出现在他们那篇著名的 1965 年文章里 [20]。

定理 1.8 (时间谱系定理, Time Hierarchy Theorem). 若 f, g 为时间可构造函数且 $f(n) \log f(n) = o(g(n))$ ，则 $\mathbf{TIME}(f(n)) \subsetneq \mathbf{TIME}(g(n))$ 。

$f(x) = o(g(x))$ 当
仅当 $f(x)$ 的增长速
度严格小于 $g(x)$ 的
增长速度

证明. 我们要证的是一个否定结果，即 $\mathbf{TIME}(g(n)) \not\subseteq \mathbf{TIME}(f(n))$ 。设 L 由如下定义的图灵机 $\mathbb{D}$ 所判定：“当输入 x 时，模拟 $\mathbb{M}_x(x)$ 计算 $g(|x|)$ 步；若 $\mathbb{M}_x(x)$ 在 $g(|x|)$ 步内完成模拟，输出 $\overline{\mathbb{M}_x(x)}$ ，否则输出 0”。此定义中，“模拟 $\mathbb{M}_x(x)$ 计算 $g(|x|)$ 步”指的是：将通用图灵机 $\mathbb{U}$ 和 $g(n)$ -时钟图灵机做硬连接，得到图灵机 $\mathbb{U}_g$ ，再计算 $\mathbb{U}_g(x)$ 。按定义， $L \in \mathbf{TIME}(g(n))$ 。假设 $L \in \mathbf{TIME}(f(n))$ ，并设 L 由图灵机 $\mathbb{M}_z$ 在 $f(n)$ 时间内判定。根据定理的前提 $f(n) \log f(n) = o(g(n))$ ，只要取 z 足够大， $g(|z|)$ 就远大于 $f(|z|) \log f(|z|)$ 。根据定理 1.2，只要 $g(|z|)$ 远大于 $f(|z|) \log f(|z|)$ ，通用图灵机就能在 $g(|z|)$ 时间内模拟完 $\mathbb{M}_z(z)$ 的计算。因此有下述两结论：

1. $\mathbb{D}(z) = \mathbb{M}_z(z)$ ，这是因为 $\mathbb{D}$ 和 $\mathbb{M}_z$ 都判定语言 L 。
2. $\mathbb{D}(z) = \overline{\mathbb{M}_z(z)}$ ，这是因为 $\mathbb{D}(z)$ 完成了对 $\mathbb{M}_z(z)$ 的模拟并将结果反转。

上述矛盾表明， $L \in \mathbf{TIME}(f(n))$ 不可能为真。 $\square$

作为时间谱系定理的一个简单应用，如下定义的无穷多个复杂性类构成一个严格包含序列：

$$\begin{aligned} \mathbf{EXP} &= \bigcup_{c>1} \mathbf{TIME}(2^{n^c}) \\ 2\mathbf{EXP} &= \bigcup_{c>1} \mathbf{TIME}(2^{2^{n^c}}) \end{aligned}$$

$$\begin{aligned}
3\text{EXP} &= \bigcup_{c>1} \text{TIME}(2^{2^{n^c}}) \\
&\vdots \\
\text{ELEMENTARY} &= \text{EXP} \cup 2\text{EXP} \cup 3\text{EXP} \cup \dots
\end{aligned}$$

复杂性类 **ELEMENTARY** 就是通常所说的初等布尔函数类。一个判定问题在 **ELEMENTARY** 里当且仅当该问题可由一个时间函数是形如 $2^{\dots^{2^{n^c}}}$ 的图灵机判定, 这里 c 和 d 都是常数。显然, 任何一个初等函数的增长速度都小于函数 $2^{\dots^{2^n}}$ 的增长速度。文献里, 称后者为一个塔函数。近年的研究表明, 很多自然问题的时间函数远高于塔函数 [58]。

tower function

上世纪七十年代初, 库克研究了非确定图灵机的谱系问题 [13], 证明了如果 $1 \leq r(x) < r'(x)$, 那么必有 $\text{NTIME}(n^{r(n)}) \subsetneq \text{NTIME}(n^{r'(n)})$ 。几年之后, 塞弗斯、费舍尔和迈耶[56] 证明了一个弱得多的条件 $f(n+1) = o(g(n))$ 就能蕴含 $\text{NTIME}(f(n)) \subsetneq \text{NTIME}(g(n))$ 。又过了几年, 扎克给出了塞弗斯-费舍尔-迈耶定理的一个简单证明 [69]。本书中, 我们介绍扎克的证明。

Cook

Seiferas, Fischer

Meyer

Zák

定理 1.9 (非确定时间谱系定理). 若 f, g 为时间可构造, 且 $f(n+1) = o(g(n))$, 则 $\text{NTIME}(f(n)) \subsetneq \text{NTIME}(g(n))$ 。

证明. 扎克设计了一个精致的非确定图灵机 $\mathbb{Z}$, 其定义如下:

1. $\mathbb{Z}$ 的输入带上的读写头和第一条工作带上的读写头同步全速向右移动。
 - 若输入长度是 1 或输入包含一个 0, $\mathbb{Z}$ 停机。
 - 第一条工作带上的读写头在第一格上写 1, 然后在大部分时间写 0, 但偶尔会写 1。设 $h_0, h_1, h_2, \dots$ 是第一条工作带上读写头写 1 的那些格子的地址, 这些地址由第二条工作带上的操作决定。
2. 在第二条工作带上, $\mathbb{Z}$ 枚举所有的非确定图灵机和一个固定的 $g(n)$ -时钟图灵机的硬连接。设 $\mathbb{L}_1, \mathbb{L}_2, \dots$ 为这些硬连接的一个有效枚举。当 $\mathbb{Z}$ 在第二条工作带上枚举完 $\mathbb{L}_i$ 后, 依次做如下操作:
 - (a) 暂停输入带和第一条工作的带读写头的同步全速向右移动。

(b) $\mathbb{Z}$ 将编码 $\mathbb{L}_{i-1}$ 从第二条工作带上拷贝到第三条工作带, 利用第一条工作带上的标记将输入 $1^{h_{i-1}+1}$ 在第三条工作带上构造出来。

(c) $\mathbb{Z}$ 将第一条工作带和第二条工作带上的读写头移到之前的位置, 然后恢复输入带和第一条工作的带读写头的同步全速向右移动。

$\mathbb{Z}$ 用暴力法计算 $\mathbb{L}_{i-1}(1^{h_{i-1}+1})$ 。计算完 $\mathbb{L}_{i-1}(1^{h_{i-1}+1})$, $\mathbb{Z}$ 将结果写在第二条工作带上, 与此同时, 在第一条工作带上写 1, 这个写了 1 的格子的地址就是 h_i 。

3. 设输入是 1^n , 其中 $n > 1$ 。当 $\mathbb{Z}$ 扫描完输入时, 做如下操作:

(a) 若 $n = h_i$, $\mathbb{Z}$ 接受 1^n 当且仅当 $\mathbb{L}_{i-1}(1^{h_{i-1}+1}) = 0$ 。

(b) 若 $h_{i-1} < n < h_i$, $\mathbb{Z}$ 非确定地模拟 $\mathbb{L}_{i-1}(1^{n+1})$ 的计算 $g(n)$ 步。

设 $\mathbb{Z}$ 所接受的语言是 L 。第 1 步的计算时间是线性的。在第 2 步里, 因为算法对每个 $\mathbb{L}_{i-1}$ 只拷贝一次, 所以是线性时间的; 构造 $1^{h_{i-1}+1}$ 是线性时间的, 因为 $\mathbb{L}_{i-1}$ 必须将输入扫描一遍。在第 3 步计算, 因为 $\mathbb{Z}$ 刚把 $\mathbb{L}_{i-1}(1^{h_{i-1}+1})$ 写在了第二条工作带上, 所以 (3a) 步几乎没有时间开销。因为“通用”非确定图灵机可做到线性时间模拟, (3b) 步的时间开销是 $O(g(n))$ 。因此 $L \in \mathbf{NTIME}(g(n))$ 。另一方面, $L \notin \mathbf{NTIME}(f(n))$ 。假定某非确定图灵机 $\mathbb{L}_i$ 在 $f(n)$ 时间内接受 L 。取 i 足够大使 (3b) 步中定义的模拟能够完成。可导出如下矛盾:

$$\mathbb{L}_i(1^{h_i+1}) = \mathbb{Z}(1^{h_i+1}) \quad (1.9.1)$$

$$= \mathbb{L}_i(1^{h_i+2}) \quad (1.9.2)$$

$$= \mathbb{Z}(1^{h_i+2}) \quad (1.9.3)$$

$$= \dots$$

$$= \mathbb{L}_i(1^{h_i+1}) \quad (1.9.4)$$

$$= \mathbb{Z}(1^{h_i+1}) \quad (1.9.5)$$

$$\neq \mathbb{L}_i(1^{h_i+1}). \quad (1.9.6)$$

上述推导里, (1.9.1)、(1.9.3)、...、(1.9.5) 是因为 $\mathbb{Z}$ 和 $\mathbb{L}_i$ 均接受 L , (1.9.2)、...、(1.9.4) 是因为 (3b) 步中定义的模拟能够完成, (1.9.6) 归因于 (3a) 步中定义的反转。证毕。 $\square$

定理1.9的证明所用的对角化方法称为迟对角化方法。非确定图灵机 $\mathbb{Z}$ 反转的是其在一个对数长输入 $1^{h_{i-1}+1}$ 上的值，这样才能用暴力法计算出该值。为确保反转有效， $\mathbb{Z}$ 错位模拟，它模拟 $\mathbb{L}_{i-1}(1^{h_{i-1}+1})$ 的计算而非 $\mathbb{L}_{i-1}(1^{h_{i-1}})$ 的计算。

lazy

在结束本节之前，考虑如下的问题：根据时间谱系定理（定理1.8），有

$$\mathbf{TIME}(n^c) \subsetneq \mathbf{TIME}(2^{n^c}).$$

那么是否对所有可计算全函数 $b(x)$ ，均有

$$\mathbf{TIME}(b(n)) \subsetneq \mathbf{TIME}(2^{b(n)})?$$

下一节将给出这个问题的答案。

1.10 间隙定理

间隙定理是又一个让人感到惊讶的结果。这个定理最早由苏联的特拉克滕布罗特宣布 [61]，八年后加拿大的鲍罗丁发表了同样的结果 [9]。间隙定理告诉我们，若不对时间函数做任何限制的话，即使多花指数多的时间也不可能多解决一个问题。

Trakhtenbrot

Borodin

定理 1.10 (间隙定理, Gap Theorem). 设 $r(x) \geq x$ 为可计算全函数。存在可计算全函数 $b(x)$ 使得等式 $\mathbf{TIME}(b(x)) = \mathbf{TIME}(r(b(x)))$ 成立。

证明. 定义 $k_0 < k_1 < k_2 < \dots < k_x$ 如下：

$$\begin{aligned} k_0 &= 0, \\ k_{i+1} &= r(k_i) + 1, \text{ for } i < x. \end{aligned}$$

这些数定义了 $x+1$ 个连续的区间 $[k_0, r(k_0)]$, $[k_1, r(k_1)]$, ..., $[k_x, r(k_x)]$ ，这些区间构成了 $[0, r(k_x)]$ 的一个有效划分。用 $P(i, k)$ 表示如下的可判定性质：“对任意 $j \leq i$ 和任意满足 $|z| = i$ 的输入 z ，或 $\mathbb{M}_j(z)$ 在 k 步内停机，或 $\mathbb{M}_j(z)$ 在 $r(k)$ 步内不停机”。设 $n_i = \sum_{j=0}^i |\Gamma_j|^i$ 。易见， n_i 表示 $\mathbb{M}_0, \dots, \mathbb{M}_i$ 中所有长度是 i 的符号串的总数。将这 n_i 个符号串分别作为 $\mathbb{M}_0, \dots, \mathbb{M}_i$ 中相应的图灵机的输入，得到的计算结果不会超过 n_i 个。在 $n_i + 1$ 个区间 $[k_0, r(k_0)]$, ...,

$[k_{n_i}, r(k_{n_i})]$ 中, 至少有一个不包含这些结果中的任何一个结果。因此, 存在一个最小的 $j \leq n_i$ 使得 $P(i, k_j)$ 为真。定义 $b(i) = k_j$, 则对所有 i , $P(i, b(i))$ 为真。

假设 $\mathbb{M}_g$ 在 $r(b(n))$ 步内接收语言 L 。按定义有性质: “对任意满足 $|x| \geq g$ 的输入 x , 或者 $\mathbb{M}_g(x)$ 在 $b(|x|)$ 步内停机, 或者 $\mathbb{M}_g(x)$ 在 $r(b(|x|))$ 步内不停机”。由假定知, 若 x 足够大, $\mathbb{M}_g(x)$ 必在 $r(b(n))$ 步内停机, 根据函数 $b(x)$ 的定义, 这必定意味着 $\mathbb{M}_g(|x|)$ 在 $b(|x|)$ 步内停机。定理得证。 $\square$

间隙定理的结论显然和时间谱系定理的相矛盾。因此, 上述证明里定义的函数 $b(x)$ 不是时间可构造的。

第 2 章 难解性

二十世纪五六十年代，苏联有一批做控制论的学者和他们的学生从控制论、算法复杂性和计算复杂性的角度研究了搜索算法 [62]。他们的研究让他们相信，很多搜索问题没有比直截了当的暴力算法（俄语称为 perebor 算法）好的算法。苏联的控制论学者研究了很多实际搜索问题的算法，包括布尔函数的最小电路实现问题的复杂性，取得了两个具有里程碑意义的结果。一个突破发生在七十年代初期，柯尔莫果洛夫的学生莱文证明了存在一类他称之为通用 perebor 问题，所有的搜索问题都可归约到这类问题，换言之，存在最难的搜索问题。另一个方面，在七十年代末期，哈奇扬在其同事工作的基础上证明了线性规划问题有多项式算法 [29]，在复杂性理论意义上，找到了一个最容易的非常实际的搜索问题。1971 年，莱文在柯尔莫果洛夫研讨会和马尔科夫研讨会上报告了他的结果，正式的文章只有两页，以莱文标志性的超凝练形式发表于 1973 年 [36]。在同一时期，库克定义了 NP-完备问题并证明了可满足性问题是 NP-完备的，文章发表于 1971 年 [12]。莱文的通用 perebor 问题就是库克的 NP-完备问题的搜索版本或构造版本，后者可看成是前者的判定版本或存在版本。本质上莱文和库克引入了同一个概念，找到了同一个完备问题。紧接着，卡普证明了 21 个熟知的组合问题是 NP-完备的 [27]，复杂性理论进入了 NP 时代。1979 年，在加里和约翰逊写的 NP 理论的专著中，作者介绍了当时知道的数百个 NP-完备问题 [18]。NP 完备问题的普遍性促使人们寻找这类问题的可行解。在所有的尝试失败后，人们试图去证明 $\mathbf{NP} \neq \mathbf{P}$ 。如今，我们知道的有关“ $\mathbf{NP} \stackrel{?}{=} \mathbf{P}$ ”的最好的结果都有如下形式的陈述“某某方法不可能解决这个问题” [5, 48, 1]。一部分计算机科学家相信 $\mathbf{NP} \neq \mathbf{P}$ 独立于现有数学体系。少数人认为 $\mathbf{NP} \neq \mathbf{P}$ 是一条物理学定律，毕竟，可行计算描述的也是一类物理过程。如果我们相信丘奇-图灵论题是物理学定律的话，计算复杂性

cybernetics

brute force

Kolmogorov
Levin

Khachijan

Markov

Garey, Johnson

理论的研究似乎在暗示，从这条定性的定律推不出相应的定量结果。

NP 问题类是复杂性理论中最本质的概念。对 NP-完备问题的研究从好几个方面推动了复杂性理论乃至整个计算机科学的发展。在计算机科学之外，NP-完备性的影响力渗透到科学、数学、工程的广泛领域。在实际应用中，NP-完备性理论提供了一个快速验证问题难易性的方法。如果一个问题 是 NP-难的，就不必费心去找可行解，而应该专注于想其它办法。

没有人真正理解 **NP** 的内在复杂性和结构，但一个合格的计算机科学家应能大致判断一个问题是否是 NP-难的，就像一个合格的计算机科学能凭直觉判断一个问题是否可计算。本章介绍 NP 理论的基本内容。在讨论之前，我们将判定 NP 问题的“猜测-然后验证”之方法做进一步的形式化。

定理 2.1. 语言 $L \subseteq \{0,1\}^*$ 在 **NP** 里当且仅当存在多项式 $p: \mathbb{N} \rightarrow \mathbb{N}$ 和多项式时间图灵机 $\mathbb{M}$ 满足：对任意 $x \in \{0,1\}^*$ ，下述等价关系成立：

$$x \in L \text{ 当且仅当 } \exists u \in \{0,1\}^{p(|x|)}. \mathbb{M}(x, u) = 1.$$

verifier

certificate

称 $\mathbb{M}$ 为 L 的验证器。若 x 和 u 满足 $|u| = p(|x|)$ 和 $\mathbb{M}(x, u) = 1$ ，称 u 为 $x \in L$ 的证书。

证明. 设非确定图灵机 $\mathbb{N}$ 在多项式时间 $p(n)$ 接受 L 。按时间可构造性，可假定对任意输入串 x ， $\mathbb{N}(x)$ 的所有计算路径长度均为 $p(|x|)$ 。用通用图灵机对 $\mathbb{N}(x)$ 的计算进行模拟，模拟时需要一长度为 $p(|x|)$ 的 0-1 串，该 0-1 串表示 $\mathbb{N}$ 计算时对迁移函数的选择。反之，一个非确定图灵机可以猜测一个证书然后验证。 $\square$

定理2.1可解释为：**NP** 是所有多项式时间可验证的问题。而 **P** 是所有多项式时间可解决的问题。解决问题时需要寻找解，验证时则不需要任何寻找过程，直觉上，前者的难度要远大于后者。我们将 **NP** 中的问题称为 NP-问题，将 **P** 中的问题称为 P-问题。

验证的视角让我们较容易证明一个问题是 NP-问题。看几个例子。

independent set

- 给定图 G 和自然数 N ，独立集问题 **IS** 问：是否存在 G 的 N 个结点，这些结点之间没有边？一个验证算法猜测一个结点集，然后验证该集合是否具有 N 个结点以及是否构成一个独立集。

- 给定图 $G = (V, E)$, 旅行商问题 **TSP** 问: G 是否包含一个简单圈, 即 G 是由有一个长度是 $|E|$ 的包含每个结点的圈? 一个验证算法猜测 $|E|$ 条边, 然后验证这些边是否构成一个简单圈。
- 给定图 G_0, G_1 , 图同构问题 **GI** 问: 是否存在 G_0 和 G_1 之间的一个同构态射? 一个验证算法猜测结点之间的一个一一对应, 然后验证其是否给出一个同构。

graph isomorphism

除直接验证外, 还可通过归约证明一个问题是 NP-问题。

定义 2.1. 设 $L, L' \subseteq \{0, 1\}^*$. 若存在一个多项式时间可计算函数 $r: \{0, 1\}^* \rightarrow \{0, 1\}^*$ 使得对任意 $x \in \{0, 1\}^*$, $x \in L$ 当且仅当 $r(x) \in L'$, 则称 L 可卡普归约到 L' , 记为 $L \leq_K L'$.

Karp reduction

若 $L \leq_K L'$, 从 L' 的一个算法可以构造出 L 的一个算法, 若 L' 有可行解, L 必有可行解。换言之, L 的难度不会超过 L' 的难度。卡普归约提供了一个强有力的比较问题难易程度的工具。作为一个比较标准, 归约必须具有传递性。下述引理的验证留给读者。

引理 2.1. 若 $L \leq_K L' \leq_K L''$, 则 $L \leq_K L''$ 。

归约是计算机科学的一个基本工具, 多看一些具体的归约例子很有好处。当我们希望证明某个问题 L 在 **NP** 里, 我们可以将该问题归约到 **NP** 中的一个已知问题 L' 。将多项式时间归约函数 $L \leq_K L'$ 和 L' 的多项式时间验证算法复合后得到 L 的多项式时间验证算法, 这就证明了 $L \in \mathbf{NP}$ 。即将给出的两个归约实例涉及那个最著名的 NP-问题。一个合取范式是一个如下形式的命题公式

conjunctive normal form

$$\bigwedge_i \left(\bigvee_j v_{i_j} \right),$$

其中 v_{i_j} 为字, 析取式 $\bigvee_j v_{i_j}$ 为语句。一个字或为命题变量 x , 或为命题变量的否定形式 $\bar{x}$ 。在一个 k -合取范式里, 每个语句最多含有 k 个字。一个含有 n 个变量的命题公式 φ 是可满足的当且仅当存在一个满足 φ 的真值指派 $\rho: \{0, 1\}^n \rightarrow \{0, 1\}$ 。著名的可满足问题定义为所有可满足合取范式的集合, 记为 **SAT**。**SAT** 有个简单的非确定算法: 先猜一个真值指派, 然后验证。因

literal, clause

Satisfiable

Satisfiability

此, SAT 是 NP-问题。将 SAT 中的 k -合取范式构成的集合记为 $k\text{SAT}$ 。显然, $k\text{SAT} \leq_K \text{SAT}$ 。当 $k \geq 3$, 反方向的归约也成立。

引理 2.2. $\text{SAT} \leq_K 3\text{SAT}$ 。

证明. 语句 $x_1 \vee x_2 \vee \dots \vee x_k$ 的一个真值指派可扩展到语句 $(x_1 \vee x_2 \vee z_1) \wedge (\bar{z}_1 \vee x_3 \vee z_2) \wedge \dots \wedge (\bar{z}_{k-4} \vee x_{k-2} \vee z_{k-3}) \wedge (\bar{z}_{k-3} \vee x_{k-1} \vee x_k)$ 的一个真值指派, 其中 $\{z_1, \dots, z_{k-3}\}$ 和 $\{x_1, \dots, x_k\}$ 不相交, 反之, 后者的一个可满足真值指派也是前者的可满足真值指派。归约过程显然是多项式的。□

上述归约表明 3SAT 和 SAT 的难度是一样的。2SAT 的难度如何? 下面的命题给出了答案。

命题 1. $2\text{SAT} \in \mathbf{P}$ 。

证明. 给定含 k 个变量 $x_1, \dots, x_k$ 的 2-合取范式 φ , 构造结点数为 $2k$ 的有向图, 用 $x_1, \dots, x_k, \bar{x}_1, \dots, \bar{x}_k$ 表示结点。若 φ 含有语句 $v \vee v'$, 在图中添加边 $\bar{v} \rightarrow v'$ 和边 $\bar{v}' \rightarrow v$ 。欲证明: φ 不可满足当且仅当, 存在某个字 v , $v \rightarrow^* \bar{v}$ 且 $\bar{v} \rightarrow^* v$ 。先证“ $\Leftarrow$ ”方向: 若对 v 赋真值, $v \rightarrow^* \bar{v}$ 强迫对 v 赋假值; 若对 v 赋假值, $\bar{v} \rightarrow^* v$ 强迫对 v 赋真值。换言之, φ 蕴含假, 即 φ 不可满足。再证“ $\Rightarrow$ ”方向: 假定不存在任何字 v , 同时有 $v \rightarrow^* \bar{v}$ 且 $\bar{v} \rightarrow^* v$ 。用归纳法构造一真值指派。假定 v 尚未被赋值, 并且没有 $v \rightarrow^* \bar{v}$ 。对所有 $v \rightarrow^* w$, 给 w 赋真值并给 $\bar{w}$ 赋假值。注意两点: 一、 w 不可能已被赋假值, 否者因为 $\bar{w} \rightarrow^* \bar{v}$, 按归纳 v 应已被赋假值; 二、不能有 $v \rightarrow^* \bar{w}$, 否者就有 $v \rightarrow^* w \rightarrow^* \bar{v}$, 和假设矛盾。此赋值为良定义, 所有的字都会被赋值, 所有语句的值为真。

$\rightarrow^*$ 表示零步或多步可达

上述构造将 2SAT 卡普归约到了图的不可达性问题。图的可达性问题有多项式算法, 因此 2SAT 有多项式算法。□

第二个归约将可满足问题归约到独立集问题。

命题 2. $3\text{SAT} \leq_K \text{IS}$ 。

证明. 从 3SAT 到独立集问题 IS 的卡普归约定义如下: 设 3-合取范式 φ 含有 m 个语句、 n 个变量, 将 φ 映射到如下定义的有 $7m$ 个结点的图 G_φ :

一个团中的任意两点有边相连

- 从 φ 的每条语句构造一个含 7 个结点的团, 每个结点对应一个使该语句为真的对三个变量的真值指派。

- 属于不同团的两结点有一条边相连当且仅当所代表的真值指派有冲突。

易见, φ 有一个满足 m' 个语句的真值指派当且仅当 G_φ 有一个大小是 m' 的独立集。所需的卡普归约将 φ 映射到 (G_φ, m) 。 $\square$

2.1 NP-完备性

我们知道 $\mathbf{P} \subseteq \mathbf{NP}$, 但不知道是否 $\mathbf{P} = \mathbf{NP}$ 。如果我们想了解 $\mathbf{NP}$ 的内部结构, 首先应该刻画出 $\mathbf{NP}$ 中最难的那类问题。我们已经知道如何用卡普归约比较问题的难易, 所以下面的定义应该是自然的。

定义 2.2. 若对任意 $L \in \mathbf{NP}$, 均有 $L \leq_K L'$, 称 L' 为 *NP-难的*。若 L' 是 *NP-难的* 且 $L' \in \mathbf{NP}$, 称 L' 是 *NP-完备的*。

NP-hard

NP-complete

按定义, 若一个 NP-完备问题有一个多项式时间算法, 任何一个 NP-问题都有多项式算法。NP-完备问题就是最难的 NP-问题。 $\mathbf{NP} = \mathbf{P}$ 当且仅当一个 NP-完备问题可以卡普归约到一个 P-问题。

是否真的存在一个 NP-完备问题? 事实上, 可以构造一台通用线性时间验证器, 验证下述语言。

$$\mathbf{TMNP} = \{\langle \alpha, x, 1^n, 1^t \rangle \mid \exists u \in \{0, 1\}^n. \mathbb{M}_\alpha(x, u) \text{ outputs } 1 \text{ in } t \text{ steps}\}.$$

易见, $\mathbf{TMNP} \in \mathbf{NP}$ 。从定义可看出, $\mathbf{TMNP}$ 是包含所有 NP-语言的 NP-语言。文献中也称 $\mathbf{TMNP}$ 为有界停机问题。

bounded halting
problem

定理 2.2. $\mathbf{TMNP}$ 是 NP-完备的。

证明. 设 $L \in \mathbf{NP}$ 有一个多项式 $q(n)$ 时间验证器 $\mathbb{M}$, 证书的长度是多项式值 $p(n)$ 。易见, “ $x \in L$ ” 当且仅当 “ $\exists u \in \{0, 1\}^{p(|x|)}. \mathbb{M}(x, u)$ 在 $q(|x|+p(|x|))$ 步内输出 1” 当且仅当 “ $\langle \perp \mathbb{M} \perp, x, 1^{p(|x|)}, 1^{q(|x|+p(|x|))} \rangle \in \mathbf{TMNP}$ ”。我们所需要的从 L 到 $\mathbf{TMNP}$ 的卡普归约将输入串 x 映射到四元组 $\langle \perp \mathbb{M} \perp, x, 1^{p(|x|)}, 1^{q(|x|+p(|x|))} \rangle$ 。 $\square$

尽管 $\mathbf{TMNP}$ 是个人为的问题, 但它是实实在在存在的。设 $\mathbf{NPC}$ 为所有 NP-完备问题类。根据卡普归约的传递性, 欲证 $L \in \mathbf{NPC}$, 只需证 $\mathbf{TMNP} \leq_K L \in \mathbf{NP}$ 。我们用 $\mathbf{TMNP}$ 的一个变种来说明此方法。在推论 1.1 的证明里, 我们构造了一台健忘通用图灵机 $\mathbb{U}$, 其时间开销会有个对数放大。定义

$$\mathbf{TMNP}_\mathbb{U} = \{\langle \alpha, x, 1^n, 1^t \rangle \mid \exists u \in \{0, 1\}^n. \mathbb{U}(\alpha, x, u) \text{ outputs } 1 \text{ in } t \text{ steps}\}.$$

显然, $\text{TMNP}_{\cup} \in \mathbf{NP}$ 且 $\text{TMNP}_{\cup} \leq_K \text{TMNP}$ 。定义函数

$$r(z) = \begin{cases} \langle \alpha, x, 1^n, 1^{c(|\alpha|)t \log t} \rangle, & \text{if } z = \langle \alpha, x, 1^n, 1^t \rangle, \\ z, & \text{otherwise,} \end{cases}$$

其中 c 函数就是推论1.1中给出的函数。根据推论1.1不难看出, r 是从 TMNP 到 $\text{TMNP}_{\cup}$ 的卡普归约。

2.2 库克-莱文定理

库克和莱文找到了第一个自然的 NP-完备问题。这就是下面的定理 [12, 36]。

定理 2.3 (库克-莱文定理). SAT 是 NP-完备的。

证明. 证明前先做点准备。首先, $(u_1 = v_1) \wedge \dots \wedge (u_n = v_n)$ 可定义为合取范式 $(u_1 \vee \overline{v_1}) \wedge (\overline{u_1} \vee v_1) \wedge \dots \wedge (u_n \vee \overline{v_n}) \wedge (\overline{u_n} \vee v_n)$ 。此公式可简写为 $\tilde{u} = \tilde{v}$ 。若 $v_1 = 1$, $(u_1 \vee \overline{v_1}) \wedge (\overline{u_1} \vee v_1)$ 可简化为 u_1 ; 若 $v_1 = 0$, $(u_1 \vee \overline{v_1}) \wedge (\overline{u_1} \vee v_1)$ 可简化为 $\overline{u_1}$ 。其次, 对任意布尔函数 $f: \{0, 1\}^\ell \rightarrow \{0, 1\}$, 有包含的字的个数不超过 $\ell 2^\ell$ 的合取范式 φ_f , 满足 $\forall u \in \{0, 1\}^\ell. \varphi_f(u) = f(u)$ 。合取范式 φ_f 的定义如下:

$$\bigwedge_{v \in \{0, 1\}^\ell \wedge f(v)=0} \overline{(u=v)}, \quad (2.2.1)$$

显然, $\overline{(v=v)} = 0$ 且 $\forall u \neq v. \overline{(u=v)} = 1$, 所以 (2.2.1) 的确定义了 f 。因为 $u=v$ 是 ℓ 个字的合取, 所以 $\overline{(u=v)}$ 是 ℓ 个字的析取, 因此 (2.2.1) 是合取范式。易见, (2.2.1) 所出现的字的个数不超过 $\ell 2^\ell$ 。若 ℓ 是常数, (2.2.1) 定义的公式是多项式长的。

根据定理2.2, 只需证 $\text{TMNP} \leq \text{SAT}$ 。设 $\mathbb{M} = (\Gamma, Q, \delta)$ 是 TMNP 的多项式时间验证器, 即存在多项式 $p(n)$ 使得对任意 $x \in \{0, 1\}^*$ 有: $x \in \text{TMNP}$ 当且仅当 $\exists u \in \{0, 1\}^{p(|x|)}. \mathbb{M}(x, u) = 1$ 。根据时间可构造, 可假定 $\mathbb{M}(x, u)$ 准确地计算 $T(n)$ 步后停机, 这里 $T(n)$ 是多项式。根据定理1.1, 可假定 $\mathbb{M}$ 是健忘的。不失一般性, 假定 $\mathbb{M}$ 只有一条读写带, 在计算过程中输入 x, u 不会被改写。进一步, 我们假定当机器进入接受 x, u 时, 读写头在第 0 格位置, 带子上的内容为 $xu1\Box \dots \Box$ 。因此接受格局是唯一的。

设 $|x| = n$ 。我们用布尔变量的值对第 0 格到第 $T(n)$ 格的符号进行编码, 用变量 $\tilde{y}_q = y_1, \dots, y_{\log |Q|}$ 的值对状态进行编码。为叙述方便, 假定用

$\tilde{z} = z_0, z_1, \dots, z_{T(n)}$ 描述带子上的内容, 这里每个 z_i 应该看成是 $\log |\Gamma|$ 个变量。因为有 $T(n) + 1$ 个格局, 我们用 $\tilde{z}^i = z_0^i, z_1^i, \dots, z_{T(n)}^i$ 描述第 i 个格局带子上的内容, 用 $\tilde{y}^i = y_1^i, \dots, y_{\log |Q|}^i$ 描述第 i 个格局机器的状态。总共有多项式个变量。定义 $\psi_x(u)$ 为如下合取范式的合取:

1. ψ_0 为 $(\tilde{z}^0 = xu \square \dots \square) \wedge (\tilde{y}^0 = q_{\text{start}})$ 。为了避免引入繁琐的符号, 在逻辑公式里, 我们就用 $\square$ 表示符号 $\square$ 的二进制编码, 用 q_{start} 表示状态 q_{start} 的二进制编码。下同。
2. 对任意 $i \in \{0, \dots, T(n)\}$, $\psi_{i \Rightarrow i+1}$ 表示第 i 个格局和第 $i+1$ 个格局的关系, 是两个合取范式的合取。设在第 i 步读写头在第 h 格。第一部分表示 $z_h^i, \tilde{y}^i$ 和 $z_h^{i+1}, \tilde{y}^{i+1}$ 的函数关系。根据 (2.2.1) 所在段的论证, 这部分公式是多项式长的。第二部分为公式 $(z_0^i = z_0^{i+1}) \wedge \dots \wedge (z_{h-1}^i = z_{h-1}^{i+1}) \wedge (z_{h+1}^i = z_{h+1}^{i+1}) \wedge \dots \wedge (z_{T(n)}^i = z_{T(n)}^{i+1})$ 。注意, 因为 $\mathbb{M}$ 是健忘的, 我们可以模拟 $\mathbb{M}$ 在输入 1^n 上的计算, 在多项式时间内将 h 算出来。因此 $\psi_{i \Rightarrow i+1}$ 可在多项式时间内构造出来。
3. $\psi_{T(n)}$ 为 $(\tilde{z}^{T(n)} = xu1\square \dots \square) \wedge (\tilde{y}^{T(n)} = q_{\text{halt}})$ 。

定义 $\psi_x(u) = \psi_0 \wedge \left(\bigwedge_{0 \leq i \leq T(n)-1} \psi_{i \Rightarrow i+1} \right) \wedge \psi_{T(n)}$ 。此公式是多项式长的, 并且可在多项式时间里构造出来。公式 $\psi_x(u)$ 中有很多变量, 但因为 $\mathbb{M}$ 是确定图灵机并且 $\psi_x(u)$ 反映了 $\mathbb{M}(x, u)$ 的计算, 一旦对 u 的真值指派确定了, 对其它变量的真值指派就完全确定了。不难看出, 一证书 u 满足 $\mathbb{M}(x, u) = 1$ 当且仅当作 u 确定了唯一一个满足 $\psi_x(u)$ 的真值指派。所以将 x 映射到 $\psi_x(u)$ 的函数是从 TMNP 到 SAT 的卡普归约。□

上述证明里构造的卡普归约 $\psi_{_}$ 有良好的性质。设 $Ctf(x)$ 为满足 $\mathbb{M}(x, u) = 1$ 的证书集合, $Tas(\psi_x(u))$ 为满足 $\psi_x(u) = 1$ 的证书集合, 这里满足 $\psi_x(u) = 1$ 的证书就是使 $\psi_x(u) = 1$ 为真的真值指派。归约 $\psi_{_}$ 是从 $Ctf(x)$ 到 $Tas(\psi_x(u))$ 的双射, 其逆函数也是多项式时间可计算的。称满足这些性质的多项式时间归约为莱文归约。莱文归约对搜索问题之间的归约是重要的。莱文性质在复杂性理论的一些证明里起着很关键的作用。我们用一个例子来说明如何使用莱文性质。假定 **NP** 问题有多项式算法。给定一个旅行商问题的输入实例, 在多项式时间内将其归约到一合取范式 $\tau(x_1, \dots, x_n)$ 。调用 SAT 问题的多项式算法

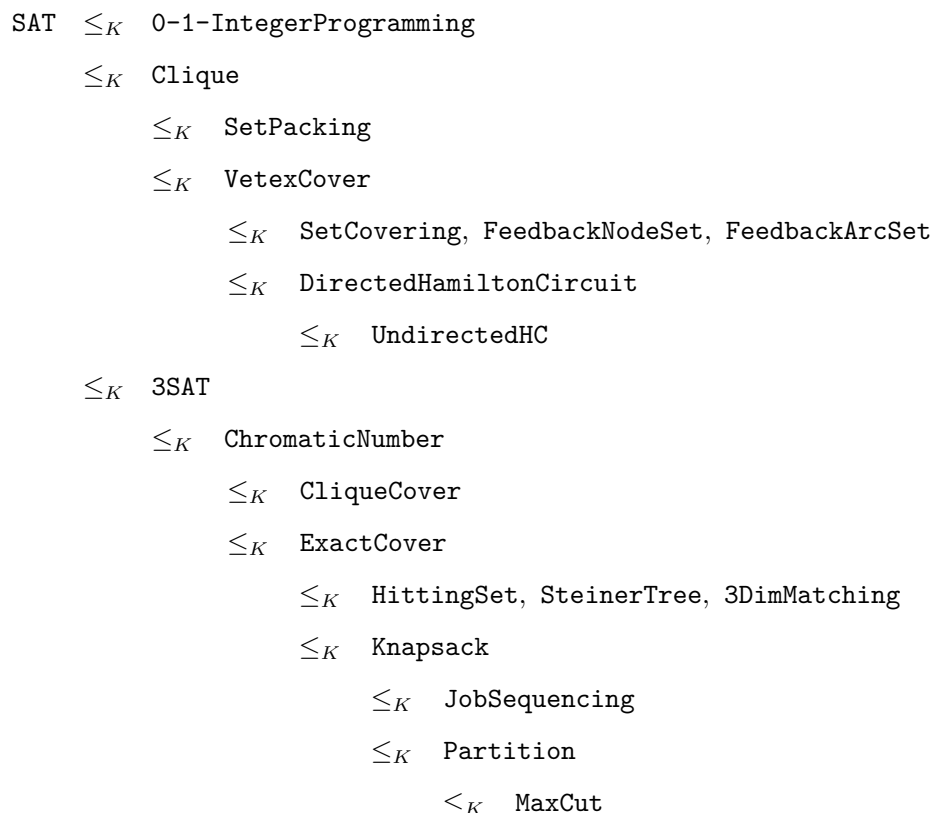


图 2.1 卡普的 21 个 NP-完备问题

self-reduction

判定 $\tau(x_1, \dots, x_n)$ 是否可满足。若可满足，再判定 $\tau(0, x_2, \dots, x_n)$ 是否可满足。若 $\tau(0, x_2, \dots, x_n)$ 可满足，继续判定 $\tau(0, 0, x_3, \dots, x_n)$ 是否可满足；若 $\tau(0, x_2, \dots, x_n)$ 不可满足，继续判定 $\tau(1, 0, x_3, \dots, x_n)$ 是否可满足。此方法可在多项式时间内找到一个满足 $\tau(x_1, \dots, x_n)$ 的真值指派。根据莱文性质，可在多项式时间内找到一个旅行商走法。此算法用到了自归约方法，即将一个问题的判定/计算规约到一个规模更小的同一问题的判定/计算。此例表明，NP-完备问题和它的搜索版本有相同的难度。

在库克的文章之后，卡普于 1972 年发表了一篇有重要意义的文章，证明了

21 个常见的组合问题是 NP-完备问题。图2.1指出了这 21 个组合问题和卡普所用的归约关系。比如, MaxCut 的完备性是通过将 Partition 归约到 MaxCut 得到的, 而 Partition 的完备性证明用的是 $\text{Knapsack} \leq_K \text{Partition}$ 。

在卡普文章发表之后, 大量的 NP-完备问题被发现。伯曼和哈特马尼斯考察了当时知道的 NP-完备问题, 发现了一个重要现象 [7]。用 $A \leq_K^1 B$ 表示从 A 到 B 有个单射卡普归约, 用 $A \cong_p B$ 表示从 A 到 B 有个双射卡普归约。若 $A \cong_p B$, 称 A 和 B 为多项式同构的。若下述条件满足, 称从 A 到 B 的卡普归约 r 是严格增长的: 对任意 x 有 $|r(x)| > |x|$ 。若一个卡普归约 f 有一个多项式时间可计算的逆函数 f^{-1} , 称 f 为可逆的。可逆的卡普归约 f 的逆函数也是卡普归约, 并且 f 和 f^{-1} 都是单射。下述定理的证明用到了递归论中的米希尔同构定理的证明思想, 后者的证明受集合论中相应结果证明的启发。

定理 2.4 (伯曼-哈特马尼斯定理). 若有严格增长的可逆卡普归约 $f: A \rightarrow B$ 和 $g: B \rightarrow A$, 则 $A \cong_p B$ 。

证明. 根据 f, g 的严格增长性质, 对任意串 x , 序列

$$x_0 = x, x_1 = f^{-1}(x_0), x_2 = g^{-1}(x_1), x_3 = f^{-1}(x_2), x_4 = g^{-1}(x_3), \dots \quad (2.2.2)$$

中的串长度严格递减, 因此 (2.2.2) 长度不会超过 $|x_0|$ 。定义 $h: A \rightarrow B$ 如下:

1. 若 (2.2.2) 止于某个偶数下标的 x_{2i} , 称 x 为 A -类的, 设 $h(x) = f(x)$ 。
2. 若 (2.2.2) 止于某个奇数下标的 x_{2i+1} , 称 x 为 B -类的, 设 $h(x) = g^{-1}(x)$ 。

设 $x \neq x'$ 。若 x, x' 均为 A -类的或均为 B -类的, 必有 $h(x) \neq h(x')$ 。若 x 为 A -类的而 x' 为 B -类的, 并且 $h(x) = h(x')$, 按定义有 $f(x) = g^{-1}(x')$, 即 $g(f(x)) = x'$ 。此等式说明 x, x' 为同类, 与假设矛盾。结论: h 是单射。一个串 z 如果不是 A -类的像, 就是 $g(z)$ 的逆像, 故 h 是满的。因为只用了线性多次 $\{f, g, f^{-1}, g^{-1}\}$ 中的函数, h 必定是多项式时间可计算的。□

伯曼和哈特马尼斯注意到, 通过添加无用信息, 容易将一个 NP-完备问题到 NP-完备问题的卡普归约变成严格增长的可逆卡普归约, 在此基础上和定理2.4的基础上提出了下述猜测。

伯曼-哈特马尼斯猜测. 所有 NP-完备问题都是多项式同构的。

伯曼-哈特马尼斯猜测的重要性在于下面的命题。

Berman

invertible

Myhill

集合论里, 若从 A 到 B 有单射且从 B 到 A 有单射, 则 A 和 B 的基数相等。

命题 3. 若伯曼-哈特马尼斯猜测成立, 则 $\mathbf{NP} \neq \mathbf{P}$ 。

dense
sparse

证明. 设 $S \subseteq \{0, 1\}^*$. 若 $|S^{\leq n}| = 2^{n^{O(1)}}$, 称 S 为稠密的. 若 $|S^{\leq n}| = n^{O(1)}$, 称 S 为稀疏的. 设 L 为稠密, L' 为稀疏, 且从 L 到 L' 有在多项式时间 $p(n)$ 内可计算的双射卡普归约函数 r . 按定义有, $r(L^{\leq n}) \subseteq (L')^{\leq p(n)}$. 这会引发矛盾, 因为 $r(L^{\leq n})$ 呈指数增长, 而 $(L')^{\leq p(n)}$ 呈多项式增长. 结论: 一个稠密的语言不可能和一个稀疏的语言多项式同构. 已知 NP-完备问题 SAT 是稠密的, 而 P-问题 $\{1^n \mid n \in \mathbf{N}\}$ 是稀疏的. 必有结论 $\mathbf{NP} \neq \mathbf{P}$. $\square$

Mahaney

撇开伯曼-哈特马尼斯猜测不说, 上述证明用到了一个简单的事实: 若不存在稀疏的 NP-完备问题, 则 $\mathbf{NP} \neq \mathbf{P}$. 事实上, 马哈尼证明了反方向的蕴含也成立 [39]. 必须指出的是, 也有迹象表明伯曼-哈特马尼斯猜测不一定成立. 在复杂性理论界, 伯曼-哈特马尼斯猜测并非广为接受。

在本节最后, 让我们回到本书开头提到的哥德尔的那个问题. 设 $\mathcal{A}$ 是读者熟悉的一个公理系统. 下述定义的问题可看成是希尔伯特问题的有限版本。

$$\mathbf{THEOREM} = \{(\varphi, 1^{|\varphi|^c}) \mid \varphi \text{ has a formal proof of length } \leq |\varphi|^c \text{ in } \mathcal{A}\}.$$

证明论中有个广为人知的说法, 公理系统中的验证都是多项式时间可判定的, 即给定命题 Prp 和证明 prf , 验证 prf 是否的确是 Prp 的证明可以在 $|Prp|$ 和 $|prf|$ 的一个多项式时间内完成. 因此 $\mathbf{THEOREM} \in \mathbf{NP}$. 另一方面, 命题公式是公理系统的一部分, 所以 SAT 可卡普归约到 $\mathbf{THEOREM}$. 结论是: $\mathbf{THEOREM}$ 是 NP-完备的. 对哥德尔来说, 这进一步巩固了他的不完备定理在数学中的地位。

2.3 拉德纳定理

Ladner

如果 $\mathbf{NP} \neq \mathbf{P}$, 那么所有的 NP-完备问题都在 $\mathbf{NP} \setminus \mathbf{P}$ 里. 我们感兴趣的是, 除了 NP-完备问题, $\mathbf{NP} \setminus \mathbf{P}$ 里还有其它的 NP-问题吗? 拉德纳研究了此问题, 他的研究表明, 若 $\mathbf{NP} \neq \mathbf{P}$, 那么 $\mathbf{NP}$ 含有无穷多个既非 P-问题也非 NP-完备的问题, 而且其结构看上去很复杂. 本书证明拉德纳定理的一个弱版本。

定理 2.5 (拉德纳定理). 若 $\mathbf{P} \neq \mathbf{NP}$, 则 $\mathbf{P} \cup \mathbf{NPC} \neq \mathbf{NP}$ 。

证明. 已知 SAT 是 NP-完备的, 但 2SAT 在 $\mathbf{P}$ 里. 作为集合, 2SAT 要比 SAT 稀疏很多. 证明的基本思路是从 SAT 中删去足够的元素, 使得剩下的集合足够稀疏, 不再是 NP-完备的, 但又不能太稀疏, 否则剩下的集合会在 $\mathbf{P}$ 里. 如何删除? 一个最简单的方法还是用填充技术, 在不改变可满足性的前提下, 将每个合取范式按一定比例拉长. 比如将 ψ 变成 $\psi \wedge v_1 \wedge \dots \wedge v_h$, 其中 $v_1, \dots, v_h$ 不出现在 ψ 里. 显然, ψ 可满足当且仅当 $\psi \wedge v_1 \wedge \dots \wedge v_h$ 可满足. 如果把 ψ 看成是二进制编码, $\psi \wedge v_1 \wedge \dots \wedge v_h$ 可用 $\psi 01^h$ 表示. 关键是确定 h 的大小. 下述容易验证的结果应给我们启发.

- $\{\psi 01^{|\psi|^c} \mid \psi \in \text{SAT}\}$ 是 NP-完备的, 而
- $\{\psi 01^{2^{|\psi|}} \mid \psi \in \text{SAT}\}$ 是多项式时间问题。

我们希望找到一个增长速度小于指数函数但快于任何一个多项式的函数 $h(n)$, 使得 $\{\psi 01^{h(|\psi|)} \mid \psi \in \text{SAT}\}$ 就是我们要的 NP-问题. 设 $h(n) = |\psi|^{H(|\psi|)}$. 拉德纳证明的难点在于: 要同时定义依赖于函数 H 的问题

$$\text{SAT}_H = \{\psi 01^{|\psi|^{H(|\psi|)}} \mid \psi \in \text{SAT}\},$$

和依赖于问题 SAT_H 的函数

$$H(n) = \begin{cases} i, & \begin{array}{l} i < \log \log n \text{ is the smallest index of a TM such that} \\ \mathbb{M}_i(x) \text{ outputs SAT}_H(x) \text{ in } i|x|^i \text{ steps for all } x \text{ with} \\ |x| \leq \log n; \end{array} \\ \log \log n, & \text{otherwise.} \end{cases}$$

在判定一个长度是 n 的串是否属于 SAT_H 时, 需要知道 H 在某个严格小于 n 的输入上的值, 当定义 $H(n)$ 的值时, 需要判定长度不超过 $\log(n)$ 的串是否在 SAT_H 里. 根据归纳, 两者都是良定义的. 函数 H 满足如下性质:

1. H 是非递减函数. 这是因为: 若 $H(n) = \log \log n$, 则 $\log \log n \leq H(n+1)$; 若 $i < H(n)$, 则按定义必有 $i < H(n+1)$. nondecreasing
2. H 可在多项式时间内计算. 计算 $H(n)$ 包含如下几项计算:
 - 对每个 $i < \log \log n$ 和长度不超过 $\log n$ 的 x , 长度不超过 $\log n$ 的串的个数不超过 $2^{1+\log n}$. 模拟 $\mathbb{M}_i(x)$ 计算 $i|x|^i$ 步. 根据定理1.2,

这部分计算时间为

$$(\log \log n) (2^{1+\log n}) c(\log \log \log n) C \log C,$$

其中 $C = (\log \log n)(\log n)^{\log \log n}$, c 是定理1.2给出的多项式。注意, 下标小于 $\log \log n$ 的自然数的二进制长度不超过 $\log \log \log n$ 。

- 对每个 $i < \log \log n$ 和长度不超过 $\log n$ 的 x , 要判定是否 $x \in \text{SAT}_H$, 用暴力法计算的时间是 $O(2^{\log n})$ 。
- 对每个 $i < \log \log n$ 和长度不超过 $\log n$ 的 x , 要计算 H 在一个长度不超过 $|x| \leq \log n$ 的输入上的值。

设 $T(n)$ 是计算 $H(n)$ 的时间函数。综上所述, 有

$$T(n) \leq (\log \log n) (2^{1+\log n}) (C' C \log C + O(2^{\log n}) + T(\log n) + \dots),$$

其中, $C' = c(\log \log \log n)$ 。因此, $T(n) = o(n^3)$ 。

假定有 $\mathbb{M}_i$ 在 cn^c 步判定 SAT_H 。只要取 $i \geq c$, 那么按照定义, 对所有足够大的 n 就有 $H(n) \leq i$ 。由 H 的非递减性知, 存在常数 D , 对所有 $n \geq D$, H 是常函数。这意味着 $\text{SAT} \leq_K \text{SAT}_H$ 。所以假设 $\text{SAT}_H \in \mathbf{P}$ 蕴含 $\mathbf{NP} = \mathbf{P}$, 与定理的前提 $\mathbf{NP} \neq \mathbf{P}$ 矛盾。结论: $\text{SAT}_H \notin \mathbf{P}$, 并且 H 的值域 $H(\mathbf{N})$ 是无限集, 即 H 不会停止增长。

设 SAT_H 是 NP-完备的。一定有某个多项式 dn^d 时间内可计算的卡普归约 $r: \text{SAT} \rightarrow \text{SAT}_H$ 。因 H 不会停止增长, 必有足够大 N , 当

$$|r(\varphi)| = |\psi 01^{|\psi|^{H(|\psi|)}}| > N$$

时, 有 $H(|\psi|) > 2d + 1$ 。因此, $|\psi|^{2d+1} < |\psi|^{H(|\psi|)} < |r(\varphi)| \leq d|\varphi|^d$ 。当 N 足够大时, 定有 $|\psi| \leq d$ 。由此推出不等式 $|\psi| < \sqrt{|\varphi|}$ 。利用此不等式, 可定义判定 SAT 的递归算法 $\text{Sat}(\varphi)$ 如下:

1. 计算 $r(\varphi)$, 其结果必定形如 $\psi 01^{|\psi|^{H(|\psi|)}}$ 。
2. 若 $|r(\varphi)| > N$, 递归调用 $\text{Sat}(\psi)$, 否则用暴力法判定 φ 的可满足性。

设此算法的递归调用深度为 k 。应有不等式 $|\varphi|^{2^{-k}} \leq N$, 即 $k \leq \log \log |\varphi|$ 。所以 Sat 为多项式算法, 和定理前提 $\mathbf{NP} \neq \mathbf{P}$ 矛盾。结论: $\text{SAT}_H \notin \mathbf{HPC}$ 。□

2.4 贝尓-吉尓-索罗维定理

一台神谕图灵机 (OTM) $M^?$ 有一条额外的读写带, 称为神谕带, 以及三个额外的状态 $q_{\text{query}}, q_{\text{yes}}, q_{\text{no}}$ 。在使用 $M^?$ 时, 需要提供一个神谕。一个神谕 B 是一个判定问题, 即 $\{0,1\}^*$ 的一个子集。用 M^B 表示连接了神谕 B 的 $M^?$ 。设输入为 x , 假定 $M^B(x)$ 的计算在某一时刻进入状态 q_{query} 时神谕带上写着字符串 z 。此时, 若 $z \in B$, 机器下一时刻进入状态 q_{yes} , 若 $z \notin B$, 机器下一时刻进入状态 q_{no} 。这一过程算作一步计算。非确定神谕图灵机的定义类似。神谕是对子程序的推广, M^B 就是一段调用判定 B 的子程序的程序。与子程序不一样, 一个神谕不一定可判定。利用哥德尔编码, 同样可以对神谕图灵机给出一个递归枚举:

$$M_0^?, M_1^?, M_2^?, \dots$$

设 O 为神谕。用带 O 的神谕图灵机可以定义复杂性类。举例说明:

- P^O 是带神谕 O 的具有多项式时间函数的神谕图灵机可判定的问题类。
- NP^O 是带神谕 O 的多项式时间非确定神谕图灵机可判定的问题类。

符号 $NP^{O[k]}$ 代表 NP^O 的一个子类。一个在 NP^O 中的问题 L 也在 $NP^{O[k]}$ 中当且仅当存在多项式时间非确定神谕图灵机 $M^?$, M^O 接受 L 且 M^O 的每次运行过程中问神谕 O 问题的次数不超过 k 。一个神谕图灵机可以将得到的答案进行反转, 所以 $NP^{\bar{O}} = NP^O$ 。如果神谕 A 是多项式时间可判定问题, 显然有 $P^A = P$ 。我们还可以用某一类神谕定义复杂性类, 比如

$$NP^{NP} = \bigcup_{L \in NP} NP^L.$$

一类神谕可以用该类中最难的做代表, 比如 $NP^{NP} = NP^{\text{SAT}}$, 这是因为问任何一个 NP-语言的问题都可以在多项式时间内转换成问 SAT 的问题。

语言 L 可库克归约到语言 L' , 记为 $L \leq_C L'$, 当且仅当存在多项式时间神谕图灵机 $M^?$ 使得 L 可由 $M^{L'}$ 判定。显然, $L \leq_C L'$ 当且仅当 $L \leq_C \bar{L}'$ 。库克在文献 [12] 里用的就是库克归约, 对应于递归论中的图灵归约。卡普在文献 [27] 中用的是卡普归约, 对应于递归论中的 m-归约。读者一定会问: 库克和卡普所定义的 NP-难和 NP-完备性是否一致? 我们现在能说的是: 一、 $L \leq_K L'$

oracle TM

Cook reduction

蕴含 $L \leq_C L'$ ；二、不知道 $L \leq_C L'$ 是否蕴含 $L \leq_K L'$ ；三、未发现一个 NP 问题在库克意义下是完备但在卡普意义下的完备性未能被证明。一个很能说明库克归约和卡普归约差别的例子是：在库克归约下， $\mathbf{NP} = \mathbf{coNP}$ ，而在卡普归约下， $\mathbf{NP} = \mathbf{coNP}$ 是否成立是个巨大的未知问题。在解决实际实际问题时，我们用的都是库克归约。

库克归约提供了一种判定一类子程序调用是否能实质性地提高计算能力的方法。

low

定义 2.3. 若 $\mathbf{A}^B = \mathbf{A}$ ，称复杂性类 $\mathbf{B}$ 对复杂性类 $\mathbf{A}$ 是低的。

直观上，若 $\mathbf{B}$ 对 $\mathbf{A}$ 是低的，那么从 $\mathbf{A}$ 的角度看，调用类型为 $\mathbf{B}$ 的子程序是不必要的。看些例子：

- $\mathbf{P}$ 对自己而言是低的。
- $\mathbf{PSPACE}$ 对自己而言是低的。
- $\mathbf{L}$ 对自己而言是低的。
- 普遍认为 $\mathbf{NP}$ 对自己而言不是低的。目前我们不知道准确答案。第5章将详细讨论这类问题。
- $\mathbf{EXP}$ 对自己而言不是低的。一台指数时间的神谕图灵机运行时可问指数大小的问题，所以 $\mathbf{EXP}^{\mathbf{EXP}} = 2\text{-}\mathbf{EXP}$ 。

Baker, Gill
Solovay

在结束本章之前，总要花点篇幅讨论一下“ $\mathbf{P} \stackrel{?}{=} \mathbf{NP}$ ”这个问题。关于这个问题的证明，贝克、吉尔和索罗维给出了一个著名的否定结果 [5]。

定理 2.6 (贝克-吉尔-索罗维定理). 存在 A, B 使得 $\mathbf{P}^A = \mathbf{NP}^A$ 且 $\mathbf{P}^B \neq \mathbf{NP}^B$ 。

证明. 神谕 A 容易构造。在第3章里，我们会引入多项式空间类 $\mathbf{PSPACE}$ 。易见， $\mathbf{PSPACE} = \mathbf{P}^{\mathbf{PSPACE}} \subseteq \mathbf{NP}^{\mathbf{PSPACE}} = \mathbf{PSPACE}$ 。取 $\mathbf{PSPACE}$ 中的一个完备问题 A ，显然有 $\mathbf{P}^A = \mathbf{NP}^A$ 。

构造 $B_0 \subseteq B_1 \subseteq B_2 \subseteq \dots$ 和 $n_0 < n_1 < n_2 < \dots$ 。定义 $B_0 = \emptyset$ 和 $n_0 = 0$ 。用对角线方法从 B_i 构造 B_{i+1} ：

1. 设 n_{i+1} 是一个大于 $n_0, n_1, \dots, n_i$ 的数。另外，确保 n_{i+1} 大于构造 $B_1, \dots, B_i$ 时所询问的所有问题的长度。

2. 让 $\mathbb{M}_i^{B_i}(1^{n_{i+1}})$ 计算 $2^{n_{i+1}-1}$ 步。

- 若 $\mathbb{M}_i^{B_i}(1^{n_{i+1}})$ 的计算在 $2^{n_{i+1}-1}$ 步内未终止, 定义 $B_{i+1} = B_i$ 。
- 若 $\mathbb{M}_i^{B_i}$ 在 $2^{n_{i+1}-1}$ 步内接受 $1^{n_{i+1}}$, 定义 $B_{i+1} = B_i$ 。
- 若 $\mathbb{M}_i^{B_i}$ 在 $2^{n_{i+1}-1}$ 步内拒绝 $1^{n_{i+1}}$, 定义 $B_{i+1} = B_i \cup \{s\}$, 其中 $|s| = n_{i+1}$ 并且计算 $\mathbb{M}_i^{B_i}(1^{n_{i+1}})$ 时 s 没被问过。

设 $B = \bigcup_{i \in \mathbb{N}} B_i$ 。定义 U_B 为问题 $\{1^n \mid B \text{ 包含一个长度为 } n \text{ 的串}\}$ 。显然 $U_B \in \mathbf{NP}^B$ 。用反证法证明 $U_B \notin \mathbf{P}^B$ 。假设 $\mathbb{M}_i^B$ 在多项式时间 $T(n)$ 内判定 U_B 。取满足 $T(n_i) < 2^{n_i-1}$ 的足够大的 i 。按构造, $\mathbb{M}_i^B(1^{n_{i+1}}) = 1$ 当仅当 $\mathbb{M}_i^{B_i}(1^{n_{i+1}}) = 1$ 当仅当 $\mathbb{M}_i^{B_i}(1^{n_{i+1}}) = 0$ 当仅当 $\mathbb{M}_i^B(1^{n_{i+1}}) = 0$ 。矛盾。 $\square$

若 $\mathbf{A} \neq \mathbf{B}$ (或 $\mathbf{A} = \mathbf{B}$) 的一个证明, 本质上也是 $\mathbf{A}^O \neq \mathbf{B}^O$ (或 $\mathbf{A}^O = \mathbf{B}^O$) 的一个证明, 则称该证明是可相对化的。需要强调的是, 在相对化过程中, 神谕 O 是任意的。贝克-吉尔-索罗维定理指出, 如果有 $\mathbf{P} = \mathbf{NP}$ 或 $\mathbf{P} \neq \mathbf{NP}$ 的证明, 该证明一定是不可相对化的。正如贝克、吉尔和索罗维在文献 [5] 中所述, “我们认为这是 $\mathbf{P} \stackrel{?}{=} \mathbf{NP}$ 这一问题难度的又一佐证。... 一般的对角线方法不太可能产生一个在 $\mathbf{NP}$ 中但不在 $\mathbf{P}$ 中的语言, 这类证明在相对化后也应能成立。... 另一方面, 我们也不觉得会有一个用非确定图灵机在多项式时间内模拟确定图灵机的一般方法, 因为这类方法也可相对化。” 贝克-吉尔-索罗维定理并没有完全排除对角线方法, 只是排除了可以相对化的对角线证明。在后面章节, 我们会看到若干不可相对化的用对角线方法的证明。

relativize

第 3 章 空间复杂性

一个简单的寻找满足合取范式的真值指派的算法可设计如下：在工作带上维护一个线性长度的计数器，枚举所有的真值指派；同时维护一个对数长度的计数器，用来对输入公式求值。这个算法是指数时间的，但其所用的工作带上的格子数是 $n + \log(n)$ 加上某个常数。类似的思路可以帮助我们在下围棋时找到一个最优策略（假设棋盘是输入），方法是枚举所有可能的下法和对手所有可能的应招，选出最好的策略。这个算法当然也是指数的，使用的工作带上的格子数也是多项式的。但是，寻找真值指派和寻找围棋的最优策略给人的感觉是难度差别很大的两个问题。前者的答案有一个简单的证明，后者的答案（一个最优策略）可能无法在多项式时间内写出来。对于难问题，我们应该用多种衡量标准对它们进行分析。空间复杂性提供了一个标准。与时间资源不一样，空间资源可以重用。可重用性导致空间复杂性对难问题有风格迥异的刻画。本章将介绍空间复杂性的基本结果和空间复杂性理论的主要研究方法。

设 $S : \mathbf{N} \rightarrow \mathbf{N}$ ，且 $L \subseteq \{0,1\}^*$ 。若有常数 c 和判定语言 L 的图灵机 $\mathbb{M}$ ，当输入 x 时， $\mathbb{M}(x)$ 计算时使用的工作带上的格子数不会超过 $cS(n)$ ，那么 L 在 $\mathbf{SPACE}(S(n))$ 中。称 $S(n)$ 为 $\mathbb{M}$ 的空间函数。我们假定图灵机要把输入完全读一遍，从空间使用的角度看，图灵机只需而且必须维持一个计数器，记住当前输入带上只读头的位置。所以我们规定所有的空间函数 $S(n)$ 都满足不等式 $S(n) \geq \log n$ 。我们还需假定所有的空间函数都是可构造的。和时间可构造一样，我们也可以定义两个强弱稍有不同的空间可构造性。

定义 3.1. 若有图灵机在 $O(S(n))$ 空间内计算函数 $1^n \mapsto \lfloor S(n) \rfloor$ ，称 S 是空间可构造的。

因为图灵机可以对访问过的格子做标记，所以下面定义的空间可构造性强

于上面定义的空间可构造性。

定义 3.2. 若有图灵机当输入为 1^n 时准确地使用了工作带上的 $S(n)$ 格后停机, 称 S 是空间可构造的。

若有常数 c 和接受 L 的非确定图灵机 N , 无论选择哪条计算路径, N 所用的工作带上的格子数不超过 $cS(n)$, 那么 L 在 $\mathbf{NSPACE}(S(n))$ 中。若 $S(n)$ 为空间可构造, 我们只要确保 N 的接受计算使用的空间不超过 $cS(n)$ 即可。用符号 $\mathbf{SPACE}(_)$ 和 $\mathbf{NSPACE}(_)$, 可定义几个常用的空间复杂性类: (非确定) 对数空间类和多项式空间类。

$$\begin{aligned}\mathbf{L} &\stackrel{\text{def}}{=} \mathbf{SPACE}(\log(n)), \\ \mathbf{NL} &\stackrel{\text{def}}{=} \mathbf{NSPACE}(\log(n)), \\ \mathbf{PSPACE} &\stackrel{\text{def}}{=} \bigcup_{c>0} \mathbf{SPACE}(n^c), \\ \mathbf{NPSPACE} &\stackrel{\text{def}}{=} \bigcup_{c>0} \mathbf{NSPACE}(n^c).\end{aligned}$$

本章后面几节将对这些类做详细研究。看一个在 $\mathbf{L}$ 中的问题。定义

$$\mathbf{MULP} \stackrel{\text{def}}{=} \{(\lfloor m \rfloor, \lfloor n \rfloor, \lfloor m \times n \rfloor) \mid m, n \in \mathbf{N}\}.$$

用小学生学的算术算法就可在对数空间判定 $\mathbf{MULP}$ 。算法只需维护一个最大值为 $m+n$ 的计数器, 用来记住正在处理哪一位, 还需有个最大值为 $n+1$ 的计数器, 用来存储进位。

空间复杂性有类似于加速定理的结果。事实上, 定理1.6的证明给出了一种空间压缩方法。在该方法中, 对一步计算的模拟可以在工作带上的常数格内完成。我们可以对这常数格的内容进一步压缩, 用一个符号表示这一段内容。

定理 3.1 (空间压缩定理). 设图灵机 M 在 $S(n)$ 空间判定 L 。对任意 $\epsilon > 0$, 存在图灵机 M' , M' 能在 $\epsilon S(n) + 1$ 空间内判定 L 。

在研究空间复杂性时, 一个重要的工具是格局图。给定图灵机 M 和输入 x , 格局图 $G_{M,x}$ 的结点是格局, 有向边表示一步计算。在讨论格局图时, 我们假定只有一个接受格局。用 C_{start} 表示起始格局, 用 C_{accept} 表示接受格局。显然, M 接受 x 当且仅在 $G_{M,x}$ 中有一条从 C_{start} 到 C_{accept} 的路径。格局图

将“ M 是否接受 x ”问题转换成了图的可达性问题。可达性问题有多项式算法，所以算法的复杂性本质上依赖于图的大小。设 M 的空间函数是 $S(n)$ 。状态的编码长度不超过 $\log n$ ，带子上的内容长度是 $O(S(n))$ ，读写头位置的编码长度是 $O(\log S(n))$ 。所以格局图 $G_{M,x}$ 结点的大小是 $O(S(n))$ ，结点数不会超过 $2^{O(S(n))}$ 。下述引理是研究空间复杂性的基础。

引理 3.1. 存在 $O(S(n))$ 大小的合取范式 $\varphi_{M,x}$ ，对任意格局 C, C' (的编码)， $\varphi_{M,x}(C, C') = 1$ 当且仅当 $G_{M,x}$ 有边 $C \rightarrow C'$ 。对 $\varphi_{M,x}(C, C') = 1$ 的求值可同时在 $O(S(n))$ 时间和 $O(\log S(n))$ 空间内完成。

证明. 简单地说， $\varphi_{M,x}(C, C')$ 将 C 和 C' 按位进行比较，在 C 的读写头位置，用一段线性长的公式对迁移函数进行编码。在定理2.3的证明中，我们已经做过这类练习。要同时满足对时间和空间的限制，必须将 C 和 C' 交叉排列。可以在每格的符号编码上附加一位，表示读写头是否指向该格。 $\square$

看一个使用格局图的简单例子。

命题 4. 设 $S(n) : \mathbf{N} \rightarrow \mathbf{N}$ 为空间可构造。有下述包含关系：

$$\mathbf{TIME}(S(n)) \subseteq \mathbf{SPACE}(S(n)) \subseteq \mathbf{NSPACE}(S(n)) \subseteq \mathbf{TIME}(2^{O(S(n))}).$$

证明. 图灵机在 $S(n)$ 时间内最多访问 $S(n)$ 格，这就是第一个包含。图灵机是非确定图灵机的特例，这就是第二个包含。格局图的大小是 $2^{O(S(n))}$ ，图的可达性问题有多项式算法，指数的多项式是指数，这就是第三个包含。 $\square$

一个多项式时间的非确定图灵机的每条计算路径都可以在多项式空间里完成，而空间是可以重用的。因此，

$$\mathbf{NP} \subseteq \mathbf{PSPACE}.$$

上述包含关系表明，“猜测，然后验证之”要比双人博弈容易。要找一个围棋的致胜策略，我们能想到的算法就是遍历博弈空间。

在第1章，我们从时间开销的角度讨论了通用图灵机。如果着眼点是空间开销，通用图灵机能做得多好？

定理 3.2. 存在通用图林机，当输入空间函数为 $S(n)$ 的图灵机 M 和串 x ，通用图灵机在 $c(n)$ 空间内完成 $M(x)$ 的计算，其中 c 依赖于 M 不依赖于 x 。

证明. 通用图灵机只需用一条读写带记录 M 的工作带内容, 用计数器将读写头的当前地址记下, 再用一个计数器将当前状态记下, 就能模拟输入图灵机的计算。模拟时还需要一些额外的空间, 这些空间的大小只依赖于输入机器, 不依赖于输入串。 $\square$

用此通用图灵机, 可以得到一个看上去很难改进的空间谱系定理 [20]。

定理 3.3 (空间谱系定理). 设空间函数 f, g 为空间可构造, 且 $f(n) = o(g(n))$ 。包含 $\text{SPACE}(f(n)) \subsetneq \text{SPACE}(g(n))$ 是严格的。

证明. 证明和时间谱系定理的证明几乎完全一样。图灵机 $\mathbb{V}$ 的定义如下:

- 利用 $g(n)$ 的可构造性, 在工作带上标注 $g(n)$ 大小的领地。
- $\mathbb{V}(x)$ 模拟 $M_x(x)$ 的计算, 每当 $M_x(x)$ 试图跨越标注的领地, 停机。
- 若模拟能够完成, $\mathbb{V}$ 将结果反转。

显然, $\mathbb{V}$ 定义的语言在 $\text{SPACE}(g(n))$ 中。如果该语言也在 $\text{SPACE}(f(n))$ 中, 它就被某个空间函数为 $f(n)$ 的图灵机 M_α 所判定。当 α 足够长时, $\mathbb{V}$ 就能模拟完 $M_\alpha(\alpha)$ 的计算。这就导出矛盾 $\overline{M_\alpha(\alpha)} = \mathbb{V}(\alpha) = M_\alpha(\alpha)$ 。 $\square$

3.1 对数空间类

无论我们关心的是何种资源, 我们首先感兴趣的是最小资源类。对时间复杂性而言, 最下资源类是 $\mathbf{P}$ 。对空间复杂性而言, 最小资源类是 $\mathbf{L}$ 。

在进一步讨论空间复杂性类之前, 先得确定比较空间复杂性类的标准。卡普归约用于研究时间复杂性类, 不适合用来研究空间复杂性类。我们需要一个保持对数空间可解性的归约, 这就是对数空间归约。

定义 3.3. 如果下述条件满足, 称 $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$ 为隐式对数空间可计算的:

1. $\exists c. \forall x. |f(x)| \leq c|x|^c$;
2. $\{\langle x, i \rangle \mid i \leq |f(x)|\} \in \mathbf{L}$;
3. $\{\langle x, i \rangle \mid f(x)_i = 1\} \in \mathbf{L}$ 。

implicitly logspace
computable

在输入 x 后, 隐式对数空间可计算函数 f 的输出 $f(x)$ 长度是多项式的。根据性质 2, 输出的准确长度可用二分法在对数空间算出。性质 3 确保输出的每一位可在对数空间算出。

定义 3.4. 问题 B 可对数空间归约到问题 C , 记为 $B \leq_L C$, 若存在隐式对数空间可计算函数 f 满足: $x \in B$ 当且仅当 $f(x) \in C$ 。

根据命题4中的最后一个包含关系, 对数空间归约一定是卡普归约。尚不清楚反方向的蕴含是否成立。所有的 NP-完备性证明用对数空间归约同样成立。事实上, 有些作者在讨论时间复杂性时也用对数归约 [46]。

引理 3.2. 若 $B \leq_L C$ 和 $C \leq_L D$, 则有 $B \leq_L D$ 。

证明. 设 $f: B \rightarrow C$ 和 $g: C \rightarrow D$ 为隐式对数空间可计算函数。从 B 到 D 的隐式对数空间可计算函数直截了当: 给定输入 x , 先计算 $f(x)$, 再计算 $g(f(x))$ 。因为 $f(x)$ 是多项式长的, 所以 $g(f(x))$ 是多项式长的。需要注意的是, 因为 $f(x)$ 是多项式长的, 所以不能把它写在工作带上。但计算 g 的图灵机每步计算只需要看 $f(x)$ 的一位。隐式对数空间可计算函数的定义确保 $f(x)$ 的任意指定位都可以在对数空间算出来, 所以根本不需要把 $f(x)$ 写在工作带上。只需要一个对数长的计数器记录 g 正在读 $f(x)$ 的哪一位。可以用通用图灵机模拟 f 和 g 的计算, 根据定理3.2, 这部分计算也是对数空间的。□

logspace com-
putable
write-only

对数归约可用另一类更直观的机器定义。函数 $f: \{0,1\}^* \rightarrow \{0,1\}^*$ 是对数空间可计算当且仅当有一台 k 带图灵机计算它, 并且这台图灵机的最后一条工作带是只写输出带。只写输出带的读写头每写一次, 必须向右移一格。所以只写输出带上的内容既不能被改写, 也不能被读。下述引理的证明留给读者。

引理 3.3. 隐式对数空间可计算函数就是对数空间可计算函数。

若对所有 $L \in \mathbf{L}$ 有 $L \leq_L L'$, 称 L' 是 L -难的。若 $L' \in \mathbf{L}$ 是 L -难的, 称 L' 是 L -完备的。类似地, 可以定义相对于 **NL**、**PSPACE**、**NPSPACE** 的难问题和完备问题。非确定对数空间类 **NL** 有个标准的完备问题, 其定义如下:

$$\text{Reachability} = \{ \langle G, s, t \rangle \mid \text{有向图 } G \text{ 中, 存在从 } s \text{ 到 } t \text{ 的路径} \}.$$

我们有下面著名的对可达性问题的刻画。

定理 3.4. *Reachability* 是 **NL**-完备的。

证明. 设输入图有 n 个结点。从图的当前结点 v 猜测其一个邻居, 然后释放 v 所占用的空间。从 s 出发重复此操作 $n-1$ 次, 看是否能到达 t 。结点的长度是对数的, 控制猜测次数的计数器也是对数长的。因此, *Reachability* 在 **NL** 中。

设非确定图灵机 N 在对数空间判定 L 。从 L 到 *Reachability* 的对数空间归约定义如下:

$$x \mapsto \langle G_{N,x}, C_{\text{start}}, C_{\text{accept}} \rangle. \quad (3.1.1)$$

格局图 $G_{N,x}$ 可用临接矩阵表示, 矩阵的每个元素可在对数空间计算出, 见引理3.1。起始格局 C_{start} 和接受格局 C_{accept} 是固定的 0-1 串。因此 (3.1.1) 定义了一个隐式对数空间可计算函数。 $\square$

我们可以假定对数空间的非确定图灵机的每条计算路径都终止, 因为我们总是可以将一台图灵机和一台多项式时间时钟图灵机做硬连接。此类图灵机的格局图是有向无圈图, 故有下述推论。

推论 3.1. 有向无圈图的可达性问题是 **NL**-完备的。

3.2 多项式空间类

多项式空间类是一个非常大的类, 本书中讨论的大多数复杂性类都包含在这个类里, 这从一个侧面反映了 **PSPACE** 中问题的多样性。**PSPACE** 中的难问题是对人类计算能力的极限挑战, 人类发明的智力对抗游戏(围棋、象棋)都在这个类里。我们已经无奈地接受了一个事实, 在这类博弈中, 人类是玩不过多项式空间图灵机的。本节证明关于 **PSPACE** 的若干性质。首先介绍 **PSPACE** 中的一个最难的问题。一个量化布尔公式 (QBF) 是如下形式的公式:

$$Q_1 x_1 Q_2 x_2 \dots Q_n x_n \cdot \varphi(x_1, \dots, x_n),$$

其中, Q_i 或为全称量词 $\forall$ 或为存在量词 $\exists$, 在 $Q_1, \dots, Q_n$ 中, $\forall, \exists$ 交替出现。 x_i 为一串布尔变元, $\varphi(x_1, \dots, x_n)$ 为不含量词的合取范式, 该公式不含有任何不在 $\{x_1, \dots, x_n\}$ 中的变元。每个量化布尔公式都有真假值。比如 $\forall x \exists y. x = y$ 为真, $\exists x \forall y. x = y$ 为假。问题 **TQBF** 定义为所有取值为真的量化布尔公式。下述引理的证明给出了判定 **TQBF** 的一个标准算法。

Quantified
Boolean Formula

True QBF

引理 3.4. *TQBF* 可在线性空间判定。

证明. 设 $\psi = Q_1x_1Q_2x_2\ldots Q_nx_n.\varphi(x_1,\ldots,x_n)$ 。不失一般性, 假定每个 x_i 都是单个变元。将 ψ 想象成一棵带标号的求值二叉树, 见图3.1, 每条边的标号表明对变量的赋值, 每层结点的标号依次为 $Q_1x_1, \ldots, Q_nx_n$, 叶子结点的标号是 φ 在从根节点到该叶子结点所定义的真值指派下的值。算法从根节点出发对树做深度优先遍历, 过程中将遍历过的结点的布尔值算出来, 直至把根节点的值算出来。用一个长度不超过 n 的 0-1 串表示算法当前访问结点的位置。命题公式 $\varphi(x_1,\ldots,x_n)$ 放在一条工作带上。给定 $x_1,\ldots,x_n$ 的一个真值指派, $\varphi(x_1,\ldots,x_n)$ 的值可在线性时间算出。 $\square$

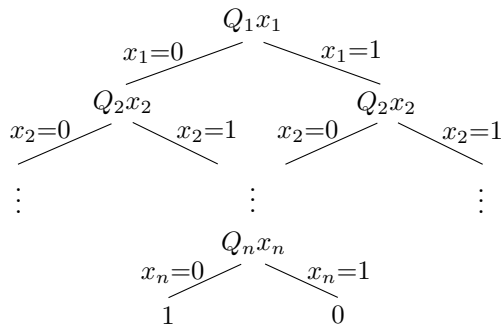


图 3.1 求值二叉树

Savitch

空间复杂性类所呈现的结构和时间复杂性类很不一样。引理3.4是一个例子。下面的萨维奇定理是又一个例子 [51]。

定理 3.5 (萨维奇定理). $\mathbf{NSPACE}(S(n)) \subseteq \mathbf{SPACE}(S(n)^2)$, 这里 $S(n)$ 为空间可构造的。

证明. 设 $\mathbb{M}$ 在 $S(n)$ 空间判定 L , $x \in \{0,1\}^*$ 。拟构造大小为 $O(S(|x|)^2)$ 的, 满足下述等价关系的量化布尔公式 φ_x : $\mathbb{M}(x) = 1$ 当且仅当 $\varphi_x \in \mathbf{TQBF}$ 。格局图 $G_{\mathbb{M},x}$ 有 $2^{S(n)}$ 个结点。 $\mathbb{M}(x) = 1$ 当且仅当从 C_{start} 到 C_{accept} 有一条长度不超过 $2^{S(n)}$ 的路径。受二分法启示, 我们用命题公式 $\psi_i(C, C')$ 表示从 C 到 C' 有一条长度不超过 2^i 的路径。这些公式可用归纳法定义。 $\psi_0(C, C')$ 的定义已

在引理3.1的证明中给出。设 $\psi_i(C, C')$ 已定义, $\psi_{i+1}(C, C')$ 定义为下述公式:

$$\exists C'' \forall D^1 \forall D^2. ((D^1 = C \wedge D^2 = C'') \vee (D^1 = C'' \wedge D^2 = C')) \Rightarrow \psi_{i-1}(D^1, D^2).$$

将 φ_x 定义为 $\psi_{S(|x|)}$ 。不难看出, 有递推公式 $|\psi_i| = |\psi_{i-1}| + O(S(|x|))$ 。因此, $|\varphi_x| = |\psi_{S(|x|)}| = O(S(|x|)^2)$ 。由引理3.4知, φ_x 可在 $O(S(|x|)^2)$ 空间判定。□

萨维奇定理的一个直接结果是下面的推论。

推论 3.2. PSPACE = NPSPACE。

下面著名的结果是斯托克迈尔和梅耶证明的 [55]。

定理 3.6 (斯托克迈尔-梅耶定理). $TQBF$ 是 $PSPACE$ -完备的。

证明. 用引理3.4的结论和定理3.5的证明即可推出本定理。□

可将量化布尔公式

$$\exists x_1 \forall x_2 \exists x_3 \forall x_4 \cdots \exists x_{2n-1} \forall x_{2n}. \varphi(x_1, \dots, x_{2n}) \quad (3.2.1)$$

的求值过程看成一个双人博弈的结果。选手一对应存在量词 $\exists$, 选手二对应全程量词 $\forall$, 选手每一步下棋就是对其变量赋值。选手一有一个致胜策略当仅当 (3.2.1) 为真。帕帕季米特里乌在文献 [45] 中讨论了双人博弈的 $PSPACE$ -完备性问题。

3.3 伊默尔曼-斯泽勒普森伊定理

根据萨维奇定理, 有如下推导:

$$\begin{aligned} \text{coNPSPACE} &= \overline{\text{NPSPACE}} \\ &= \overline{\text{PSPACE}} \\ &= \text{PSPACE} \\ &= \text{NPSPACE}. \end{aligned}$$

读者可以验证, 此推导不适用于对数空间类。斯泽勒普森伊[60] 和伊默尔曼[25] 证明了 **NL** 也在补运算下封闭。这一结果多少有点出人意料。

Szelepcsényi
Immerman

定理 3.7 (伊默尔曼-斯泽勒普森伊定理). $\overline{\text{Reachability}} \in \text{NL}$ 。

证明. 需设计一对数空间非确定图灵机, 当输入一个可达性问题实例 (G, s, t) 时, $N((G, s, t)) = 1$ 当且仅当从 s 到 t 没有路径。设 G 有 n 个结点。对任意 $i \in [n-1]$, 引入集合 C_i 和数 c_i , 其定义如下:

- C_i 为从 s 出发 i 步内能到达的结点集合。显然有 $C_1 \subseteq C_2 \subseteq \dots \subseteq C_{n-1}$ 。
- $c_i = |C_i|$ 。

每个 c_i 的长度是对数的, 所以可以把固定数目的 c_i 存放在工作带子上, 但不能把所有的 c_i 存放在工作带子上。每个 C_i 一般含有线性多个结点, 所以不能把任何 C_i 存放在工作带上。考察下述非确定算法:

1. 设所有 $c_1, \dots, c_{n-1}$ 初值为 0。
2. 计算 c_1 。
3. 对所有 $i \in [i-1]$, 从 c_i 计算 c_{i+1} 。
4. 从 c_{n-1} 判定是否 $t \notin C_{n-1}$ 。

算法的第 2 步显然可以在对数空间完成。第 3 步的计算可用如下方法:

- 对每个不为 s 的结点 v , 若 $v \in C_i$ 或从某个 C_i 中的结点 u 到 v 有条边, 计算器 c_{i+1} 加 1。

问题是, 如何知道 C_i ? 因为 C_i 不可存放在工作带上, 所以算法只能去猜 C_i 。要确保对 C_i 猜测的正确性, 得做到如下两点:

- C_i 中的每个结点都可以从 s 出发在 i 步内到达。这其实就是 **Reachability** 的非确定算法。
- C_i 包含了所有从 s 出发在 i 步内能到达的结点。这可以用一个计数器计算 $|C_i|$, 确保 $|C_i| = c_i$ 。

非确定算法总会有一条计算路径将 C_i 正确地猜出来的。第 4 步, 只要正确地猜出 C_{n-1} , 就能判定 $t \notin C_{n-1}$ 。

上述非确定算法自始至终都在猜测, 但只要 t 从 s 不可达, 就一定会有一条成功的路径。 □

推论 3.3. $\text{coNL} = \text{NL}$ 。

证明. 因 Reachability 是 NL -完备的, 所以 $\overline{\text{Reachability}}$ 是 coNL -完备的。此完备性结果和定理3.7一起推出 $\text{coNL} \subseteq \text{NL}$ 。按补问题的定义, $\text{coNL} \subseteq \text{NL}$ 当且仅当 $\text{NL} \subseteq \text{coNL}$ 。□

伊默尔曼-斯泽勒普森伊定理的证明可被推广到下述结果的证明。

推论 3.4. $\text{coNSPACE}(S(n)) = \text{NSPACE}(S(n))$, 这里 $S(n)$ 为空间可构造的。

下面的定理也是伊默尔曼-斯泽勒普森伊定理的一个推论。

定理 3.8. 2SAT 是 NL -完备的。

证明. 给定有向无圈图 G 和其中的两个结点 s, t , 将 G 的每条边 $x \rightarrow y$ 变成一个语句 $\bar{x} \vee y$, 将 $\bar{s}$ 和 $\bar{t}$ 也作为语句, 得到 2-合取范式 φ_G 。易见, 从 s 不可达 t 当且仅当 φ_G 不蕴含 t 当且仅当 φ_G 可满足。所以 $\overline{\text{Reachability}}$ 可对数空间归约到 2SAT 。因此, 2SAT 是 NL -难的。在命题1的证明中, 给出了从 2SAT 到 Reachability 的对数空间归约。根据定理3.4, $2\text{SAT} \in \text{NL}$ 。定理得证。□

在第2章, 我们提到过 $\mathbf{L}^{\mathbf{L}} = \mathbf{L}$ 和 $\mathbf{P}^{\mathbf{P}} = \mathbf{P}$, 以及 $\mathbf{NP}^{\mathbf{NP}}$ 不太可能和 $\mathbf{NP}$ 相等。不等式 $\mathbf{NP}^{\mathbf{NP}} \neq \mathbf{NP}$ 是第4章讨论的基于 $\mathbf{P}$ 的多项式时间谱系的基础。推论3.3指出, 研究 $\text{coNL}^{\text{coNL}}$ 是没有意义的。用第4章的术语说就是: 基于 $\mathbf{L}$ 的空间谱系塌陷到 $\mathbf{L}$ 。对空间复杂性类而言, 取补操作和引入不确定性无法都增加问题的难度。如果原问题用一类格局图、计数器、深度优先搜索方法能处理, 它的补问题和非确定版本也可以。

在结束本节之前, 让我们看一个让计算机科学家足够尴尬的复杂性类包含序列:

$$\mathbf{L} \subseteq \text{NL} \subseteq \mathbf{P} \subseteq \mathbf{NP} \subseteq \mathbf{PSPACE} \subseteq \mathbf{EXP}. \quad (3.3.1)$$

根据时间谱系定理和空间谱系定理, (3.3.1) 中必定有一些包含关系是严格的。事实上, 大多数人认为 (3.3.1) 中的每个包含关系都是严格的。但我们不知道哪个包含关系是严格的!

3.4 时空定理

霍普克罗夫特、保罗、瓦里昂特[24].

Hopcroft

Paul

Valiant

定理 3.9. 设 $S(n)$ 为空间可构造。有 $\mathbf{TIME}(S(n)) \subseteq \mathbf{SPACE}(S(n)/\log S(n))$ 。

第 4 章 电路复杂性

图灵机的一次计算经历一个状态序列，在相同的状态下，图灵机可以执行同一条指令，也可以执行不同的指令。在一次计算过程中，同一指令可能被执行很多遍。如果输入的某一位从 0 变成了 1，计算可能完全不一样，结果也可能相反。一台“小”的图灵机可能计算时间很长，而一台“大”的图灵机在相同的输入下可能会计算得很快。图灵计算的这些特点让我们很难用组合方法、代数方法分析图灵计算，因此也很难推出算法的好的上下界。鉴于这些难点，人们研究了一些具体模型的计算复杂性理论 [35, 14, 26, 44]。具体模型有各自的应用场景。相比图灵机，一些具体的模型有简单得多的计算结构和逻辑结构。在理论层面，起初的希望是证明出一些绝对的复杂性结果，甚至解决 $P \neq NP$ 问题。现在我们知道，基于这些具体模型的复杂性理论研究毫无例外地遇到了瓶颈。

concrete model

图灵机模型是一致模型，一台图灵机可以处理所有长度的输入。不难想象，一个处理所有输入的算法要比一个处理特定长度输入的算法复杂。在实际应用中，我们并不需要一个处理所有输入的程序。对某个应用，一个能处理输入长度不超过 10^6 的程序可能就足够好。我们可以为每个输入长度设计一个算法，该算法的时间开销小于图灵机模型在同样输入上的时间开销。非一致模型允许不同长度的输入用不同的算法计算。不得不问的是，非一致性能在多大程度上提高计算效率？NP-完备问题是否可以由多项式时间的非一致模型判定？对后一个问题，我们会给出一个有条件的否定答案。

nonuniform

电路模型是非一致模型，有非常简单的计算结构。电路模型的最大特点是，它的计算复杂度和它的组合复杂度是一致的。一个电路由门和它们之间的连线组成。一个门有 n 个扇入（输入）和一个扇出（输出）。一个有 n 个输入、单输出的电路是一个有向无圈图，它有 n 个扇入为 0 的源，有一个扇出为 1

circuit model

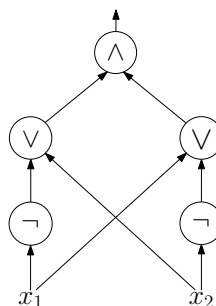
gate

fan-in, fan-out

source, sink

monotone

的池。基本的门有与门、或门、非门，分别用 $\wedge$ 、 $\vee$ 和 $\neg$ 标注。一个电路等价于一个命题公式，其输入为布尔值。一个不包含非门的电路称为单调电路。图4.1定义了一个计算 $x_1 = x_2$ 的电路。用 C, C', C_1 等表示电路，用 $C(x)$ 表示电路 C 在输入 x 上的计算输出。

图 4.1 计算 $x_1 = x_2$ 的电路

circuit family

一个电路族由一个无穷电路系列 $\{C_n\}_{n \in \mathbf{N}}$ 构成，其中 C_n 有 n 个输入。若有问题 L 满足：对所有 $x \in \{0, 1\}^*$ ， $L(x) = 1$ 当且仅当 $C_{|x|}(x) = 1$ ，称电路族 $\{C_n\}_{n \in \mathbf{N}}$ 接受 L 。

例 4.1. 设 $\mathbb{R} = \{1^n \mid n \in R, \text{ 且 } R \text{ 是一个非递归的递归可枚举集}\}$ 。语言 $\mathbb{R}$ 不可判定，但有个电路族接受它。

例4.1表明，基于非一致特性，电路模型能接受一些不可判定的语言。

circuit complexity

电路 C 的电路复杂性指的是它含有的门的个数。用 $|C|$ 表示 C 的电路复杂性。设 $S : \mathbf{N} \rightarrow \mathbf{N}$ 。语言 L 在 $\mathbf{SIZE}(S(n))$ 中当且仅当存在接受 L 的电路族 $\{C_n\}_{n \in \mathbf{N}}$ 使得对所有 $n \in \mathbf{N}$ 都有 $|C_n| \leq S(n)$ 。不同于 $\mathbf{TIME}(_)$ 和 $\mathbf{SPACE}(_)$ ，等式 $\mathbf{SIZE}(3S(n)) = \mathbf{SIZE}(S(n))$ 不成立。

4.1 电路谱系定理

Shannon

香浓很早就证明了绝大多数布尔函数的电路复杂性是指数的 [52]。下述香浓定理的证明是本书中第一个用概率方法证明的例子。

定理 4.1 (香浓定理). 多数 n -元布尔函数的电路复杂性大于 $\frac{2^n}{n} - o(\frac{2^n}{n})$ 。

证明. 固定一个输出门。在一个有 n 输入和 S 个门的电路里, 每个输入有 $S+n+2$ 种选择, 每个门最多有两个输入、三种类型, 总的可能数是 $(S+n+2)^{2S}3^S$ 。这要除以对 $S-1$ 个门的排列数。因此, 由 S 个门组成的电路所计算的函数的个数不超过

$$\begin{aligned}
 \frac{(S+n+2)^{2S}3^S}{(S-1)!} &< \frac{(S+n+2)^{2S}(3e)^S}{S^S} S \\
 &= \left(1 + \frac{n+2}{S}\right)^S (3e(S+n+2))^S S \\
 &< (e^{\frac{n+2}{S}} 3e(S+n+2))^S S \\
 &< (3e^2(S+n+2))^S S \\
 &< (6e^2 S)^S S.
 \end{aligned}$$

设 $(6e^2 S)^S S$ 为 2^{2^n} 的 ϵ -部分, 应有 $(7e^2 S)^S \geq \epsilon 2^{2^n}$ 。由此推出,

$$S(\log(7e^2) + \log S) \geq 2^n - \log \epsilon^{-1}. \quad (4.1.1)$$

易见, $S \leq \frac{2^n}{n} - \log(\frac{1}{\epsilon})$ 和 (4.1.1) 矛盾。 $\square$

香浓定理并未给出最难的布尔函数的电路复杂性的任何上界。卢帕诺夫研究了这个问题 [37]。

Lupanov

定理 4.2. 最难的 n -元布尔函数的电路复杂性小于 $\frac{2^n}{n} + o(\frac{2^n}{n})$ 。

卢茨则给出了最难的布尔函数的电路复杂性的一个下界 [38], 改进了香浓定理提到的下界。

Lutz

定理 4.3. 存在 $c < 1$, 对足够大的 n , 最难的 n -元布尔函数的电路复杂性大于 $\frac{2^n}{n} (1 + c \frac{\log n}{n})$ 。

所以最难的 n -元布尔函数的电路复杂性远大于 $\frac{2^n}{n}$ 。弗兰森和米尔特森进一步改进了卢帕诺夫和卢茨的上下界 [17], 给出了更精细的界。

Frandsen
Miltersen

定理 4.4. 最难的 n -元布尔函数 f 的电路复杂性 $|C_f|$ 满足下列不等式:

$$\frac{2^n}{n} \left(1 + \frac{\log n}{n} - O\left(\frac{1}{n}\right)\right) \leq |C_f| \leq \frac{2^n}{n} \left(1 + 3 \frac{\log n}{n} + O\left(\frac{1}{n}\right)\right).$$

用上述定理给出的上下界，可以证明电路谱系定理。与时间谱系定理和空间谱系定理相比，电路谱系定理更加细致。

定理 4.5 (电路谱系定理). 若有 $\epsilon > 0$ 满足 $n < (2 + \epsilon)S(n) < S'(n) \ll 2^n/n$, 则有 $\mathbf{SIZE}(S(n)) \subsetneq \mathbf{SIZE}(S'(n))$ 。

证明. 设

$$m = \max m. \left(S'(n) \geq \left(1 + \frac{\epsilon}{4}\right) \frac{2^m}{m} \right). \quad (4.1.2)$$

定理的前提蕴含下述不等式:

$$S(n) < \left(1 - \frac{\epsilon}{4 + 2\epsilon}\right) \frac{2^m}{m}. \quad (4.1.3)$$

假设 (4.1.3) 不成立, 会有

$$(2 + \epsilon)S(n) \geq \left(2 + \frac{\epsilon}{2}\right) \frac{2^m}{m} > \left(1 + \frac{\epsilon}{4}\right) \frac{2^{m+1}}{m+1}.$$

此不等式与 (4.1.2) 矛盾. 设 $\mathcal{B}_n$ 为所有只依赖于前 m 个输入的 n -元布尔函数. 从定理4.2和 (4.1.2) 推出, 对足够大的 n 有 $\mathcal{B}_n \subseteq \mathbf{SIZE}(S'(n))$. 从定理4.3和 (4.1.3) 推出, 对足够大的 n 有 $\mathcal{B}_n \not\subseteq \mathbf{SIZE}(S(n))$. 定理得证. $\square$

4.2 一致电路

uniform

电路族 $\{C_n\}_{n \in \mathbb{N}}$ 是一致的当且仅当存在一个将 1^n 映射到 C_n 的隐式对数空间可计算函数. 一致的电路族对 $\mathbf{P}$ 给出了一个很有用的等价刻画。

定理 4.6. 一个语言 L 可被一致的电路族接受当且仅当 $L \in \mathbf{P}$ 。

证明. 设 L 可被一致的电路族接受. 当输入 x 时, 隐式对数空间可计算函数在多项式时间产生一个多项式大小的电路 $C_{|x|}$, 然后在多项式时间完成 $C_{|x|}(x)$ 的计算. 因此 $L \in \mathbf{P}$ 。

设具有一条读写带的图灵机 $\mathbb{M}$ 在多项式时间 $T(n)$ 判定 L . 设输入 x 的长度为 n , 设 $t = T(n)$. 用 $T_0, \dots, T_t$ 表示 $M(x)$ 计算的格局序列. 我们假定每个 T_i 包含了 $t + 1$ 个格子里的符号. 用 T_{ij} 表示 T_i 中第 j -个符号, 这里 $0 \leq j \leq t + 1$. 显然有 $T_{i0} = \triangleright$ 和 $T_{i(t+1)} = \square$. 为了便于在对数空间输出产生的电路, 我们将 T_{ij} 换成一个三元组 (s, h, q) , 其中 s 就是所指格子中的符号,

Circuit Hi
Theorem

h 取布尔值, 表示读写头位置, q 是格局所处的状态。当 $h = 1$ 时, 读写头指向此格; 当 $h = 0$ 时, 读写头不指向此格。第 $t+1$ 个格局和 t 个格局有如下的关系: 对任意 $i, j \in [t]$, T_{ij} 可从 $T_{(i-1)(j-1)}$ 、 $T_{(i-1)j}$ 、 $T_{(i-1)(j+1)}$ 中算出来, 见图4.3。这个计算只依赖于 $\mathbb{M}$, 不依赖于 x , 可以用一个电路 C_{ij} 实现。不难

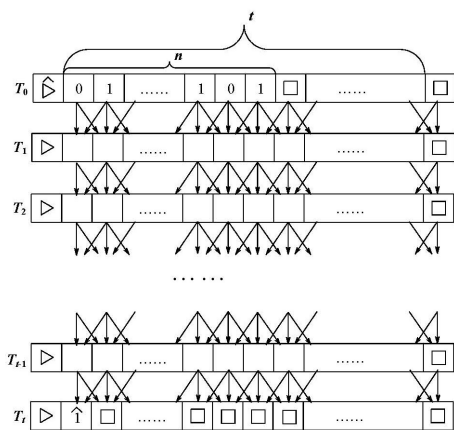


图 4.2 相邻格局的关系

看出, 对所有的 $i, j \in [t]$, 这个计算是同一个计算, 可以用同一个电路 C 实现。电路 C_{1j} 的输入是起始格局 (的二进制编码)。对于 $i > 1$ 和 $1 < j < t$, 电路 C_{ij} 的输入是 $C_{(i-1)(j-1)}$, $C_{(i-1)j}$, $C_{(i-1)(j+1)}$ 的输出。对于 $i > 1$, 电路 C_{i1} 的输入是 $\triangleright$ 、 $C_{(i-1)j}$ 的输出和 $C_{(i-1)(j+1)}$ 的输出, 电路 C_{it} 的输入是 $C_{(i-1)(t-1)}$ 的输出、 $C_{(i-1)t}$ 的输出和 $\square$ 。连接之后的电路看上去如图4.3所示。此电路可接受输入长度是 n 的 L 中的串。从 C_{t1} 出发, 可以用一个类似于深度优先树遍历的算法将最终电路的临接矩阵在对数空间里输出。□

用表示 CKT-SAT 电路版本的可满足性问题。一个二进制串在 CKT-SAT 里当且仅当它是一个电路 C 的编码并且 $\exists u \in \{0, 1\}^n. C(u) = 1$, 这里 n 是 C 的输入长度。

引理 4.1. 对所有 $L \in \mathbf{NP}$, 有 $L \leq_L \text{CKT-SAT}$ 。

证明. 设 $L \in \mathbf{NP}$, p 是一个多项式, $\mathbb{M}$ 是个多项式时间图灵机, 满足:

$$x \in L \text{ 当且仅当 } \exists u \in \{0, 1\}^{p(|x|)}. \mathbb{M}(x, u) = 1.$$

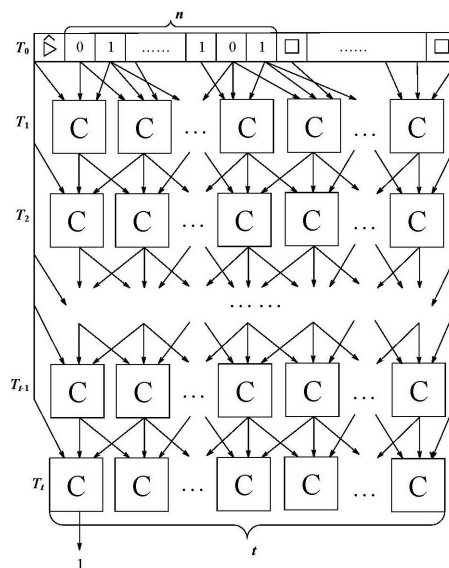


图 4.3 计算 P-问题的电路

用定理4.6将 $M(x, u)$ 在对数空间归约到计算 $M(x, u)$ 的电路 C ，然后将输入 x 和 C 做硬连接。这给出了从 L 到 $CKT-SAT$ 的对数归约。 $\square$

引理 4.2. $CKT-SAT \leq_L SAT$.

证明. 设 C 为输入电路。为 C 的每个输入、每个门、每个输出引入一个布尔变量。输入的 C 包含了所有门的输入编码和输出编码，可将这些编码作为相应变量的编码。若一个与门的输入为 x, y ，输出为 z ，引入布尔公式 $z = x \wedge y$ ，此公式等价于合取范式 $(\bar{x} \vee \bar{y} \vee z) \wedge (\bar{z} \vee x) \wedge (\bar{z} \vee y)$ 。或门和非门也同样可用合取范式表示。这些合取范式的大小是对数长的，可逐个输出。 $\square$

引理4.1和引理4.2指出了—个值得关注的现象：库克-莱文归约是对数空间归约。

4.3 P_{poly}

图灵机在处理某一长度的输入时，如果能得到一些建议，它或许能更高效地进行计算。一个建议就是 $\{0, 1\}^*$ 中—元素。—个建议类型是—函数 $a : \mathbf{N} \rightarrow \mathbf{N}$ 。

advice

一个满足建议类型 a 的建议族是一无穷建议序列 $\{\alpha_n\}_{n \in \mathbf{N}}$, 使得对任意 $n \in \mathbf{N}$, 有 $|\alpha_n| = a(n)$ 。一个具有建议类型 $a(n)$ 的图灵机 $\mathbb{M}$ 需要一类型为 $a(n)$ 的建议族 $\{\alpha_n\}_{n \in \mathbf{N}}$, 当接受长度为 n 的输入串 x 时, 计算 $\mathbb{M}(x, \alpha_n)$ 。所有被具有建议类型 $a(n)$ 的图灵机在 $T(n)$ 时间内判定的语言构成

$$\mathbf{DTIME}(T(n))/a(n)。$$

带建议的图灵机是由卡普和立普顿在文献 [28] 中引入的。用带有建议的图灵机可定义相应的复杂性类。

Lipton

- $L \in \mathbf{P}_{/\text{poly}}$ 当且当 L 可被具有多项式建议函数的多项式时间图灵机判定。
- $L \in \mathbf{NP}_{/\text{poly}}$ 当且当 L 可被具有多项式建议函数的多项式时间非确定图灵机判定。
- $L \in \mathbf{L}_{/\text{poly}}$ 当且当 L 可被具有多项式建议函数的对数空间图灵机判定。

卡普和立普顿证明了下面的等价性定理。

定理 4.7. $\mathbf{P}_{/\text{poly}} = \bigcup_{c,c'} \mathbf{SIZE}(cn^{c'})$.

证明. 设 L 由多项式大小的电路族 $\{C_n\}_{n \in \mathbf{N}}$ 接受。当输入为长度为 n 的 x , 将 C_n 的二进制编码作为建议, 在多项式时间完成 $C(x)$ 的计算。故 $L \in \mathbf{P}_{/\text{poly}}$ 。

反向包含关系的证明使用定理4.6证明中给出的归约。输入 x , 使用建议 $\alpha(|x|)$ 时图灵机的计算可转换成电路的计算, 再将 $\alpha(|x|)$ 和电路做硬连接得到输入长度为 $|x|$ 的多项式大小的电路。此过程为隐式对数空间可计算。□

4.4 NC 和 AC

本节讨论在并行计算领域众所周知的高效并行计算类。大规模并行计算的特点是: 众多的处理器通过互联构成一个计算网络, 处理器同步计算, 网络通信延迟时间不超过处理器数的对数。大规模并行计算的目标是要在 polylog 时间内完成计算任务。称一个问题具有高效并行算法当且当在输入长度为 n 时, 一台并行处理机可用 $n^{O(1)}$ 多个处理器在 $\log^{O(1)}(n)$ 时间里完成计算。看三个并行算法的例子。

$O(\log^{O(1)}(n))$

$O(1)$ 指代一个常数

1. 给定数 $x_1, \dots, x_n$, 要求计算连续和

$$x_1, x_1 + x_2, x_1 + x_2 + x_3, \dots, x_1 + x_2 + \dots + x_n \quad (4.4.1)$$

用一步并行计算可得: $x_1 + x_2, x_3 + x_4, \dots, x_{n-1} + x_n$ 。用归纳法, 可以在 $2(\log(n) - 1)$ 步并行计算得到

$$x_1 + x_2, x_1 + x_2 + x_3 + x_4, x_1 + x_2 + x_3 + x_4 + x_5 + x_6, \dots。$$

再用一步并行计算就可算出 (4.4.1)。忽略网络延迟, 我们用了 $2\log(n)$ 个并行步算出了 (4.4.1)。

2. 上述并行算法的思想可用来加速二进制加法。假设 $a_n a_{n-1} \dots a_1 a_0$ 与 $b_n b_{n-1} \dots b_1 b_0$ 为二进制数, 且 $a_n = b_n = 0$ 。设 $0 \leq i \leq n-1$, 用 x_i 表示第 i -位加法产生的进位。定义

$$g_i = a_i \wedge b_i, \text{ 进位产生位,}$$

$$p_i = a_i \vee b_i, \text{ 进位传播位。}$$

易见, $x_i = g_i \vee (p_i \wedge x_{i-1}) = g_i \vee (p_i \wedge g_{i-1}) \vee (p_i \wedge p_{i-1} \wedge x_{i-2})$ 。引入二元运算

$$(g', p') \odot (g, p) = (g \vee (p \wedge g'), p \wedge p')。$$

设 $(g_0, p_0) = (a_0 \wedge b_0, 0)$ 。利用上述计算连续和的方法, 进位 $x_0, \dots, x_{n-1}$ 可在 $2\log n$ 并行步通过计算下列序列得到。

$$(g_0, p_0), (g_0, p_0) \odot (g_1, p_1), (g_0, p_0) \odot (g_1, p_1) \odot (g_2, p_2), \dots。$$

最后用一个并行步计算 $a_1 \vee b_1 \vee x_0, \dots, a_n \vee b_n \vee x_{n-1}$, 就得到 $a_n a_{n-1} \dots a_1 a_0$ 与 $b_n b_{n-1} \dots b_1 b_0$ 的和。此并行算法称为超前进位加法。

carry lookahead

3. 在计算两个 $(n \times n)$ -矩阵相乘时, 可以为每个点积计算分配 n 个处理器做两个数的并行乘法运算, 然后再进行两个数的并行加法运算, 这需要 n^3 个处理器。加法运算可用超前进位加法在对数时间内可完成。利用超前进位加法, 乘法运算可在对数平方时间内完成。因此矩阵乘法有高效并行算法。

电路中的每一层的门都是可以并行计算的。电路越矮，并行度越高。因此用电路模型可以完美地描述并行性。设 $H : \mathbf{N} \rightarrow \mathbf{N}$ 。称 $H(n)$ 为电路族 $\{C_n\}_{n \in \mathbf{N}}$ 的高度函数当且仅当对所有 $n \in \mathbf{N}$ ，电路 C_n 的高度不超过 $H(n)$ 。

定义 4.1. 语言 L 在 $\mathbf{NC}^d$ 中当且仅当 L 可由一个高度函数为 $O(\log^d(n))$ 的一致电路族所接受。高效并行计算类 $\mathbf{NC}$ 定义为 $\bigcup_{d \in \mathbf{N}} \mathbf{NC}^d$ 。

一个 $\mathbf{NC}^d$ 中的语言显然可以由一台高效并行计算机计算。反之，只要每个处理器完成的是常量的计算，一台高效并行计算机就可以由 polylog 高度的一致电路接受。

$\mathbf{NC}^d$ 有个常用的变种， $\mathbf{AC}^d$ 。在前者的定义中，与门和或门的扇入为 2，在后者的定义中，与门和或门的扇入为大于等于 2。定义

$$\mathbf{AC} = \bigcup_{d \in \mathbf{N}} \mathbf{AC}^d.$$

一个扇入为 k 的与门可以用一个由扇入为 2 的与门构成的电路实现，该电路的高度为 $\log(k)$ 。因此有 $\mathbf{NC}^i \subseteq \mathbf{AC}^i \subseteq \mathbf{NC}^{i+1}$ 。

引理 4.3. $\mathbf{AC} = \mathbf{NC}$ 。

在一个允许任意扇入的电路里，若一个与门的输入均为与门的输出，可以用一个与门代替这些与门；若一个或门的输入均为或门的输出，可以用一个或门代替这些或门。因此，在一个允许任意扇入的电路里，可以将与门、或门、非门交替分层安排，便于用归纳法证明一些性质。这是引入 $\mathbf{AC}$ 的一个好处。

回到本节开头讨论的矩阵相乘。如果我们把注意力集中在布尔矩阵，证明会有更简单。

例 4.2. 设 $\mathfrak{A}$ 为 $(n \times n)$ -布尔矩阵。若想计算布尔矩阵自乘

$$(\mathfrak{A}^2)_{ij} = \bigvee_{k=1}^n \mathfrak{A}_{ik} \wedge \mathfrak{A}_{kj},$$

可用 n^3 与门在一步计算所有的 $\mathfrak{A}_{ik} \wedge \mathfrak{A}_{kj}$ ，然后用最多 $n^2(n-1)$ 个或门在 $\log(n)$ 步计算所有 $(\mathfrak{A}^2)_{ij}$ ，全部门可以安排在 $\log(n)+1$ 层。因此有下述重要的结论：

- $\mathfrak{A}^2 \in \mathbf{NC}^1$ 。
- $\mathfrak{A}^n \in \mathbf{NC}^2$ 。

一个有 n 个结点的图 G 的临接矩阵 $\mathfrak{A}_G$ 就是一个布尔矩阵, $\mathfrak{A}_G^{n-1}$ 中的元素 $(\mathfrak{A}_G^{n-1})_{i,j} = 1$ 当且仅当从结点 i 到结点 j 有一条路径。因此有下述引理。

引理 4.4. 图的可达性问题在 $\mathbf{NC}^2$ 中。

4.5 P-完备性

一个 P-问题是否有高效并行算法提供了研究 $\mathbf{P}$ 的新视角。是否每个 $\mathbf{P}$ 中的问题都有高效并行算法? 和复杂性理论中类似的问题一样, 这肯定不是一个容易回答的问题。我们能做的, 就是和我们在研究 $\mathbf{NP}$ 类时所做的一样, 刻画出 $\mathbf{P}$ 中最不可能具有并行高效算法的一类问题。首先必须回答的是: 应该用什么标准比较问题的可高效并行化? 卡普归约不是个选项, 不可能用多项式时间归约比较多项式时间可解问题的任何特性。下述引理给出了一个答案。

引理 4.5. 隐式对数空间可计算函数可高效并行化。

证明. 隐式对数空间可计算函数输出的每一位都可在对数空间算出来, 这些位的计算当然是可并行进行的。每一位的计算可高效并行化, 这是因为有两点:

1. 对数空间可计算函数的格局图可用一个多项式大小的临接矩阵表示。因为格局的大小是对数的, 所以可以在对数时间里完成计算, 见引理3.1。因此, 对数空间可计算函数的格局图的临接矩阵可被高效并行地产生。
2. 计算每一位的值就相当于判定图的可达性, 根据引理4.4, 可达性判定具有高效并行算法。

根据对数空间归约的传递性, 引理所述性质成立。 □

基于引理4.5和对数归约的可复合性, 引入下述定义是有意义的。

定义 4.2. $\mathbf{P}$ 中语言 A 是 P -完备的当且仅当对任意 $\mathbf{P}$ 中语言 A' 有 $A' \leq_L A$ 。

定义4.2和定理4.6证明中构造的电路告诉我们如何定义一个 P -完备语言。电路求值问题 **Circuit-Eval** 包含了所有有序对 (C, v) , 其中 C 为有 n 个

输入的电路, $v \in \{0,1\}^n$ 并且 $C(v) = 1$ 。此完备性结果可进一步加强。设 **Monotone-Circuit-Eval** 是 **Circuit-Eval** 的子集, 其中的电路都是单调电路, 即电路不含非门。

引理 4.6. *Monotone-Circuit-Eval 是 P-完备的。*

证明. 我们可以将定理4.6证明中构造的电路做些变换, 使其成为单调的。用德·摩根律可将电路 C 中的非门向下推, 直到输入端, 得到电路 C^- 。设 $\overline{C}$ 是在 C 的输出端加了个非门后得到的电路。按同样方法可将 $\overline{C}$ 中的非门向下推, 直到输入端, 得到电路 $\overline{C}^-$ 。用 C^- 和 $\overline{C}^-$ 构造最终的电路。注意, 必须将每个输入 $v = v_1 \dots v_n$ 变成两倍长的输入 $v_1 \overline{v_1} \dots v_n \overline{v_n}$ 。□

De Morgan law

下述定理表明 **NC** 的确是 **P** 中最难的那类问题。

定理 4.8. 设 L 是 **P**-完备的。 $L \in \mathbf{NC}$ 当且仅当 $\mathbf{P} = \mathbf{NC}$ 。

证明. 对数空间归约有高效并行算法, 两个高效并行算法的复合还是高效并行算法。□

高效并行性给出了对 **P** 的内部结构的一个划分。下述结果只是给出了一个可供想象的结构图, 我们压根就不知道其中的任何一个包含关系是否为严格的。

定理 4.9. $\mathbf{NC}^1 \subseteq \mathbf{L} \subseteq \mathbf{NL} \subseteq \mathbf{NC}^2 \subseteq \dots \subseteq \mathbf{NC}^i \subseteq \dots \mathbf{P}$ 。

证明. 只有两个包含关系需要验证。首先, **NL**-完备问题 **Reachability** 在 $\mathbf{NC}^2$ 中, 故 $\mathbf{NL} \subseteq \mathbf{NC}^2$ 。其次, 设 $\{C_n\}_{n \in \mathbf{N}}$ 接受 $L \in \mathbf{NC}^1$, 设 $x \in \{0,1\}^n$ 。下面这个证明思路读者应该熟悉了: 用深度优先法遍历所有的门, 计算出访问过的门的输出值。需要用一个长度为 $\log(n)$ 的计数器记住当前访问的门的位位置。因此 $\mathbf{NC}^1 \subseteq \mathbf{L}$ 。□

第 5 章 多项式谱系

定理2.1证明了, $L \in \mathbf{NP}$ 当仅当存在多项式 $p_1 : \mathbf{N} \rightarrow \mathbf{N}$ 和多项式时间图灵机 $\mathbb{M}$, 对任意 $x \in \{0, 1\}^*$, $x \in L$ 当仅当 $\exists u_1 \in \{0, 1\}^{p_1(|x|)}. \mathbb{M}(x, u_1) = 1$ 。用库克-莱文归约, 可将 $\mathbb{M}(x, u_1) = 1$ 换成一合取范式 $\psi(x, u_1)$ 。如果读者熟悉递归论中的算术谱系 [49], 很可能会问: 由如下形式的逻辑公式

$$\forall u_2 \in \{0, 1\}^{p_2(|x|)}. \exists u_1 \in \{0, 1\}^{p_1(|x|)}. \psi(x, u_1, u_2) \quad (5.0.1)$$

定义的语言类是什么? 公式 (5.0.1) 中, p_2 为多项式, $\psi(x, u_1, u_2)$ 为合取范式。有没有自然的问题属于这类? 有没有自然的语言属于如下形式的逻辑公式

$$\exists u_1 \in \{0, 1\}^{p_1(|x|)}. \forall u_2 \in \{0, 1\}^{p_2(|x|)}. \psi(x, u_1, u_2) \quad (5.0.2)$$

arithmetic hierarchy

定义的类? 更一般地, 如何建立算术谱系的复杂性版本? 早在 1949 年, 香浓已经注意到用类似逻辑公式刻画的问题是困难的。在文献 [53] 中, 他写道“电路合成异常困难。更难的是, 证明一个电路是满足一定函数功能的电路中最经济的”。这段话里提到的问题可定义如下:

$$\mathbf{MIN-DNF} = \{ \langle \varphi, k \rangle \mid \varphi \text{ DNF} \wedge \forall \text{ DNF } \psi. |\psi| > k \vee \exists u. \varphi(u) \Leftrightarrow \psi(u) \}.$$

判定一个析取范式的极小性就是问 $(\varphi, |\varphi|)$ 是否在 $\mathbf{MIN-DNF}$ 中。易见, 定义这个问题的逻辑公式形如 (5.0.1)。 $\mathbf{MIN-DNF}$ 在语义上的补问题, 即

$$\overline{\mathbf{MIN-DNF}} = \{ \langle \varphi, k \rangle \mid \varphi \text{ DNF} \wedge \exists \text{ DNF } \psi. |\psi| \leq k \wedge \forall u. \varphi(u) \Leftrightarrow \psi(u) \}$$

由形如 (5.0.2) 的逻辑公式所定义。注意, 在 $\overline{\mathbf{MIN-DNF}}$ 的定义中, $\varphi(u) \Leftrightarrow \psi(u)$ 是合取范式, 公式 $\forall u. \varphi(u) \Leftrightarrow \psi(u)$ 相当于判定 $\varphi \Leftrightarrow \psi$ 是否为永真式。因此 $\overline{\mathbf{MIN-DNF}}$ 可用一个多项式时间带神谕的非确定图灵机通过询问 $\mathbf{SAT}$ 来判

定, 即 $\overline{\text{MIN-DNF}} \in \mathbf{NP}^{\text{SAT}}$ 。直观上而言, $\overline{\text{MIN-DNF}}$ 比 NP-问题要难。不难看出, $\text{MIN-DNF} \in \mathbf{coNP}^{\text{SAT}}$, 所以 MIN-DNF 只可能更难。

斯托克迈尔和梅耶在上世纪七十年代初研究了如何建立复杂性理论的“算术谱系” [54], 即多项式谱系, 并指出了一些在谱系中第二层的问题, 其中一些问题的完备性直至上世纪末才由乌曼斯[66] 证明的。本章介绍多项式谱系的几个等价刻画和无限谱系假设。

Umans

5.1 多项式谱系

简单地说, 多项式谱系就是 $\mathbf{NP} \subseteq \mathbf{NP}^{\mathbf{NP}} \subseteq \mathbf{NP}^{\mathbf{NP}^{\mathbf{NP}}} \subseteq \dots$ 。我们相信, 这里面的每个包含关系都是严格的。为便于归纳证明, 我们用带神谕的图灵机定义复杂性类 Σ_i^p 、 Π_i^p 、 Δ_i^p 如下:

polynomial hierarchy

$$\begin{aligned}\Sigma_0^p &= \mathbf{P}, \\ \Sigma_{i+1}^p &= \mathbf{NP}^{\Sigma_i^p}, \\ \Delta_{i+1}^p &= \mathbf{P}^{\Sigma_i^p}, \\ \Pi_i^p &= \overline{\Sigma_i^p}.\end{aligned}$$

根据定义, 有

$$\Sigma_i^p \subseteq \Delta_{i+1}^p \subseteq \Sigma_{i+1}^p. \quad (5.1.1)$$

因为 $\overline{\Delta_i^p} = \Delta_i^p$, 所以

$$\Pi_i^p \subseteq \Delta_{i+1}^p \subseteq \Pi_{i+1}^p. \quad (5.1.2)$$

(5.1.1) 和 (5.1.2) 的一个直接结论是

$$\Sigma_i^p \cap \Pi_i^p \subseteq \Delta_{i+1}^p \subseteq \Sigma_{i+1}^p \cap \Pi_{i+1}^p.$$

同样根据定义, 有 $\Pi_{i+1}^p = \mathbf{coNP}^{\Sigma_i^p}$ 。在第 0 层, 有 $\Sigma_0^p = \Pi_0^p = \Delta_0^p = \mathbf{P}$ 。在第 1 层, 有 $\Sigma_1^p = \mathbf{NP}$ 和 $\Pi_1^p = \mathbf{coNP}$ 。多项式谱系定义为

$$\mathbf{PH} = \bigcup_{i \geq 0} \Sigma_i^p. \quad (5.1.3)$$

由 (5.1.1) 和 (5.1.1) 知,

$$\mathbf{PH} = \bigcup_{i \geq 0} \Pi_i^p = \bigcup_{i \geq 0} \Delta_i^p.$$

记 $\mathbf{PH}_i = \Sigma_i^P \cup \Pi_i^P$ ，并称 $\mathbf{PH}_i$ 为多项式谱系的第 i 层。我们对 $\mathbf{PH}_3$ 知道的不多，已知的一些自然问题在 $\mathbf{PH}_2$ 里。乌曼斯在文献 [66] 中证明了一些组合问题是 Σ_2^P -完备的，包括 $\overline{\text{MIN-DNF}}$ 和如下定义的问题。

SUCCINCT SET COVER.

给定 3-DNF 集 $S = \{\varphi_1, \dots, \varphi_m\}$ 和自然数 k ，是否存在大小不超过 k 的子集 $S' \subseteq \{1, \dots, m\}$ 使得 $\bigvee_{i \in S'} \varphi_i$ 为永真式？

接着看一个在 Δ_2^P 中的问题： $\text{EXACT-IS} = \{\langle G, k \rangle \mid G \text{ 中最大独立集有 } k \text{ 各结点}\}$ 。要说明 (G, k) 在 EXACT-IS 中，只需说明：存在一个大小为 k 的独立集，任意独立集不小于 k 。这句话也可以这么说：任意独立集不小于 k ，存在一个大小为 k 的独立集。因此 $\text{EXACT-IS} \in \Delta_2^P$ 。不难看出， $\text{EXACT-IS} = \text{IS} \cap \overline{\text{IS}}$ 。所以事实上 $\text{EXACT-IS} \in \mathbf{NP} \cap \mathbf{coNP} \subseteq \Delta_2^P$ 。

5.2 谱系的逻辑刻画

根据库克-莱文归约， $\mathbf{P}$ 中的神谕都有逻辑刻画。根据定理 2.1， $\mathbf{NP}$ 和 $\mathbf{coNP}$ 中的神谕也有等价的逻辑刻画。用归纳法，应该可以给出 Σ_j^P 的逻辑刻画。斯托克迈尔和拉索尔研究了多项式谱系的逻辑定义 [59, 68]，后者证明了用逻辑定义的多项式谱系和用神谕定义的多项式谱系是相同的。

Wrathall

定理 5.1. 设 $i \geq 1$ 。下述命题成立。

1. $L \in \Sigma_i^P$ 当且仅当存在多项式时间图灵机 $\mathbb{M}$ 和多项式 q ，对任意 $x \in \{0, 1\}^*$ ， $x \in L$ 与下述命题等价：

$$\exists u_1 \in \{0, 1\}^{q(|x|)} \forall u_2 \in \{0, 1\}^{q(|x|)} \dots Q_i u_i \in \{0, 1\}^{q(|x|)}. \mathbb{M}(x, \tilde{u}) = 1.$$

2. $L \in \Pi_i^P$ 当且仅当存在多项式时间图灵机 $\mathbb{M}$ 和多项式 q ，对任意 $x \in \{0, 1\}^*$ ， $x \in L$ 与下述命题等价：

$$\forall u_1 \in \{0, 1\}^{q(|x|)} \exists u_2 \in \{0, 1\}^{q(|x|)} \dots Q_i u_i \in \{0, 1\}^{q(|x|)}. \mathbb{M}(x, \tilde{u}) = 1.$$

证明. 只证 1. 设 $\mathbb{M}$ 为多项式时间图灵机， q 为多项式，满足： $x \in L$ 当且当

$$\exists u_1 \in \{0, 1\}^{q(|x|)} \dots Q u_{i+1} \in \{0, 1\}^{q(|x|)}. \mathbb{M}(x, u_1, \dots, u_{i+1}) = 1.$$

设计多项式时间非确定图灵机如下：当输入为 x 时，猜测 $u_1 \in \{0, 1\}^{q(|x|)}$ ，问一个神谕下述量化布尔公式是否为真，然后将得到的答复作为结果输出。

$$\forall u_2 \in \{0, 1\}^{q(|x|)} \dots Q u_{i+1} \in \{0, 1\}^{q(|x|)} . \mathbb{M}(x, u_1, \dots, u_{i+1}) = 1. \quad (5.2.1)$$

根据归纳假定，有一个 Σ_i^P 中的神谕能够回答公式 (5.2.1) 是否为真。

设 L 被使用神谕 $A \in \Sigma_i^P$ 的多项式时间非确定图灵机 N 所接受。根据库克-莱文定理， $x \in L$ 当且仅当

$$\exists z_1 \dots z_n . \exists s_1, \dots, s_m, d_1, \dots, d_k . \exists w_1, \dots, w_k . \psi_1 \wedge \psi_2, \quad (5.2.2)$$

其中 $z_1 \dots z_n$ 是库克-莱文归约引入的变量，公式 ψ_1 是合取范式，表示 $N(x)$ 的计算进行了 m 步，依次使用了迁移函数 $\delta_{s_1}, \dots, \delta_{s_m}$ ，向神谕 A 提出了问题 $w_1, \dots, w_k$ ，得到神谕的答案 $d_1, \dots, d_k$ ；公式 ψ_2 说明 $d_1, \dots, d_k$ 是正确答案，即

$$\left(\bigwedge_{i \in [k]} d_i = 1 \Rightarrow w_i \in A \right) \wedge \left(\bigwedge_{i \in [k]} d_i = 0 \Rightarrow w_i \in \overline{A} \right). \quad (5.2.3)$$

根据归纳，在 (5.2.3) 中的 $w_i \in \overline{A}$ 可用形如 $\forall \exists \forall \dots _$ 的公式替换。所以 (5.2.2) 是满足要求的公式。注意， $(\forall u. \varphi) \wedge (\forall v. \psi)$ 和 $\forall u. \forall v. (\varphi \wedge \psi)$ 等价；同样地， $(\exists u. \varphi) \wedge (\exists v. \psi)$ 和 $\exists u. \exists v. (\varphi \wedge \psi)$ 等价。□

受定理5.1启发，定义 $\Sigma_i \text{SAT}$ 为 TQBF 的子集，包含所有值为真的如下形式的量化布尔公式：

$$\exists u_1 \forall u_2 \dots Q_i u_i . \varphi(u_1, \dots, u_i),$$

其中 $\varphi(u_1, \dots, u_i)$ 是合取范式。按对偶性， $\overline{\Sigma_i \text{SAT}}$ 应该包含所有值为真的如下形式的量化布尔公式：

$$\forall u_1 \exists u_2 \dots Q_i u_i . \varphi(u_1, \dots, u_i)。$$

下述定理应该不会让读者感到惊讶。

定理 5.2. $\Sigma_i \text{SAT}$ 是 Σ_i^P -完备的。

证明. $\Sigma_i\text{SAT} \in \Sigma_i^p$ 由定理5.1直接推出。设 $L \in \Sigma_{2i+1}^p$, $x \in L$ 当且仅当

$$\exists u_1 \forall u_2 \dots \exists u_{2i+1}. \mathbb{M}(x, \tilde{u}) = 1.$$

用库克-莱文归约将 $\mathbb{M}(x, \tilde{u}) = 1$ 归约到形如 $\exists z. \varphi$ 的公式。就有 $x \in L$ 当且仅当 $\exists u_1 \forall u_2 \dots \exists u_{2i+1} z. \varphi$ 为真。若 $L \in \Sigma_{2i}^p$, 将定理5.1中的第2个等价性命题用于 $\bar{L} \in \Pi_{2i}^p$, 也可得到从 L 到 $\Sigma_{2i}\text{SAT}$ 的卡普归约。□

定理5.2和定理3.6的一个直接的推论是下面著名的结果。

定理 5.3. $\text{PH} \subseteq \text{PSPACE}$ 。

5.3 谱系的交替机刻画

Chandra, Kozen

钱德拉、科赞和斯托克迈尔在文献 [10] 中引入了交替图灵机, 这类机器模型有不少应用, 其中之一就是给出多项式谱系的又一个等价刻画。

alternating TM, 或
ATM

定义 5.1. 一交替图灵机是非确定图灵机, 其每个状态带有一个标号, 标号为集合 $\{\exists, \forall, \text{accept}, \text{halt}\}$ 中的一个元素。

带标号 **accept** 的状态就是接受状态, 带标号 **halt** 的状态就是停机状态。标号为 $\exists$ 的状态就相当于非确定图灵机里的既非接受也非停机的状态, 在此状态下图灵机不确定地选择下一步计算。直觉上, 在标号为 $\forall$ 的状态下, 交替图灵机要选择所有下一步计算, 换言之, 在 $\forall$ 的状态下机器要引入一个并行计算。设交替图灵机 A 预置了输入 x 。 $A(x)$ 的一棵接受树是满足如下条件的二叉树:

1. 结点为格局, 根结点为 $A(x)$ 的初始格局, 叶结点为 $A(x)$ 的接受格局; 边表示一步计算。
2. 若一个结点的状态标号为 $\exists$, 该节点有一个儿子; 若一个结点的状态标号为 $\forall$, 该节点有两个儿子。

称 A 接受 x , 记为 $A(x) = 1$, 当且仅当 $A(x)$ 有一棵接受树。若对任意 x , $x \in L$ 当且仅当 $A(x) = 1$, 称 A 接受 L 。

交替图灵机能比较贴切地描述博弈过程, 存在状态是我方的状态, 全称状态是对手的状态。直观上, 交替图灵机在多项式时间判定的问题就是图灵机在

多项式空间判定的问题。为了验证这一直觉，得引入交替图灵机定义的时间类和空间类。设 $T: \mathbf{N} \rightarrow \mathbf{N}$ 为时间函数。称 $T(n)$ 为交替图灵机 A 的时间函数当且仅当对任意输入 $x \in \{0, 1\}^*$ ， $A(x)$ 的所有计算路径长度不超过 $T(|x|)$ 。一个语言 L 在 $\mathbf{ATIME}(T(n))$ 里当且仅当存在 c 和接受 L 的具有时间函数 $cT(n)$ 的交替图灵机 A 。设 $S: \mathbf{N} \rightarrow \mathbf{N}$ 为空间函数。称 $S(n)$ 为交替图灵机 A 的空间函数当且仅当对任意输入 $x \in \{0, 1\}^*$ ， $A(x)$ 在所有的计算路径上使用的工作带上的格子数不超过 $S(|x|)$ 。一个语言 L 在 $\mathbf{ASPACE}(S(n))$ 里当且仅当存在 c 和接受 L 的具有空间函数 $cS(n)$ 的交替图灵机 A 。

例 5.1. 用定理 3.5 证明中的算法，容易验证 $TQBF$ 可在平方空间内由交替图灵机判定。只要用标号为 $\exists$ 的状态猜测一个格局，用标号为 $\forall$ 的状态进行并发验证即可。格局的大小是线性的，进行二分的次数也是线性的。所以空间开销是平方的。

用 $\mathbf{ATIME}(_)$ 和 $\mathbf{ASPACE}(_)$ 可定义下述复杂性类：

$$\begin{aligned} \mathbf{AL} &= \mathbf{ASPACE}(\log n), \\ \mathbf{AP} &= \bigcup_{c>0} \mathbf{ATIME}(n^c), \\ \mathbf{APSPACE} &= \bigcup_{c>0} \mathbf{ASPACE}(n^c), \\ \mathbf{AEXP} &= \bigcup_{c>0} \mathbf{ATIME}(2^{n^c}), \\ \mathbf{AEXPSPACE} &= \bigcup_{c>0} \mathbf{ASPACE}(2^{n^c}). \end{aligned}$$

钱德拉、科赞和斯托克迈尔研究了这些类型，给出了这些类型的一个漂亮的刻画。下述定理叙述了他们证明中使用的关键技术。

定理 5.4 (钱德拉-科赞-斯托克迈尔定理). 设 $S(n)$ 、 $T(n)$ 均为时间可构造和空间可构造。下述包含关系成立。

1. $\mathbf{NSPACE}(S(n)) \subseteq \mathbf{ATIME}(S^2(n))$.
2. $\mathbf{ATIME}(T(n)) \subseteq \mathbf{SPACE}(T(n))$.
3. $\mathbf{ASPACE}(S(n)) \subseteq \bigcup_{c>0} \mathbf{TIME}(c^{S(n)})$.

4. $\mathbf{TIME}(T(n)) \subseteq \mathbf{SPACE}(\log T(n))$.

证明. 1. 用萨维奇定理（定理3.5）的证明思路。在格局图中，用标号为 $\exists$ 的状态猜测可达路径的中间结点，然后用标号为 $\forall$ 的状态并行验证长度减半的可达路径的存在性。猜测结点需要 $S(n)$ 时间，递过深度也为 $S(n)$ 。此算法中，需要用到 $S(n)$ 的时间可构造性。

2. 根据可构造性，可以假定格局图是个有向无圈图。用深度优先法遍历格局图，判定是否存在一棵接受树。要用到长度是 $T(n)$ 的计数器，并需假定 $T(n)$ 是空间可构造的。

3. 用深度优先法遍历格局图，判定是否存在一棵接受树。

4. 这个算法再次用到定理4.6的证明里用到的计算 P-问题的一致电路族，见图4.3。假设最后的一个 C （即在图4.3中左下角的 C ）的输出为 1。用标号为 $\exists$ 的状态反向猜测该电路的两组输入，然后验证猜测的输入是否正确；若验证通过，用标号为 $\forall$ 的状态并行地做递过计算。计算过程中，猜测的输入长度是对数的，递归高度的控制要用一个长度是对数的计数器。 $\square$

定理5.4的 1 和 2 及定理3.5蕴含 $\mathbf{AP} = \mathbf{PSPACE}$, $\mathbf{AEXP} = \mathbf{EXPSPACE}$, 等等。定理5.4的 3 和 4 蕴含 $\mathbf{AL} = \mathbf{P}$, $\mathbf{ASPACE} = \mathbf{EXP}$, 等等。这些等式和明显的包含关系可用以下定理总结。

定理 5.5. 下述等式和包含关系成立。

$$\begin{array}{ccccccc}
 \mathbf{AL} & \subseteq & \mathbf{AP} & \subseteq & \mathbf{PSPACE} & \subseteq & \mathbf{AEXP} \dots \\
 \parallel & & \parallel & & \parallel & & \parallel \dots \\
 \mathbf{L} & \subseteq & \mathbf{P} & \subseteq & \mathbf{SPACE} & \subseteq & \mathbf{EXP} \subseteq \mathbf{EXPSPACE} \dots
 \end{array}$$

可以看出，用交替图灵机定义的和用图灵机定义类有个错位对应。定理5.5也证实了我们的直觉，即 $\mathbf{AP} = \mathbf{PSPACE}$ 。

在本节的最后，我们解释如何用交替图灵机刻画多项式谱系。首先引入如下定义： $L \in \Sigma_i \mathbf{TIME}(T(n)) / \Pi_i \mathbf{TIME}(T(n))$ 当且仅当 L 可被一个具有 $O(T(n))$ 时间函数的、初始状态 q_{start} 标号为 $\exists/\forall$ 的交替图灵机 $\mathbf{A}$ 所判定，并且该交替图灵机在计算时标号为 $\exists$ 的状态和标号为 $\forall$ 的状态最多交替 $i - 1$ 次。下述定理给出的就是多项式谱系的交替图灵机刻画。

定理 5.6. 对所有 $i \geq 1$, 下列等式成立:

$$\begin{aligned}\Sigma_i^P &= \bigcup_{c>0} \Sigma_i \mathbf{TIME}(n^c), \\ \Pi_i^P &= \bigcup_{c>0} \Pi_i \mathbf{TIME}(n^c).\end{aligned}$$

证明. Σ_i^P -完备问题 $\Sigma_i \mathbf{SAT}$ 可被一个多项式时间的、初始状态 q_{start} 标号为 $\exists$ 的交替图灵机所接受, 该交替图灵机在计算时标号为 $\exists$ 的状态和标号为 $\forall$ 的状态最多交替 $i-1$ 次。因此, $\Sigma_i^P \subseteq \bigcup_{c>0} \Sigma_i \mathbf{TIME}(n^c)$ 。反之, 若 $L \in \bigcup_{c>0} \Sigma_i \mathbf{TIME}(n^c)$ 被交替图灵机 A 所接受。设 x 为输入, 用库克-莱文归约可将 $A(x)$ 的计算归约到合取范式, 得到 $x \in L$ 的逻辑刻画。 $\square$

5.4 无限谱系假设

复杂性理论中常有如下形式的定理, “若多项式谱系不塌陷, 某性质成立”。此类相对结果基于所谓的无限谱系假设。

无限谱系假设. $\forall i. \mathbf{PH}_i \subsetneq \mathbf{PH}_{i+1}$ 。

显然, $\mathbf{PH}_i \subsetneq \mathbf{PH}_{i+1}$ 当仅当 $\Sigma_i^P \subsetneq \Sigma_{i+1}^P$ 当仅当 $\Pi_i^P \subsetneq \Pi_{i+1}^P$ 。图5.1是从无限谱系假设的视角看到的 $\mathbf{PH}$ 。本节讨论一些与多项式谱系相关的相对性结论。

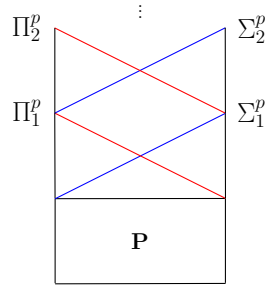


图 5.1 无限谱系假设

定理 5.7. 若 $\mathbf{NP} = \mathbf{P}$, 则 $\mathbf{PH} = \mathbf{P}$ 。

证明. 假定 $\Sigma_i^P = \mathbf{P}$, 就有 $\Sigma_{i+1}^P = \mathbf{NP}^{\Sigma_i^P} = \mathbf{NP}^{\mathbf{P}} = \mathbf{NP} = \mathbf{P}$ 。 $\square$

根据定理5.7, 无限谱系假设是一个比 $\mathbf{NP} \neq \mathbf{P}$ 还要强的假设。下述结果出现在文献 [54] 中。

定理 5.8. 对任一 $i \geq 1$, 若 $\Sigma_i^p \subseteq \Pi_i^p$ 或 $\Pi_i^p \subseteq \Sigma_i^p$, 则 $\mathbf{PH} = \mathbf{PH}_i$ 。

证明. $\Sigma_i^p \subseteq \Pi_i^p$ 当仅当 $\Pi_i^p \subseteq \Sigma_i^p$ 。故 $\Sigma_i^p \subseteq \Pi_i^p$, $\Pi_i^p \subseteq \Sigma_i^p$ 和 $\Sigma_i^p = \Pi_i^p$ 三者等价。假设 $\Sigma_k^p = \Pi_k^p$, 通过合并相同的量词就有 $\Sigma_{k+1}^p = \Sigma_k^p = \Pi_k^p = \Pi_{k+1}^p$ 。□

定理 5.9. 若有 $\mathbf{PH}$ -完备语言, 则 $\mathbf{PH}$ 必塌陷到某一层。

证明. 设有 $\mathbf{PH}$ -完备语言 L 。 L 必定在某一层 $\mathbf{PH}_i$ 。那么 $\mathbf{PH}$ 可卡普归约到 L , 即 $\mathbf{PH} = \mathbf{PH}_i$ 。□

定理 5.10. 若 $\mathbf{PH} = \mathbf{PSPACE}$, $\mathbf{PH}$ 必塌陷。

证明. 若 $\mathbf{PH} = \mathbf{PSPACE}$, $\mathbf{TQBF}$ 必定是 $\mathbf{PH}$ -完备的。用定理5.9即得本定理。□

下面的结果是卡普和立普顿证明的 [28]。

定理 5.11 (卡普-立普顿定理). 若 $\mathbf{NP} \subseteq \mathbf{P}_{/\text{poly}}$, 则 $\mathbf{PH} = \mathbf{PH}_1$ 。

证明. 假设 $\mathbf{NP} \subseteq \mathbf{P}_{/\text{poly}}$, 欲证 $\mathbf{PH} = \Sigma_1^p$ 。根据定理5.8, 只需证 $\Pi_2^p \subseteq \Sigma_2^p$ 。从假设 $\mathbf{NP} \subseteq \mathbf{P}_{/\text{poly}}$ 知, $\mathbf{SAT}$ 由一多项式大小的电路族 $\{C_n\}_{n \in \mathbf{N}}$ 接受。给定公式 $\psi = \forall u \in \{0, 1\}^m. \exists v \in \{0, 1\}^m. \varphi(u, v)$, 一定存在多项式时间图灵机, 对任意输入 u , 输出一个和 $\varphi(u, v)$ 等价的合取范式 $\varphi'(u, v)$ 。 $\varphi'(u, v) \in \mathbf{SAT}$ 可由一个多项式大小的电路 $C' \in \{C_n\}_{n \in \mathbf{N}}$ 判定。利用自归约, 可以从 C' 在多项式时间内构造出一个大小被一多项式 q 界定的电路 C 使得 $C(u) = v$ 当仅当 $\varphi'(u, v)$ 可满足。上述过程将公式 $\forall u \in \{0, 1\}^m. \exists v \in \{0, 1\}^m. \varphi(u, v) = 1$ 归约到了公式

$$\exists C \in \{0, 1\}^{q(|\psi|)}. \forall u \in \{0, 1\}^m. C \text{ 是电路的编码, 且 } \varphi'(u, C(u)) = 1.$$

从逻辑刻画的角度看, 我们将一个 Π_2^p 中的问题卡普归约到了 Σ_2^p 中的一个问题。证毕。□

如果我们坚信无限谱系假设, 卡普-立普顿定理告诉我们 $\mathbf{NP} \not\subseteq \mathbf{P}_{/\text{poly}}$, 即 $\mathbf{NP}$ -完备问题不可能有多项式时间的非一致解。

第 6 章 随机计算

6.1 尾分布

6.2 概率图灵机

6.3 PP

6.4 BPP

6.5 ZPP

6.6 随机游走和 RL

第 7 章 扩张图与去随机

第 8 章 计数复杂性

第 9 章 交互证明系统

第 10 章 PCP 定理

参 考 文 献

- [1] S. Aaronson and A. Wigderson. Algebrization: A New Barrier in Complexity Theory. *ACM Transactions on Computation Theory*, 2009.
- [2] Abramsky, S. The Lazy Lambda Calculus. In D. Turner, editor, *Declarative Programming*, Addison-Wesley, 65-116, 1988.
- [3] Agrawal M., Kayal N. and Saxena N. PRIMES is in P, *Annals of Mathematics*, 160 (2): 781-793, 2004.
- [4] Babai L. Graph Isomorphism in Quasipolynomial Time, *STOC'16*, 684-697, 2016.
- [5] T. Baker, J. Gill, and R. Solovay. Relativizations of the $P=?NP$ question. *SIAM Journal of Computing*, 4(4):431-442, 1975.
- [6] Barendregt, H. *The Lambda Calculus: Its Syntax and Semantics*. North-Holland. 1994.
- [7] Berman, L. and Hartmanis, J. On isomorphisms and density of NP and other complete sets, *SIAM Journal on Computing*, 6 (2): 305-322, 1977.
- [8] M. Blum. A Machine-Independent Theory of the Complexity of Recursive Functions. *Journal of ACM*, 1967.
- [9] A. Borodin. Computational Complexity and the Existence of Complexity Gaps. *Journal of the ACM* 19(1):158-174, 1972.
- [10] Ashok Chandra, Dexter Kozen and Larry Stockmeyer. Alternation. *Journal of the ACM*, 28(1):114-133, 1981.
- [11] Church, A. An Unsolvable Problem of Elementary Number Theory. *American Journal of Mathematics*, 58:345-363, 1936.
- [12] S. Cook. The complexity of theorem proving procedures. In *Proc. 3rd Ann. ACM Symp. Theory of Computing*, pages 151-158. ACM, 1971.

-
- [13] S. Cook. A Hierarchy for Nondeterministic Time Complexity. *Journal of Computer and System Sciences*, 1973.
 - [14] S. Cook and P. Nguyen. *Logical Foundations of Proof Complexity*. CUP, 2010.
 - [15] Davis, M. *The Undecidable: Basic Papers on Undecidable Propositions, Unsolvability Problems and Computable Functions*. Raven Press, 1965.
 - [16] van Emde Boas, P. Machine Models and Simulations. In J. van Leeuwen, editor, *Handbook of Theoretical Computer Science: Algorithm and Complexity, volume A*, Elsevier, 65-116, 1990.
 - [17] G. Frandsen and P. Miltersen. Reviewing Bounds on the Circuit Size of the Hardest Functions. *Information Processing Letters*, 2005.
 - [18] M. R. Garey and S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*, W. H. Freeman Co., New York, NY, 1979.
 - [19] Gödel, K. Über Formal Unentscheidbare Sätze der Principia Mathematica und Verwandter Systeme. *Monatshefte für Mathematik und Verwandter Systeme I*, 38:173-198, 1931.
 - [20] J. Hartmanis and R. Stearns. On the Computational Complexity of Algorithms. *Transactions of AMS*, 117:285-306, 1965.
 - [21] F. Hennie and R. Stearns. Two-Tape Simulation of Multitape Turing Machines. *Journal of ACM*, 13:533-546, 1966.
 - [22] 洪家威. On similarity and duality of computation (I). *FOCS'80*, 348-359, 1980.
 - [23] 洪家威. On similarity and duality of computation (I). *Information and Control*, 62:109-128, 1984.
 - [24] Hopcroft, Paul and Valiant. On Time versus Space and Related Problems. *FOCS*, 1975.
 - [25] N. Immerman. Nondeterministic Space is Closed under Complementation. *SIAM Journal Computing*, 1988.
 - [26] S. Jukna. *Boolean Function Complexity: Advances and Frontiers*. Springer, 2011.
 - [27] R. Karp. Reducibility among combinatorial problems. In R. E. Miller and J. W. Thatcher, editors, *Complexity of Computer Computations*, pages 85-103. Plenum, 1972.
 - [28] R. Karp and R. Lipton. Turing Machines that Take Advice. *STOC*, 1980.

- [29] Khachijan, L. A polynomial algorithm for linear programming. Dokl. A.N. SSSR 244, 1093-1096, 1979.
- [30] Kleene, S. General Recursive Functions of Natural Numbers. *Mathematische Annalen*, 112:727-742, 1936.
- [31] Kleene, S. λ -Definability and Recursiveness. *Duke Mathematical Journal*, 2:340-353, 1936.
- [32] Kleene, S. Introduction to Metamathematics. Amsterdam, North-Holland, 1952.
- [33] Kleene, S. Origin of Recursive Function Theory. *Annals of the History of Computing*, 3:52-67, 1981.
- [34] Kleene, S. The Inconsistency of Certain Formal Logics. *Annals of Mathematics*, 36:630-636, 1935.
- [35] E. Kushilevitz and N. Nisan. Communication Complexity. Cambridge University Press, 1997.
- [36] L. Levin. Universal sequential search problems. PINFTRANS: Problems of Information Transmission (translated from Problemy Peredachi Informatsii), 9, 265-266, 1973.
- [37] O. Lupanov. The Synthesis of Contact Circuits. Dokl. Akad. Nauk SSSR (N.S.) 119:23-26, 1958.
- [38] J. Lutz. Almost Everywhere High Nonuniform Complexity. JCSS, 1992.
- [39] Mahaney, S. Sparse complete sets for NP: solution of a conjecture of Berman and Hartmanis. Journal of Computer and System Sciences, 25 (2): 130-143, 1982.
- [40] Markov, A. The Theory of Algorithms. *American Mathematical Society Translations, series 2*, 15:1-14, 1960.
- [41] Matiyasevich Y. Hilbert's Tenth Problem, MIT Press, Cambridge, Massachusetts, 1993.
- [42] Minsky, M. *Computation: Finite and Infinite Machines*. Prentice-Hall, 1960.
- [43] M. Nielsen and I. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, 2000.
- [44] R. O'Donnell. Analysis of Boolean Functions. CUP, 2014.
- [45] C. Papadimitriou. Games Against Nature. FOCS, 1983.
- [46] C. Papadimitriou. Computational Complexity. Addison-Wesley, 1994.

-
- [47] Post, E. Formal Reduction of the General Combinatorial Decision Problem. *American Journal of Mathematics*, 65:197-215, 1943.
 - [48] A. Razborov and S. Rudich. Natural proofs. *Journal of Computer and System Science*, 55(1):24-35, 1997. Preliminary version in STOC '94.
 - [49] Rogers, H. *Theory of Recursive Functions and Effective Computability*. MIT Press, 1987.
 - [50] Rudich Y. and Wigderson A. *Computational Complexity Theory*, American Mathematical Society, 2004.
 - [51] W. Savitch. Relationships between Nondeterministic and Deterministic Tape Complexities. *JCSS*, 177-192, 1970.
 - [52] C. Shannon. The Synthesis of Two-Terminal Switching Circuits. *Bell System Technical Journal*. 28:59-98, 1949.
 - [53] C. Shannon. Communication theory of secrecy systems. *Bell Sys. Tech. J.*, 28:656-715, 1949.
 - [54] Larry Stockmeyer and Albert Meyer. The Equivalence Problem for Regular Expressions with Squaring Requires Exponential Space. SWAT'72.
 - [55] L. Stockmeyer and A. Meyer. Word Problems Requiring Exponential Time. STOC, 1973.
 - [56] Seiferas, Fischer and Meyer. Separating Nondeterministic Time Complexity Classes. *Journal of ACM*, 1978.
 - [57] Shepherdson, J. and Sturgis, H. Computability and Recursive Functions. *Journal of Symbolic Logic*, 32:1-63, 1965.
 - [58] S. Schmitz. Complexity Hierarchy Beyond Elementary. *ACM Transactions on Computation Theory*, 2016.
 - [59] L. Stockmeyer. The Polynomial-Time Hierarchy. *Theoretical Computer Science*, 3:1-22, 1976.
 - [60] R. Szelepcsényi. The Method of Forcing for Nondeterministic Automata. *Bulletin of EATCS*, 1987.
 - [61] B. Trakhtenbrot. Turing Computations with Logarithmic Delay. *Algebra and Logic* 3(4):33-48, 1964. (in Russian)
 - [62] B. Trakhtenbrot. A survey of Russian approaches to perebor (brute-force search) algorithms. *Annals of the History of Computing*, 6(4):384-400, 1984.

- [63] Turing, A. On Computable Numbers, with an Application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, 42:230-265, 1936.
- [64] Turing, A. On Computable Numbers, with an Application to the Entscheidungsproblem: A Correction. *Proceedings of the London Mathematical Society*, 43:544-546, 1937.
- [65] Turing, A. Computability and λ -Definability. *Journal of Symbolic Logic*, 2:153-163, 1937.
- [66] C. Umans. The Minimum Equivalent DNF Problem and Shortest Implicants. JCSS, 597-611, 2001. Preliminary version in FOCS 1998.
- [67] Vadhan S. Pseudorandomness, 2012.
- [68] C. Wrathall. Complete Sets and the Polynomial-Time Hierarchy. *Theoretical Computer Science*. 3:23-33, 1976.
- [69] Zák. A Turing Machine Time Hierarchy. *Theoretical Computer Science*, 1983.